
Deep Orthogonal Decomposition for surrogate modelling of time dependent PDEs

written and submitted by DAWID KOTOWSKI

A thesis presented for the degree of
Master of Science

Supervisor: PROF. DR. MARIO OHLBERGER
Faculty of Mathematics and Computer Science
University Münster
Germany, DE
submitted on October 12, 2025

Contents

0.1	Notation	1
1	Introduction	3
1.1	Motivation	4
1.2	Literature review	4
1.3	Outline	5
1.4	Time Dependent PDEs	5
1.5	Neural networks	7
2	Theoretical Background	9
2.1	Linear Algebra	9
2.2	Parametric Partial Differential Equations	11
2.2.1	Sobolev Spaces	11
2.2.2	Reduced Basis Framework	12
2.3	Neural Networks	16
2.3.1	Network concatenations	18
2.3.2	Network parallelization	19
2.3.3	Standard Neural Networks	19
2.3.4	Approximation Theory	20
2.3.5	Statistical Learning	30
3	Formulation of the General Problem	33
3.1	Intrusive Reduced Order Models	36
3.1.1	Galerkin Proper Orthogonal Decomposition	37
3.1.2	Greedy Proper Orthogonal Decomposition	38
3.1.3	Non-linear Reduced Order Models	39
3.2	Data-driven Reduced Order Models	40
3.2.1	Proper Orthogonal Decomposition Deep Learning ROM	42
3.2.2	Deep Orthogonal Decomposition based ROMs	43
3.2.3	A CoLoRA inspired approach	46
4	Approximation Theory for Reduced Order Models	49
4.1	Error Decomposition Formulas	50
4.2	Upper Complexity Bounds	63
4.3	Comparison between existing Techniques	71
4.3.1	Comparison between POD-based and DOD-based ROMs	72
4.3.2	Literature Comparison to other data-driven ROMs	75

5	Numerical Experiments	79
5.1	Two Visual Presentations	82
5.1.1	Example 1: Wind from a Hole in a complex domain	82
5.1.2	Example 2: Wind Generation in a plain domain	86
5.2	An Error Decomposition Example	89
6	Conclusion	93
6.1	Review	93
6.2	Research	93

0.1 Notation

Let $\mathbb{N} := \{1, 2, \dots\}$, $\mathbb{N}_0 := \{0, 1, \dots\}$ and $\mathbb{N}_{\geq n} := \{n, n+1, \dots\}$ for $n \in \mathbb{N}$. Next the set $\mathbb{R}^+ := \{x \in \mathbb{R} \mid x \geq 0\}$ will represent all real numbers, which are non-negative. Furthermore, for any finite set A , we will write $|A|$ to denote its cardinality. However, if $A \subset \mathbb{R}^n$ measurable for some $n \in \mathbb{N}$, we will use $|A| := \lambda(A)$, i.e. the Lebesgue measure. If $x \in \mathbb{R}$, the gaussian ceiling function is given via $\lceil x \rceil := \min\{k \in \mathbb{Z} : k \geq x\}$.

If $d \in \mathbb{N}$ and $\|\cdot\|$ is a norm, then for $x \in \mathbb{R}^d$ and $r > 0$ we set $B_{r, \|\cdot\|}(x)$ as the open ball around x with radius r . Additionally, $|\cdot|$ denotes the regular euclidean norm on $\mathbb{R}^d$. We usually do not differ between vectors and scalars in ubiquitous notation.

The transpose of a matrix $A \in \mathbb{R}^{d_1 \times d_2}$ for some $d_1, d_2 \in \mathbb{N}$ is given by $A^T \in \mathbb{R}^{d_2 \times d_1}$. The identity matrix will be denoted with $\mathbb{I}_d := \text{diag}(1, \dots, 1) \in \mathbb{R}^{d \times d}$ for any $d \in \mathbb{N}$. Sometimes we abbreviate $\mathbb{I} = \mathbb{I}_d$, when the dimensionality is clear.

Let $(a_n)_{n \in \mathbb{N}}, (b_n)_{n \in \mathbb{N}} \in \mathbb{R}$ be some sequences, then we say $(a_n)_n$ has (*convergence of*) *order* n , if there is $c \in \mathbb{R}$ independent of n , s.t. $a_n = c \cdot n$ for all $n \geq N$ and some $N \in \mathbb{N}$. We then denote $a_n = \mathcal{O}(n)$. In a similar vein, we use $a_n \lesssim b_n$, to describe the situation of $a_n \leq C b_n$ for all $n \in \mathbb{N}$ and some constant $C > 0$.

When writing $a \ll b$ for $a, b \in \mathbb{R}$, the reader can infer, that $a < b$, but also, that the difference is "substantial". We do not provide a rigorous definition of this, even though some literature does on occasion.

Moreover, we define the *multi-index* notation as follows. For $\vec{n} = (n_1, \dots, n_d) \in \mathbb{N}_0^d$, we write $|\vec{n}| := n_1 + \dots + n_d$ and $\vec{n}! := n_1! \dots n_d!$. If $x \in \mathbb{R}^d$, then we set

$$x^{\vec{n}} := \prod_{i=1}^d x_i^{n_i}.$$

Partial Differential Equations For a function $f : X \rightarrow Y$ we call $\text{supp}(f) := \{x \in X \mid f(x) \neq 0\}$ as the *support* of f . Let $g : Y \rightarrow Z$ be another function, then $g \circ f : X \rightarrow Z$ is their *composition*. Consider the standard topology of $\mathbb{R}^d$. Then we define $\bar{A}$ for some $\emptyset \neq A \subset \mathbb{R}^d$ as the *closure* of A , ∂A its *boundary*, and $\mathring{A} := \bar{A} - \partial A$ the *interior*. We set $\text{diam}(A) := \sup_{x, y \in A} |x - y|$ as the diameter of A .

Let $\Omega \subset \mathbb{R}^d$ be open. For $n \in \mathbb{N}_0 \cup \{\infty\}$, we denote by $C^n(\Omega)$ the set of n -times continuously differentiable functions on Ω , where $C^0(\Omega)$ is the set of continuous functions.

For $f \in C^n(\Omega)$ and $\vec{n} \in \mathbb{N}_0^d$ with $|\vec{n}| \leq n$ we write

$$D^{\vec{n}} f := \frac{\partial^{|\vec{n}|} f}{\partial x_1^{\alpha_1} \partial x_2^{\alpha_2} \dots \partial x_d^{\alpha_d}}.$$

A function $f : \Omega \rightarrow \mathbb{R}^m$ for $\Omega \subset \mathbb{R}^d$ is called *Lipschitz continuous*, if there is a constant $L \geq 0$ such that

$$|f(x) - f(y)| \leq L|x - y| \quad \text{for all } x, y \in \Omega.$$

For $\Omega \in \mathbb{R}^d$ being a non-zero set, we further denote with

$$L^p(\Omega) := \left\{ f : \Omega \rightarrow \mathbb{R} \mid f \text{ is measurable, and } \|f\|_{L^p(\Omega)} < \infty \right\} / \sim_{\text{a.e.}}$$

the Lebesgue space and with L_{loc}^p the space of $L^p(\Omega)$ functions with local support. The measure

here is always Lebesgue, if not specified otherwise. Moreover, the norm is defined by

$$\|f\|_{L^p(\Omega)} = \begin{cases} \left(\int_{\Omega} |f(x)|^p d(x) \right)^{1/p} & \text{for } 1 \leq p < \infty, \\ \operatorname{ess\,sup}_{x \in \Omega} |f(x)| & \text{for } p = \infty. \end{cases}$$

Neural Networks Consider $d_1, d_2 \in \mathbb{N}$ and a matrix $A \in \mathbb{R}^{d_1 \times d_2}$, then the number of non-zero entries of A is given by

$$\|A\|_{\ell^0} := |\{(i, j) : A_{i,j} \neq 0\}|.$$

Let $B \in \mathbb{R}^{d_1 \times d_3}$ be some other matrix for $d_3 \in \mathbb{N}$. Then we write

$$\begin{bmatrix} A & B \end{bmatrix} \in \mathbb{R}^{d_1 \times (d_2 + d_3)} \quad \text{or} \quad \begin{bmatrix} A & | & B \end{bmatrix} \in \mathbb{R}^{d_1 \times (d_2 + d_3)},$$

for the stacking of matrices, where the latter notation is meant to symbolize a strong delineation between blocks. The same applies to vertical stacking, if $A \in \mathbb{R}^{d_1 \times d_2}$ and $B \in \mathbb{R}^{d_3 \times d_2}$. As a special case, this applies to vectors as well.

Chapter 1

Introduction

Recent advances in artificial intelligence and specifically neural networks have sparked interest in approximation theory. This field nowadays includes investigations of the complexity of neural networks, that approximate functions with certain regularity. In itself, approximation theory existed in the context of partial differential equations long before the invention and popularization of neural networks, and has dealt with questions arising from discretization and interpolation approximations. After all, numerical analysis emerged as a field of mathematics to primarily deal with those analytical problems. It was intended to answer questions like "How can one find a solution to a partial differential equation?", "In which space does the approximative solution from a numerical solver live?" and finally "For a given finite-dimensional space, how well can *a solution* be approximated?", which is an example of a typical approximation theoretic problem. This thesis will be handling the concept of that last question in a slightly varied form; instead of asking how well *one* solution could be approximated for a choice of a finite-dimensional sub-space of the solution space, the question of interest for us will be how one could approximate *a group* of solutions for such a choice. This *grouping* of partial differential equations is called a parametric partial differential equation, if there are parameters that directly influence the equations and hence its solutions. In such a case, the meta-solution for which we actually search an approximation, is not $\text{PDE} \mapsto u$, but more accurately

$$\mathcal{G}_\infty : \Theta \rightarrow W; \quad \mu \mapsto u(\mu),$$

where Θ denotes some parameter set and W is the (time-sensitive) solution space of the respective parametric partial differential equation. The infinity here denotes the general setting of "a solution in a potentially infinite-dimensional solution space". The first direct approximation, which can be done, uses a finite-dimensional spatial solution space. Otherwise, it would be generally difficult to portray a non-analytical problem on a computer, and hence to use a numerical algorithm in order to find a solution. From here, we can infer a map

$$\mathcal{G}_h : \Theta \times [0, T] \rightarrow \mathcal{H}_h; \quad (\mu, t) \mapsto u_h(\mu, t),$$

where $\mathcal{H}_h$ is some N_h -dimensional subspace for $N_h \in \mathbb{N}$ and its related map $\mathcal{G} : \Theta \times [0, T] \rightarrow \mathbb{R}^{N_h}; (\mu, t) \mapsto u_h^{\mu, t}$ mapping to the coefficient representation. Note, that N_h means to represent some spatial discretization and time discretization, for example using a piecewise constant functions. In general, we will assume, that this basis will be of finite size $N_t \in \mathbb{N}$. In fact, such a map always exists in the case, that there is some way of "solving" the partial differential equation for every parameter in a numerical way; it is simply the corresponding numerical algorithm, for example one using Finite Differences or the Finite Element Method. Problematic can however be, that this solution can deviate substantially from the actually interesting strong solution mapped by $\mathcal{G}_\infty$.

Generally, such a *parameter-to-solution map* $\mathcal{G}$ is expensive to compute, because the associated Full Order Model (FOM) has a very high dimension N_h . To resolve this, we might finally assert the main starting point of the thesis: Reduced Order Modeling.

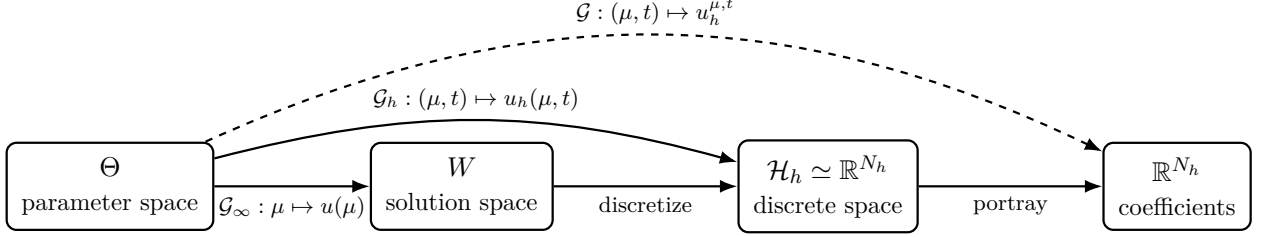


Figure 1.1: From parameters to solutions.

In this thesis, we will lay the groundwork for the use of approximation theory for data-driven Reduced Order Modelling. Furthermore, we propose a novel Reduced Order Model heavily inspired by the Deep Orthogonal Decomposition, which has been proposed for the stationary case in (Franco et al., 2024), and combine it with the autoencoder-based latent architecture of the DL-ROM introduced in the paper (Fresca et al., 2020) to attain the DOD-DL-ROM. As such, this model will have a lot of similarity to the POD-DL-ROM given by (Fresca & Manzoni, 2022). Thus, we will seek out comparisons in terms of complexity in relation to accuracy. We will find, that the adaptive nature of the Deep Orthogonal Decomposition will allow us to circumvent the Kolmogorov N -width barrier for certain problems, where a hybrid nature of the Kolmogorov N -width decay can be assumed. This will be shown in theory and, with some room for statistical misidentification, by numerical implementation. Furthermore, we outline another novel idea adapting the Continuous Low Rank Adaptation neural network architecture of (Berman & Peherstorfer, 2024). However, there will not be any numerical or approximation theoretic results presented for this approach.

1.1 Motivation

In a lot of applications the precision of Full Order Models can reach machine precision for large enough N_h . However, these models uniformly suffer from the computational cost, which grows significantly quicker, than linear. Indeed, if we consider the solution to a classical linear problem arising from a discretization of some parabolic problem, we get computational cost of order $\mathcal{O}(N_t N_h^\alpha)$ for $\alpha \in [1, 2]$ depending on the sparsity qualities of the associated matrices. Results like this can be found in any classical numerics literature such as (Quarteroni et al., 2015). This is a large problem in cases, where we face *many query* settings, such as research situations for aerodynamics engineering, medical identification, weather prediction and many more. Furthermore, describing latent dynamics by using lower dimensions gives a nice new view onto the true nature of mathematical PDE, as it allows to focus attention on regions of interest.

1.2 Literature review

Over the past decades the field of Reduced Order Modeling has undergone a noticeable progress, shifting from classical projection-based methods to modern data-driven architectures. The earliest intrusive ROMs originated from the *Proper Orthogonal Decomposition* (POD) framework, introduced in turbulence analysis by (Lumley, 1967) and (Sirovich, 1987), and later consolidated in the POD-Galerkin formulation for parametrized PDEs by (Holmes et al., 1996) and (Kunisch & Volkwein, 2001).

Subsequent developments sought to address the computational bottlenecks of nonlinear problems through hyper-reduction techniques such as the *Empirical Interpolation Method* (EIM) (Barrault et al., 2004) and its discrete counterpart *DEIM* (Chaturantabut & Sorensen, 2010). Alternative intrusive formulations include the least-squares Petrov-Galerkin projection (Carlberg et al., 2011) and the GNAT method (Carlberg et al., 2013), which extended ROMs to highly nonlinear and turbulent regimes.

Simultaneously, a non-intrusive and data-driven perspective emerged, aiming to approximate the parameter-to-solution map directly from snapshot data. Early methods such as *Dynamic Mode Decomposition* (DMD) (Schmid, 2010) and its interpretation via the Koopman operator (Rowley et al., 2009) established a foundation for learning reduced dynamics without explicit projection. Further, regression-based frameworks such as *Operator Inference* (Peherstorfer et al., 2016) and *POD with interpolation* (PODI) (Barrault et al., 2004) demonstrated the feasibility of reconstructing reduced operators from data.

The integration of deep learning with model reduction around 2020 led to the first *Deep Learning Reduced Order Models* (DL-ROMs) (Fresca et al., 2020), which combined convolutional autoencoders and neural regressors to emulate low-dimensional dynamics. Their refinement, the *POD-DL-ROM* (Brivio et al., 2023; Fresca & Manzoni, 2022), fused the interpretability of POD with the flexibility of deep networks, inspiring numerous follow-ups.

1.3 Outline

In order to briefly summarize the structure of this thesis, we want to point out the line of thought here, and remark the important results from each chapter.

The following two sections put the development and advances of partial differential equations and neural networks in a historical context and introduce some meaningful results and classification thereof. In Chapter (2) we introduce the reduced basis methods and auxiliary information, which will be used to explain the Proper Orthogonal Decomposition and Deep Orthogonal Decomposition as linear projectors in a theoretical setting. Then we discuss the needed approximation theory for the complexity bounds, we will achieve for the POD-DL-ROM and DOD-DL-ROM.

In Chapter (3), we introduce the actual problem formulations and all necessary assumptions in a rigorous notation, as well as some legacy Reduced Order Models. Finally, we will define the proposed DOD-DL-ROM and a related, simpler model that can be treated separately, but also acts as a benchmark in testing. The main results of this thesis, however, are presented in Chapter (4), where we prove an upper and lower bounds for the relative errors of any POD-based and DOD-based Reduced Order Model, dictated by their intrinsic projection qualities, and thus emergent mismatches. Afterwards, we use this to set up upper bounds for the complexities of the three proposed ROMs: the POD-DL-ROM, sourced from (Brivio et al., 2023), the DOD-DL-ROM and the DOD+DFNN.

In Chapter (5), we compare those three algorithms in a numerical environment for one pathological, one positive and one analytic example.

At last, we will conclude the results in Chapter (6)

1.4 Time Dependent PDEs

The study and analysis of partial differential equations (PDEs) began with the works of Euler, d'Alembert, Lagrange and Laplace (see (Brezis & Browder, 1998) for this introduction). The foundation was a set of simple problems, such as the vibrating string in one dimension as modelled

by d'Alembert (1752)

$$\frac{\partial^2}{\partial t^2}u(x, t) = \frac{\partial^2}{\partial x^2}u(x, t), \quad \text{for } u \in C^2(\mathbb{R} \times \mathbb{R}^+), x \in \mathbb{R} \text{ and } t \in \mathbb{R}^+,$$

the extension to three dimensions by Euler (1759) and Bernoulli (1762), the Laplace equation for gravitational fields (1780)

$$\Delta u(x) = 0, \quad \text{for } u \in C^2(\mathbb{R}^3) \text{ and } x \in \mathbb{R}^3,$$

and finally the heat equation, as introduced by Fourier (1810 - 1822)

$$\frac{\partial}{\partial t}u(x, t) = \Delta u(x, t), \quad \text{for } u \in C^2(\mathbb{R}^3 \times \mathbb{R}^+), x \in \mathbb{R}^3 \text{ and } t \in \mathbb{R}^+.$$

There have been simultaneous developments with regard to the calculation methods. The method of separation of variables was given by Euler and d'Alembert for the study of spherical harmonics and Fourier introduced similar ideas for the heat equation. For the Laplace equation, the Green functions were introduced in 1835. Green also discovered an important principle for solvability of the (later so called) Dirichlet problem

$$\begin{cases} \Delta u(x) = 0, & \text{for } x \in \Omega \subset \mathbb{R}^3, \\ u(x) = g(x), & \text{for } x \in \partial\Omega, \end{cases}$$

for some function $g \in C^0(\mathbb{R}^3)$. It revolved around minimizing an energy functional $E : C^2(\mathbb{R}^3) \rightarrow \mathbb{R}^+$, thus founding the variational principle.

Though these were first attempts at solving such problems, the idea of Green in particular led to some analytical problems, by the lack of mathematical rigor. Together with the contribution of Cauchy on the *initial value* problem, this laid the groundwork for the field of *Existence Theory*, which tries to establish existence results for PDEs.

Notable here are the alternating method of Schwarz (1870) using the idea of domain decomposition and especially Neumanns method of integral equations developed in 1877, which would go on to be the starting point for the weak form and subsequently the Sobolev spaces. These were a necessity after B. Levi showed in 1906, that a general minimizing sequence for the Dirichlet integral was a Cauchy sequence and would therefore converge in the right space, which was then given due to the Sobolev embedding theorems.

To tackle the problem arising from "exotic" examples and the complicated nature of some PDEs, Hadamard proposed a classification of the most common linear PDEs using its characteristic polynomial, which is given by replacing all partial derivatives $\partial/\partial x_j$ by the algebraic variable ξ_j . In that case the PDE is called

1. *elliptic*, if after a change of variables the polynomial can be written as $\xi_1^2 + \xi_2^2 + \dots + \xi_n^2$,
2. *parabolic*, if after a change of variables the polynomial can be written as $\xi_1 + \xi_2^2 + \dots + \xi_n^2$,
3. *hyperbolic*, if after a change of variables the polynomial can be written as $\xi_1^2 - (\xi_2^2 + \dots + \xi_n^2)$.

These analytical results can be expanded a lot more. Most interestingly, in terms of this paper, are the Sobolev spaces. They, and the closely connected Bochner spaces, are the relevant solution spaces to the limits of all, in some sense, consistent numerical schemes, since the solutions are at least embedded in them. This follows quite natural, as any numerical scheme is basically bounded by the necessity of being run on a computer. Thus, it defines its approximations on a finite dimensional space, and these can most often be nicely embedded in Bochner or Sobolev spaces.

1.5 Neural networks

In its essence, the origin of the artificial neural network is the paper (McCulloch & Pitts, 1943), where the authors found a consistent logical calculation description of biological neurons and proposed a mathematical construction. This, in a sense, became the leading idea for most of the later theory, since it served as inspiration for the *perceptron*, the term coined by Rosenblatt in (Rosenblatt, 1958). In today's notation, this describes a neural layer with very limited range and a specific activation function, i.e.

$$f : \mathbb{R}^n \rightarrow \{0, 1\}; \quad f(x) = h(w \cdot x + b),$$

where h is the Heaviside function, returning 1 for any positive value and 0 otherwise and $w, b \in \mathbb{R}^n$ being trainable constant vectors for $n \in \mathbb{N}$. This inspired the later recognized definition of a Multi-Layered Perceptron (MLP), which is another name for the (Deep) Feedforward Neural Network. It is interpreted as an composed evaluation of some layers realizing

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^m; \quad f(x) = \rho(Wx + b),$$

for $W \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$ being trainable features for $n, m \in \mathbb{N}$ and ρ some (most often the ReLU) activation function, with the last convolution skipping the activation function to retain full range. In the common nomenclature, a Neural Network is called *deep*, if it has more than two layers (thus at least two hidden layers).

In the following decades, the first steps towards universal approximation theory were made by Cybenko (Cybenko, 1989). He has build on the works of others to prove the fact, that any feedforward neural network with one hidden layer can approximate any continuous function. Following this Hornik proved in (Hornik, 1991), that any Neural Network with at least two hidden layers can approximate any L^p function on a compact domain arbitrarily well.

After this, a lot of subsequent architectures have been proposed, such as Convolutional Autoencoders for image classification and the method of *backpropagation*, marking the first major breakthrough in computational regard (LeCun et al., 1989). The following research included a wide range of different architecture propositions and some notable adaptations for general function fitting purposes, which finally culminated in the work of Yarotsky (Yarotsky, 2017) and Gühring (Gühring et al., 2019). These two results not only show rigorous mathematical framework for Neural Networks; they also include complexity bounds for a fixed error tolerance for arbitrary Sobolev functions on Lipschitz domains. We will not go into further detail here, since this will be done extensively in the upcoming chapter.

Chapter 2

Theoretical Background

2.1 Linear Algebra

For this section, we will specifically set up the euclidean (Hilbert) space $(\mathbb{R}^{N_h}, \langle \cdot, \cdot \rangle_{\mathbb{G}})$, with the inner product

$$\langle u, v \rangle_{\mathbb{G}} := u^T \mathbb{G} v,$$

for some symmetric positive definite $\mathbb{G} \in \mathbb{R}^{N_h \times N_h}$ for each $u, v \in \mathbb{R}^{N_h}$ and $\|\cdot\|$ its induced norm. As preparation for the upcoming Bauer Fike Theorem, we further want to define the matrix norm for the matrix space $\mathbb{R}^{N_h \times N}$, where $N \leq N_h$, as follows

$$\|A\|_{\text{op}} := \sqrt{\lambda_{\max}(A^T \mathbb{G} A)}.$$

Here, $\lambda_{\max}(A^T \mathbb{G} A)$ is the maximal spectral radius of $A^T \mathbb{G} A$, which exists because $A^T \mathbb{G} A$ is symmetric and positive semi definite and hence subject to the spectral theorem. This norm satisfies the wanted property mentioned in the original paper (Bauer & Fike, 1960, p. 137-141). Namely, for all $x \in \mathbb{R}^N$, we have

$$\|Ax\|^2 = x^T A^T \mathbb{G} A x \leq \lambda_{\max}(A^T \mathbb{G} A) x^T x \leq \|A\|_{\text{op}}^2 |x|^2.$$

Theorem 2.1 (Bauer-Fike Theorem). *Let $K \in \mathbb{R}^{N_h \times N_h}$ be some diagonalizable matrices with σ_k eigenvalues and $V = (\xi_1, \dots, \xi_N) \in \mathbb{R}^{N_h \times N_h}$ be the stacked eigenvector matrix of K . Let $\delta K \in \mathbb{R}^{N_h \times N_h}$ be some symmetric matrix. Then if λ_k are the eigenvalues of $K + \delta K$, we have*

$$\min_k |\lambda_k - \sigma_k| \leq \kappa(V) \|(K + \delta K) - K\|_{\text{op}},$$

where $\kappa(V) = \|V\|_{\text{op}} \|V^{-1}\|_{\text{op}}$.

Proof. See (Bauer & Fike, 1960, p. 137-141). □

Furthermore, we will often discuss the notion of orthonormality with regard to a matrix. This can be understood with the following definition:

Definition 2.2 (Matrix Orthonormality). Let $\mathbb{G} \in \mathbb{R}^{N_h \times N_h}$ be a symmetric positive definite matrix for some $N_h \in \mathbb{N}$. We then say that any matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N}$ is $\mathbb{G}$ -orthonormal for some $N \leq N_h$, if

$$\mathbb{I} = \mathbb{A}^T \mathbb{G} \mathbb{A}.$$

Remark 2.3. If some matrix $\mathbb{V} \in \mathbb{R}^{N_h \times N}$ can be written as $\mathbb{V} = \mathbb{A}\tilde{\mathbb{V}}$, where $\tilde{\mathbb{V}} \in \mathbb{R}^{N_A \times N}$ is any matrix and $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is a $\mathbb{G}$ -orthonormal matrix, then $\mathbb{V}$ is $\mathbb{G}$ -orthonormal if and only if $\tilde{\mathbb{V}}$ is orthonormal in the euclidean sense. In fact,

$$\mathbb{V}^T \mathbb{G} \mathbb{V} = \tilde{\mathbb{V}}^T \mathbb{A}^T \mathbb{G} \mathbb{A} \tilde{\mathbb{V}} = \tilde{\mathbb{V}}^T \tilde{\mathbb{V}}.$$

Definition 2.4 (Orthogonal Projector). Let $P : V \rightarrow V$ be an operator, where V is a Hilbert space with inner product $\langle \cdot, \cdot \rangle_V$. Then it is called an *orthogonal projector* onto $W \subset V$, if

1. $P^2 = P$,
2. $\text{Im}(P) = W$ and
3. $\langle Px, y \rangle_V = \langle x, Py \rangle_V$ for all $x, y \in V$.

Lemma 2.5. Let $(\mathbb{R}^{N_h}, \langle \cdot, \cdot \rangle_{\mathbb{G}})$ be the Hilbert space from above. If $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is a $\mathbb{G}$ -orthonormal matrix, then

$$P := \mathbb{A} \mathbb{A}^T \mathbb{G}$$

is a orthogonal projector on $\text{span}(\mathbb{A})$ with regard to the $\mathbb{G}$ induced inner product. We refer to this as P being the $\mathbb{G}$ -orthonormal projector on $\text{span}(\mathbb{A})$.

Proof. It suffices to simply check the properties of Definition (2.4); the image statement holds, since $\mathbb{A} \mathbb{A}^T \mathbb{G} = \mathbb{A}(\mathbb{A}^T \mathbb{G})$. Next, the idempotency can be shown via

$$P^2 = (\mathbb{A} \mathbb{A}^T \mathbb{G})(\mathbb{A} \mathbb{A}^T \mathbb{G}) = \mathbb{A}(\mathbb{A}^T \mathbb{G} \mathbb{A}) \mathbb{A}^T \mathbb{G} = \mathbb{A} \mathbb{A}^T \mathbb{G} = P.$$

Finally for $u, v \in \mathbb{R}^{N_h}$ arbitrary, we have

$$\begin{aligned} \langle Pu, v \rangle_{\mathbb{G}} &= (Pu)^T \mathbb{G} v \\ &= u^T P^T \mathbb{G} v \\ &= u^T (\mathbb{A} \mathbb{A}^T \mathbb{G})^T \mathbb{G} v \\ &= u^T \mathbb{G} \mathbb{A} \mathbb{A}^T \mathbb{G} v \\ &= u^T \mathbb{G} P v = \langle u, P v \rangle_{\mathbb{G}}. \end{aligned}$$

□

Lemma 2.6. Let $(\mathbb{R}^{N_h}, \langle \cdot, \cdot \rangle_{\mathbb{G}})$ be the Hilbert space from above. If now $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is a $\mathbb{G}$ -orthonormal matrix, then

$$\|\mathbb{A}\|_{\text{op}} = 1.$$

Proof. This is directly clear, since $\|\mathbb{A}\|_{\text{op}} = \sqrt{\lambda_{\max}(\mathbb{A}^T \mathbb{G} \mathbb{A})} = \sqrt{\lambda_{\max}(\mathbb{I})} = 1$. □

These preliminaries will play an important role, since we will use such a gramian $\mathbb{G}$ of an a priori chosen basis under a discretization of a PDE to set up a linear projector. This projector will attain any coefficient representation onto the solution manifold in consideration of that gramian $\mathbb{G}$. In fact, any projector

$$\mathbb{V} \mathbb{V}^T \mathbb{G} : \mathbb{R}^{N_h} \rightarrow \mathbb{R}^N$$

for a $\mathbb{G}$ -orthonormal matrix $\mathbb{V} \in \mathbb{R}^{N_h \times N}$ and some $N \leq N_h$ can be seen as a special case for the differential geometric framework of (nonlinear) projectors on manifolds. This is discussed in more detail in (Buchfink et al., 2023).

2.2 Parametric Partial Differential Equations

In this section, we will prepare for the upcoming results by discussing statements in direct relation with parametric partial differential equation solutions and their linear embedding. However, we will refrain from introducing any discretization methods or mathematical formulation beyond the needed space definition, since this will not be the focus of this work.

2.2.1 Sobolev Spaces

For our work with partial differential equations, and subsequent error approximations, we introduce Sobolev spaces. These are generalized differentiable spaces, where a weak derivative is given, as an analog to the classic derivative in the $C^n(\Omega)$ space. They are employed in order to exploit their compact embedding in the Hölder spaces, which broadly are generalizations of $C^n(\Omega)$. We will not go in much detail about the theory of Sobolev spaces, since we will focus on finite dimensional subspaces, but some ground truth must be discussed.

For this subsection we primary use (Alt, 2016), and for the first upcoming definition (Dacorogna, 2004).

Definition 2.7 (Lipschitz Domain). Let $d \in \mathbb{N}$, and $\Omega \subset \mathbb{R}^d$. Then Ω is called a *Lipschitz domain* if for every $x \in \partial\Omega$ and some $r > 0$, there is $U := B_{r,|\cdot|}(x) \subset \mathbb{R}^d$ and an isomorphic map $H : Q \rightarrow B_{r,|\cdot|}(x)$ with $Q := \{x \in \mathbb{R}^d \mid \|x\|_{\ell^1} < d\}$ such that

$$H \in C^k(\overline{Q}), \quad H^{-1} \in C^k(\overline{U}), \quad H(Q_+) = U \cap \Omega, \quad H(Q_0) = U \cap \partial\Omega.$$

Here, $Q_+ := \{x \in Q \mid x_d > 0\}$ and similarly $Q_0 := \{x \in Q \mid x_d = 0\}$, and H is only in $C^{0,1}$.

From now on, let $d \in \mathbb{N}$ and, if not given otherwise, $\Omega \subset \mathbb{R}^d$ be a Lipschitz domain.

Definition 2.8 (Weak Derivative). Let $\alpha \in \mathbb{N}_0^d$ be a multiindex. A function $u \in L_{\text{loc}}^1(\Omega)$ is said to have an α -th weak derivative, if there is a function $v_\alpha \in L_{\text{loc}}^1(\Omega)$ such that

$$\int_{\Omega} u D^\alpha \varphi = (-1)^{|\alpha|} \int_{\Omega} v_\alpha \varphi \quad \forall \varphi \in C_0^\infty(\Omega).$$

We then write $v_\alpha = D^\alpha u$.

Definition 2.9 (Sobolev Space). Let $n \in \mathbb{N}_0$ and $1 \leq p \leq \infty$. Then we define the *Sobolev space* as

$$W^{n,p}(\Omega) := \{f \in L^p(\Omega) \mid \|f\|_{W^{n,p}(\Omega)} < \infty\},$$

where the norm on this space is given via

$$\|f\|_{W^{n,p}(\Omega)} := \begin{cases} \left(\sum_{|\alpha| \leq n} \|D^\alpha f\|_{L^p(\Omega)}^p \right)^{1/p}, & \text{if } p < \infty \\ \max_{|\alpha| \leq n} \|D^\alpha f\|_{L^\infty(\Omega)}, & \text{if } p = \infty, \end{cases}$$

for any $f \in W^{n,p}(\Omega)$.

It is possible to extend this definition for $0 < n < 1$, namely the *Sobolev-Slobodeckij spaces*. In the analysis of partial differential equations these are used to characterize the regularity of Sobolev functions, which are restricted to the boundary $\partial\Omega$. This restricted function is then called a trace operator. One of the statements of relevance for such fractal Sobolev spaces is the following.

Let $\Omega \subset \mathbb{R}^d$ be a regular open set (for example, if Ω is a Lipschitz domain), and $u \in W^{1,p}(\Omega)$. Then the trace operator $u \mapsto u|_{\partial\Omega}$ is surjective from $W^{1,p}(\Omega)$ onto $W^{1-(1/p),p}(\partial\Omega)$ and

$$\|u|_{\partial\Omega}\|_{W^{1-(1/p),p}(\partial\Omega)} \leq C\|u\|_{W^{1,p}(\Omega)},$$

for some $C > 0$ independent of p and u , see (Brezis, 2011, p. 315). We make sense of this space via the next definition.

Definition 2.10 (Sobolev-Slobodeckij Space). Let $0 < s < 1$ and $1 \leq p \leq \infty$. Then we define the *Sobolev-Slobodeckij space* as

$$W^{s,p}(\Omega) := \{f \in L^p(\Omega) \mid \|f\|_{W^{s,p}(\Omega)} < \infty\},$$

where the norm on this space is given via

$$\|f\|_{W^{s,p}(\Omega)} := \begin{cases} \left(\|f\|_{L^p(\Omega)}^p + \int_{\Omega} \int_{\Omega} \left(\frac{|f(x) - f(y)|}{|x - y|^{s+d/p}} \right)^p dx dy \right)^{1/p}, & \text{if } p < \infty \\ \max \left\{ \|f\|_{L^\infty(\Omega)}, \operatorname{ess\,sup}_{x,y \in \Omega} \frac{|f(x) - f(y)|}{|x - y|^s} \right\}, & \text{if } p = \infty, \end{cases}$$

for any $f \in W^{s,p}(\Omega)$.

Furthermore, a very important result is the following theorem, translating Lipschitz continuity into Sobolev-regularity. It will allow us to employ the later introduced approximation theory for neural networks from the sole position of Lipschitz continuity.

Theorem 2.11. *Let $f : \Omega \rightarrow \mathbb{R}$ be a bounded and L -Lipschitz map, then $f \in W^{1,\infty}(\Omega)$, the weak gradient of f agrees almost everywhere with the classical gradient, and*

$$\|Df(x)\|_{L^\infty(\Omega)} \leq L.$$

Proof. See (Heinonen, 2005, Theorem 4.1) and (Heinonen, 2005, Remark 4.2). $\square$

2.2.2 Reduced Basis Framework

For this subsection we define $u(\mu_j, t_k) \in \mathbb{R}^{N_h}$ for $(\mu_j)_{j=1}^{N_s} =: \mathcal{P} \subset \Theta$ and $(t_k)_{k=1}^{N_t} =: \mathcal{T} \subset [0, T]$ to be some vectors. They will later represent the solutions to a parametric PDE for a determined parameter set and time point in a given discrete subspace of the coefficient solution manifold. Here Θ represents an arbitrary parameter space, $T > 0$ some arbitrary time point, and $N_s, N_t \in \mathbb{N}$ are the parameter and time sample size, respectively. In context of the aforementioned coefficient solution manifold, the inner product on $\mathbb{R}^{N_h}$ is defined by using a symmetric positive definite matrix $\mathbb{G} \in \mathbb{R}^{N_h \times N_h}$ via

$$\langle u, v \rangle_{\mathbb{G}} := u^T \mathbb{G} v, \quad \text{for all } u, v \in \mathbb{R}^{N_h},$$

and the corresponding norm $\|\cdot\| := \langle \cdot, \cdot \rangle_{\mathbb{G}}$.

It is assumed, that we sample the parameters independently and identically distributed, and the time points equidistantly determined from their respective spaces, and that

$$\sup_{(\mu,t) \in \Theta \times [0,T]} \|u(\mu, t)\| \leq M < \infty, \quad \inf_{(\mu,t) \in \Theta \times [0,T]} \|u(\mu, t)\| \geq m > 0,$$

for all $\mu \in \Theta$ and $t \in [0, T]$ in an essential sense. Let also

$$N_{\text{data}} := N_s \cdot N_t,$$

be the total sample size.

Notice, that it is possible to switch to a lower dimensional euclidean norm, if the projector in consideration is an orthogonal projector on the span of a $\mathbb{G}$ -orthonormal matrix with specific identity. Namely, this is true, if the matrix $\mathbb{V} \in \mathbb{R}^{N_h \times N}$ for some $N \leq N_h$ is $\mathbb{G}$ -orthonormal and can be expressed as $\mathbb{V} = \mathbb{A}\tilde{\mathbb{V}}$, where $\mathbb{A} \in \mathbb{R}^{N_A \times N}$ for $N \leq N_A \leq N_h$ is also $\mathbb{G}$ -orthonormal. This is quantified by the following result.

Proposition 2.12. Let $\mathbb{V} \in \mathbb{R}^{N_h \times N}$ for some $N \leq N_h$ be $\mathbb{G}$ -orthonormal such that

$$\mathbb{V} = \mathbb{A}\tilde{\mathbb{V}},$$

where $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is also $\mathbb{G}$ -orthonormal and $\tilde{\mathbb{V}} \in \mathbb{R}^{N_A \times N}$ for $N \leq N_A \leq N_h$. Then we have

$$\|u - \mathbb{V}\mathbb{V}^T \mathbb{G}u\|^2 = c_{\mathbb{A}} + |u_{\text{pre-red}} - \tilde{\mathbb{V}}\tilde{\mathbb{V}}^T u_{\text{pre-red}}|^2,$$

for any $u \in \mathbb{R}^{N_h}$ with $c_{\mathbb{A}} > 0$ being a constant only depending on $\mathbb{A}$ and defining $u_{\text{pre-red}} = \mathbb{A}^T \mathbb{G}u$.

Proof. By Lemma (2.5), we know that $\mathbb{A}\mathbb{A}^T \mathbb{G}$ is a $\mathbb{G}$ -orthonormal projector into the span of $\mathbb{A}$. Thus, its residual $u - \mathbb{A}\mathbb{A}^T \mathbb{G}u$ must be $\mathbb{G}$ -orthogonal to all columns of $\mathbb{A}$, since we can decompose $u = u_{\perp} + u_{\parallel}$, where $u_{\parallel} \in \text{span}(\mathbb{A})$ and $u_{\perp} \in \mathbb{R}^{N_h} - \text{span}(\mathbb{A})$. Then the Pythagoras decomposition gives

$$\begin{aligned} \|u - \mathbb{V}\mathbb{V}^T \mathbb{G}u\|^2 &= \|u - \mathbb{A}\tilde{\mathbb{V}}\tilde{\mathbb{V}}^T \mathbb{A}^T \mathbb{G}u\|^2 \\ &= \|u - \mathbb{A}\mathbb{A}^T \mathbb{G}u\|^2 + \|\mathbb{A}\mathbb{A}^T \mathbb{G}u - \mathbb{A}\tilde{\mathbb{V}}\tilde{\mathbb{V}}^T \mathbb{A}^T \mathbb{G}u\|^2 \\ &= \|u - \mathbb{A}\mathbb{A}^T \mathbb{G}u\|^2 + |\mathbb{A}^T \mathbb{G}u - \tilde{\mathbb{V}}\tilde{\mathbb{V}}^T \mathbb{A}^T \mathbb{G}u|^2. \end{aligned}$$

The last step is justified by showing that for any $u \in \mathbb{R}^{N_A}$

$$\|\mathbb{A}u\| = \langle \mathbb{A}u, \mathbb{A}u \rangle_{\mathbb{G}} = (\mathbb{A}u)^T \mathbb{G} \mathbb{A}u = u^T \mathbb{A}^T \mathbb{G} \mathbb{A}u = u^T u = |u|.$$

Now setting $u_{\text{pre-red}} := \mathbb{A}^T \mathbb{G}u$ gives us the wanted statement. $\square$

This proposition will be the core idea to assert closeness with regard to the representations of the solution and simultaneously provide a computationally feasible option with the euclidean N_A -dimensional norm. The term $c_{\mathbb{A}}$ is the quantity describing the energy loss by considering an a priori *ambient* reduction. *Ambient* is a heuristic word, describing some determined space in which solutions are embedded in a non-trainable fashion. Its construction can, for example, be done by a *Proper Orthogonal Decomposition*.

Next, we will discuss this Proper Orthogonal Decomposition and related concepts. For instance the Kolmogorov N -width, which lets us derive many key concepts and is an important quantity in the field of ROM methods. We base this subsection on (Quarteroni et al., 2015), but import the generalizations to arbitrary inner products from (Franco et al., 2024).

Definition 2.13 (Generalized Proper Orthogonal Decomposition). Assume $N_{\text{data}} < \infty$ and let $\mathbb{G} \in \mathbb{R}^{N_h \times N_h}$ be a mass matrix. We define the time trajectory $U_j = [u(\mu_j, t_k)]_{k=1}^{N_t} \in \mathbb{R}^{N_h \times N_t}$ for a given $\mu_j \in \mathcal{P}$ and consequently $U = [U_j]_{j=1}^{N_s} \in \mathbb{R}^{N_h \times N_{\text{data}}}$ the collective snapshot matrix. Then the (discrete) correlation matrix is given by

$$K = \frac{|\Theta \times [0, T]|}{N_{\text{data}}} U \mathbb{G} U^T \in \mathbb{R}^{N_h \times N_h}. \quad (2.2.1)$$

K is symmetric and semi-positive definite and thus has real and positive eigenvalues $\sigma^2 \geq \dots \geq \sigma_r^2$. Let ψ_k^μ be the corresponding eigenvectors of the correlation matrix K . Then the POD basis of dimension $N < N_h$ is given via

$$\xi_k = \frac{1}{\sigma_k} U \psi_k, \quad 1 \leq k \leq N,$$

for the N largest eigenvalues and their corresponding eigenvectors. Finally, stacking these eigenvectors gives us the *POD matrix*

$$\mathbb{A} := [\xi_k]_{k=1}^N \in \mathbb{R}^{N_h \times N}.$$

This matrix is $\mathbb{G}$ -orthonormal by construction. Further, if $N_s, N_t = \infty$, we define the *continuous correlation matrix* via

$$K_\infty = \int_{\Theta \times [0, T]} u(\mu, t) \mathbb{G} u^T(\mu, t) d(\mu, t) \in \mathbb{R}^{N_h \times N_h}, \quad (2.2.2)$$

and its real eigenvalues by $\sigma_{k, \infty}^2$.

Algorithm 1 Generalized Proper Orthogonal Decomposition (POD)

Require: snapshot matrix $U = [U_j]_{j=1}^{N_s} \in \mathbb{R}^{N_h \times N_{\text{data}}}$, mass matrix $\mathbb{G} \in \mathbb{R}^{N_h \times N_h}$, reduction $N < N_h$

Ensure: POD basis matrix $\mathbb{A}^\mu = [\xi_1^\mu, \dots, \xi_N^\mu] \in \mathbb{R}^{N_h \times N}$

- 1: **if** $N_{\text{data}} \leq N_h$ **then**
 - 2: Form the correlation matrix $C = \frac{|\Theta \times [0, T]|}{N_{\text{data}}} U^T \mathbb{G} U \in \mathbb{R}^{N_{\text{data}} \times N_{\text{data}}}$
 - 3: Solve the eigenvalue problem $C \psi_i = \sigma_i^2 \psi_i, \quad i = 1, \dots, r$
 - 4: Set $\xi_i = \frac{1}{\sigma_i} U \psi_i, \quad i = 1, \dots, r$
 - 5: **else**
 - 6: Form the correlation matrix $K = \frac{|\Theta \times [0, T]|}{N_{\text{data}}} U \mathbb{G} U^T \in \mathbb{R}^{N_h \times N_h}$
 - 7: Solve the eigenvalue problem $K \xi_i = \sigma_i^2 \xi_i, \quad i = 1, \dots, r$
 - 8: **end if**
 - 9: $\mathbb{A} \leftarrow \text{stack}(\xi_1, \dots, \xi_N)$
-

Following this definition, is a known legacy result, which will be among the most used statements in Chapter (4) of this thesis. It shows optimality of the POD and relates the projection error of the Proper Orthogonal Decomposition with its eigenvalues. These values are attained quite easily, which is important to keep in mind.

Proposition 2.14. Let $\mathcal{V}_N = \{W \in \mathbb{R}^{N_h \times N} \mid W^T \mathbb{G} W = I_N\}$ be the set of all N -dimensional $\mathbb{G}$ -orthonormal matrices, $N_{\text{data}} := N_t N_s$ and $\mathbb{A} = [\xi]_{i=1}^N$ be the POD matrix according to Definition (2.13), then

$$\begin{aligned} \frac{|\Theta \times [0, T]|}{N_{\text{data}}} \sum_{i \in I, k=1, \dots, N_t} \|u(\mu_j, t_k) - \mathbb{A} \mathbb{A}^T \mathbb{G} u(\mu_j, t_k)\|^2 \\ = \min_{W \in \mathcal{V}_N} \frac{|\Theta \times [0, T]|}{N_{\text{data}}} \sum_{i \in I, k=1, \dots, N_t} \|u(\mu_j, t_k) - W W^T \mathbb{G} u(\mu_j, t_k)\|^2 = \sum_{j=N+1}^r \sigma_k^2. \end{aligned}$$

Proof. See (Quarteroni et al., 2015, p.125–126). □

Remark 2.15. The equivalent statement can be made for a POD matrix $\mathbb{A}_\infty \in \mathbb{R}^{N_h \times N}$ coming from the continuous correlation matrix $K_\infty \in \mathbb{R}^{N_h \times N_h}$. See (Kunisch & Volkwein, 2001).

In addition, we can further explore the relation between the discrete correlation matrix and its singular values attained as a by-product from Algorithm (1) and the continuous correlation matrix defined above. For this we pose the following.

Proposition 2.16. Let K and K_∞ and their respective eigenvalues be defined as in Definition (2.13), then letting $N_s, N_t \rightarrow \infty$, we get

$$\sum_{k>N} \sigma_k^2 \longrightarrow \sum_{k>N} \sigma_{k,\infty}^2, \quad \text{a.s.}$$

Proof. Let $N_s, N_t < \infty$, conforming to the sampling assumption, and

$$M := \operatorname{ess\,sup}_{(\mu,t) \in \Theta \times \mathcal{T}} \|u(\mu, t)\| < \infty$$

(see introduction in this Section (2.2.2)). Following Remark (2.15), we know, that there exists a $\mathbb{G}$ -orthonormal matrix $\mathbb{A}_\infty \in \mathbb{R}^{N_h \times N}$ such that

$$\begin{aligned} \sum_{k>N} \sigma_{k,\infty}^2 &= \int_{\Theta \times [0,T]} \|u(\mu, t) - \mathbb{A}_\infty \mathbb{A}_\infty^T \mathbb{G} u(\mu, t)\|^2 d(\mu, t) \\ &= \min_{W \in \mathbb{R}^{N_h \times N} : W^T \mathbb{G} W = I} \int_{\mu \times \mathcal{T}} \|u(\mu, t) - W W^T \mathbb{G} u(\mu, t)\|^2 d(\mu, t). \end{aligned}$$

This is the optimal choice of an approximation matrix for the continuous case. However, $\mathbb{A} \in \mathbb{R}^{N_h \times N}$ given by Definition (2.13) is only optimal in the discrete setting and not necessarily the continuous case. Thus, we have

$$\sum_{k>N} \sigma_{k,\infty}^2 \leq \int_{\Theta \times [0,T]} \|u(\mu, t) - \mathbb{A} \mathbb{A}^T \mathbb{G} u(\mu, t)\|^2 d(\mu, t) = \sum_{k>N} \sigma_k^2.$$

Note, that we can uniquely identify $\mathbb{G}^{1/2}$ as the matrix solving $\mathbb{G}^{1/2} \mathbb{G}^{1/2} = \mathbb{G}$, since it is symmetric and positive definite by definition. We then write entry-wise for $m, l = 1, \dots, N_h$

$$\begin{aligned} [K_\infty - K]_{ml} &:= \int_{\Theta \times [0,T]} [\mathbb{G}^{1/2} u(\mu, t)]_m [\mathbb{G}^{1/2} u(\mu, t)]_l d(\mu, t) \\ &\quad - \frac{|\Theta \times [0, T]|}{N_{\text{data}}} \sum_{j=1}^{N_s} \sum_{k=1}^{N_t} [\mathbb{G}^{1/2} u(\mu_j, t_k)]_m [\mathbb{G}^{1/2} u(\mu_j, t_k)]_l. \end{aligned}$$

The resulting matrix is subject to the strong law of large numbers. This is straight forward by the uniform iid sampling assumed, and the general bound on $\sup_{(\mu,t)} \|u(\mu, t)\| \leq M$ before. Hence, this matrix converges almost surely entry-wise to zero as $N_t, N_s \rightarrow \infty$.

Now using Bauer-Fike's Theorem (2.1) with the $\|\cdot\|_{\text{op}}$ -norm, we have upon ordering, that for each $\sigma_{k,\infty}^2$ there is a σ_k^2 in the spectrum of K , such that

$$|\sigma_k^2 - \sigma_{k,\infty}^2| \leq \kappa(\mathbb{A}_\infty) \|K - K_\infty\|_{\text{op}}.$$

Here $\kappa(\mathbb{A}_\infty)$ denotes the condition number of the stacked eigenvector matrix of K_∞ , i.e. $\mathbb{A}_\infty = (\xi_{1,\infty}, \dots, \xi_{N_h,\infty})$. This number is independent of N_s and N_t . Now by convergence of $\|K - K_\infty\|_{\text{op}} \rightarrow 0$ almost surely for $N_s, N_t \rightarrow \infty$, we conclude the proof. $\square$

As already hinted, Proposition (2.14) can be understood as an optimality statement under a sample $i \in I, k = 1, \dots, N_t$ or even the whole space. In combination with Proposition (2.16), this enables us to classify parametric partial differential equations regarding their projectability onto a linear space. This is done using the *linear Kolmogorov N-width* (KnW).

Definition 2.17 (Linear Kolmogorov N -width). Let $N \in \mathbb{N}$, $N \leq N_h$ and $\mathcal{S} = \{u(\mu, t) \in \mathbb{R}^{N_h} \mid (\mu, t) \in \Theta \times [0, T]\}$ be some parametric manifold. The linear Kolmogorov N -width of $\mathcal{S}$ is defined as

$$d_N(\mathcal{S}) = \inf_{\mathbb{V} \in \mathbb{R}^{N_h \times N}} \sup_{u \in \mathcal{S}_{N_h}} \|u - \mathbb{V}\mathbb{V}^T \mathbb{G}u\|.$$

The Kolmogorov N -width can generally be approximated quite well via the tail sums of the eigenvalues of the discrete correlation matrix for large samples. This quantity then allows us to assert a priori, how well any POD-based reduction method will work on the specific PPDE. Namely, if the tail sum for a choice of reduction dimension $N \leq N_h$ is small, then the POD retains most of the energy and a linear reduction can yield high efficiency.

On the other hand, non-linear problems, and specifically hyperbolic and transport dominated PPDEs, have the tendency to have slowly decaying Kolmogorov N -width. This means they retain energy only for large choices of N . Thus, the reduction must then potentially commit a non-linear mapping, motivating the following definition:

Definition 2.18 (Nonlinear Kolmogorov N -width). Let $N \in \mathbb{N}$, $N \leq N_h$ and $\mathcal{S} = \{u(\mu, t) \in \mathbb{R}^{N_h} \mid (\mu, t) \in \Theta \times [0, T]\}$ be some parametric manifold. The nonlinear Kolmogorov N -width of $\mathcal{S}$ is defined as

$$\delta_N(\mathcal{S}) = \inf_{\psi \in C(\mathbb{R}^{N_h}, \mathbb{R}^N), \psi' \in C(\mathbb{R}^N, \mathbb{R}^{N_h})} \sup_{u \in \mathcal{S}_{N_h}} |u - \psi(\psi'(u))|.$$

2.3 Neural Networks

In this section, we will introduce the general notion of neural networks using the notation and most arguments of (Gühring et al., 2019), if not given otherwise. Specifically important for us will be concatenations of dense linear layers, which are a specific subclass of networks, the so-called *Deep Feedforward Neural Network* (DFNN).

Definition 2.19 (Neural Network). Let $d, L \in \mathbb{N}$. A *neural network* Φ with input dimension d and L layers is defined via

$$\Phi = ((A_1, b_1), (A_2, b_2), \dots, (A_L, b_L)),$$

such that $N_0 = d$ and $N_1, \dots, N_L \in \mathbb{N}$, where each A_l is an $N_l \times \sum_{k=0}^{l-1} N_k$ dimensional matrix, and $b_l \in \mathbb{R}^{N_l}$.

We further define $R_\rho(\Phi) : \mathbb{R}^d \rightarrow \mathbb{R}^{N_L}$ as a *realization of Φ with activation function ρ* for some arbitrary map $\rho : \mathbb{R} \rightarrow \mathbb{R}$ as the result of the following scheme for some input $x \in \mathbb{R}^d$.

$$\begin{aligned} x_0 &:= x, \\ x_l &:= \rho \left(A_l \begin{bmatrix} x_0^T & \dots & x_{l-1}^T \end{bmatrix}^T + b_l \right), \quad \text{for } l = 1, \dots, L-1, \\ x_L &:= A_L \begin{bmatrix} x_0^T & \dots & x_{L-1}^T \end{bmatrix}^T + b_L, \end{aligned}$$

where ρ acts entry-wise, i.e. $\rho(y) := [\rho(y_1), \dots, \rho(y_m)]^T \in \mathbb{R}^m$ for each $y = [y_1, \dots, y_m] \in \mathbb{R}^m$. Note that we can write

$$A_l = \begin{bmatrix} A_{l,x_0} & \dots & A_{l,x_{l-1}} \end{bmatrix},$$

with A_{l,x_k} being a $(N_l \times N_k)$ -dimensional matrix for $k = 0, \dots, l-1$ and $l = 1, \dots, L$. This simplifies the scheme via

$$\begin{aligned} x_l &:= \rho \left(A_{l,x_0}x_0 + \dots + A_{l,x_{l-1}}x_{l-1} + b_l \right), \quad \text{for } l = 1, \dots, L-1, \\ x_L &:= A_{L,x_{L-1}}x_{L-1} + b_L. \end{aligned}$$

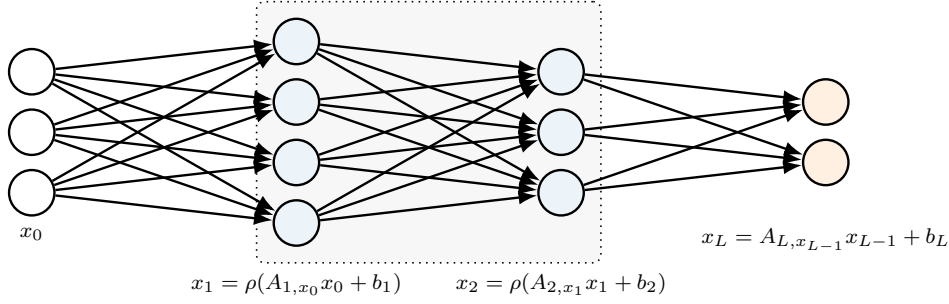


Figure 2.1: Standard Neural Network.

We call $N_\Phi := d + \sum_{l=1}^L N_l$ the *number of neurons* of the network, $L_\Phi := L$ the *number of layers*, and $\omega_\Phi := \sum_{l=1}^L (\|A_l\|_{\ell^0} + \|b_l\|_{\ell^0})$ the *number of active weights*.

Remark 2.20. Note that, we can rewrite known neural network architectures, such as a Convolutional Autoencoder, as a neural network according to Definition (2.19), see also (Gühring et al., 2020, p. 22).

From this we can infer a simpler definition, describing a network which omits any connection of non-neighboring layers.

Definition 2.21 (Standard Neural Network). If a neural network $\Phi = ((A_1, b_1), \dots, (A_L, b_L))$ now has the characteristic

$$A_{l,x_i} = 0, \quad \text{for } l = 1, \dots, L, \text{ and } i = 0, \dots, l-2,$$

then we call Φ a *standard neural network*.

A visualization of a standard neural network can be found in Figure (2.1). If we compare two distinct neural networks, with the same dimensional in- and output, $\Phi_1 = ((A_1, b_1), \dots, (A_L, b_L))$ and $\Phi_2 = ((\hat{A}_1, \hat{b}_1), \dots, (\hat{A}_L, \hat{b}_L))$, it is implied, that if $[A_1]_{ij} = 0$, while $[\hat{A}_1]_{ij} \neq 0$ for some i, j , these networks are severely different. But if they only differ in the value of a specific non-zero weight, they could be recovered from each other through training. This inspires the definition of an *architecture of a network*.

Definition 2.22 (Neural Network Architecture). For some $d, L \in \mathbb{N}$, we define a *neural network architecture* $\mathcal{A}$ with input dimension d and L layers to be a binary neural network, i.e. for all entries of $A_i, b_i \in \{0, 1\}$ for $i = 1, \dots, L$ of $\mathcal{A}$.

We further say that a *neural network* $\Phi = ((\hat{A}_1, \hat{b}_1), \dots, (\hat{A}_L, \hat{b}_L))$ has *architecture* $\mathcal{A}$, if

1. $N_l(\Phi) = N_l$ for all $l = 1, \dots, L$, and
2. $[\hat{A}_l]_{ij} \neq 0$ implies $[A_l]_{ij} \neq 0$ for all $l = 1, \dots, L$ with $i = 1, \dots, N_l$ and $j = 1, \dots, \sum_{k=0}^{l-1} N_k$.

The associated realization of a neural network $R_\rho(\Phi)$ requires the definition of ρ , the activation function. Therefore, we will define the most commonly used one in literature. It will help with some of the constructions, since it has a simple nature, but also provides the network with the necessary non-linearity.

Definition 2.23 (ReLU). The function

$$\rho : \mathbb{R} \rightarrow \mathbb{R}; \quad x \mapsto \max(0, x),$$

is called *ReLU* (Rectified Linear Unit) *activation function*.

Given such an activation function, Kurt Hornik provided in 1990 a significant contribution to the field of approximation theory. He proved, that for any $L^p(\mu)$ function, with μ some finite measure, there is an arbitrarily close ReLU valued neural network realization (Hornik, 1991).

Theorem 2.24 (Hornik's Theorem). *Define for some continuous, unbounded and non-constant activation function ρ the set of all feedforward neural networks, as*

$$\mathcal{N}_d(\rho) := \bigcup_{L=1}^{\infty} \left\{ R_\rho(\Phi) : \mathbb{R}^d \rightarrow \mathbb{R} \mid \Phi \text{ is neural network with } L \text{ layers} \right\}.$$

Then $\mathcal{N}_d(\rho)$ lies dense in $L^p(\mu)$ for every finite measure μ , $p \in (0, \infty)$.

The proof of this statement leverages on Hahn Banach. Thus, it does not provide the specific architecture of the required neural network, which would be used for such an approximation. In our case, we will however focus on an explicit construction to attain bounds for the complexity of the network used. We will hence not mention the proof beyond this outline.

2.3.1 Network concatenations

In this subsection we will discuss the concatenations of multiple neural networks and the effects on its characteristic numbers. The complexity of a concatenated network can be simplified, by using the ReLU as the activation function.

Definition 2.25 (Network Concatenation). Let $L_1, L_2 \in \mathbb{N}$ and

$$\Phi^1 = ((A_1, b_1), \dots, (A_{L_1}, b_{L_1})), \quad \Phi^2 = ((B_1, c_1), \dots, (B_{L_2}, c_{L_2})),$$

be two neural networks such that the input layer of Φ^1 has the same dimension as the output layer of Φ^2 . Then, $\Phi^1 \bullet \Phi^2$ denotes the following $L_1 + L_2 - 1$ layered network.

$$\Phi^1 \bullet \Phi^2 := \left(((B_1, c_1), \dots, (B_{L_2-1}, c_{L_2-1})), ((\tilde{A}_1, \tilde{b}_1), \dots, (\tilde{A}_{L_1}, \tilde{b}_{L_1})) \right),$$

where

$$\tilde{A}_l^1 := [A_{l,x_0} \ B_{L_2} \mid A_{l,x_1} \mid \dots \mid A_{l,x_{l-1}}], \quad \text{and} \quad \tilde{b}_l^1 := A_{l,x_0} c_{L_2} + b_l,$$

for $l = 1, \dots, L_1$. We call $\Phi^1 \bullet \Phi^2$ the *concatenation* of Φ^1 and Φ^2 .

Lemma 2.26 (Neutral Neural Network). Let $\Phi^{\text{Id}} = ((A_1, b_1), (A_2, b_2))$ be a d -dimensional neural network, with

$$A_1 := \begin{bmatrix} \text{Id}_{\mathbb{R}^d} \\ -\text{Id}_{\mathbb{R}^d} \end{bmatrix}, \quad b_1 := 0, \quad A_2 := \begin{bmatrix} 0_{\mathbb{R}^d \times d} \mid \text{Id}_{\mathbb{R}^d} \mid -\text{Id}_{\mathbb{R}^d} \end{bmatrix}, \quad b_2 := 0.$$

Then $R_\rho(\Phi^{\text{Id}}) = \text{Id}_{\mathbb{R}^d}$.

Lemma 2.27. Let two neural networks Φ^1 and Φ^2 be according to Definition (2.25). Then $\Phi^1 \odot \Phi^2 := \Phi^1 \bullet \Phi^{\text{Id}} \bullet \Phi^2$ is a neural network with d -dimensional input, $L_1 + L_2$ layers with $\omega_{\Phi^1 \odot \Phi^2} \leq 2\omega_{\Phi^1} + 2\omega_{\Phi^2}$ active weights, $N_{\Phi^1 \odot \Phi^2} \leq 2N_{\Phi^1} + 2N_{\Phi^2}$ neurons. Additionally, for all $x \in \mathbb{R}^d$, we have

$$R_\rho(\Phi^1 \odot \Phi^2)(x) = R_\rho(\Phi^1) \circ R_\rho(\Phi^2)(x).$$

Proof. This can be proven by direct computation, see (Gühring et al., 2019). □

2.3.2 Network parallelization

We have shown, that it is possible to concatenate networks, such that they show the same behavior as composed functions do. To further build upon that idea, we will now introduce parallelization. For this, define a network $P(\Phi^1, \Phi^2)$ from two networks Φ^1 and Φ^2 , such that it emulates the stacked evaluation of two functions, i.e. $R_\rho(P(\Phi^1, \Phi^2))(x) = (R_\rho(\Phi^1)(x), R_\rho(\Phi^2)(x))$ for all $x \in \mathbb{R}^d$.

Definition 2.28 (Network Parallelization). Let $L_1, L_2 \in \mathbb{N}$ s.t. $L_1 \leq L_2$ and let

$$\Phi^1 = ((A_1, b_1), \dots, (A_{L_1}, b_{L_1})), \quad \Phi^2 = ((B_1, c_1), \dots, (B_{L_2}, c_{L_2})),$$

be two neural networks with d -dimensional input. We call

$$P(\Phi^1, \Phi^2) := ((C_1, d_1), \dots, (C_{L_2}, d_{L_2})),$$

the *parallelization of Φ^1 and Φ^2* with

$$C_l := \left[\begin{array}{c|cc|ccc} A_{l,x_0} & A_{l,x_1} & 0 & \cdots & A_{l,x_{l-1}} & 0 \\ B_{l,x_0} & 0 & B_{l,x_1} & \cdots & 0 & B_{l,x_{l-1}} \end{array} \right], \quad d_l := \begin{bmatrix} b_l \\ c_l \end{bmatrix}, \quad \text{for } 1 \leq l < L_1,$$

$$C_l := \left[\begin{array}{c|c|ccc|c|ccc} B_{l,x_0} & 0 & B_{l,x_1} & \cdots & 0 & B_{l,x_{L_1-1}} & B_{l,x_{L_1}} & \cdots & B_{l,x_{L_2-1}} \end{array} \right], \quad d_l := c_l,$$

for $L_1 \leq l < L_2$, and

$$C_{L_2} := \left[\begin{array}{c|cc|ccc|c|ccc} A_{L_1,x_0} & A_{L_1,x_1} & 0 & \cdots & A_{L_1,x_{L_1-1}} & 0 & 0 & \cdots & 0 \\ B_{L_2,x_0} & 0 & B_{L_2,x_1} & \cdots & 0 & B_{L_2,x_{L_1-1}} & B_{L_2,x_{L_1}} & \cdots & B_{L_2,x_{L_2-1}} \end{array} \right],$$

$$d_{L_2} := \begin{bmatrix} b_{L_1} \\ c_{L_2} \end{bmatrix}.$$

A similar construction is used for the case $L_1 > L_2$.

Lemma 2.29. Let Φ^1 and Φ^2 be as in Definition (2.28). Then $P(\Phi^1, \Phi^2)$ is a neural network with d -dimensional input, $\max\{L_1, L_2\}$ layers, $\omega_{P(\Phi^1, \Phi^2)} = \omega_{\Phi^1} + \omega_{\Phi^2}$ active weights and $N_{P(\Phi^1, \Phi^2)} = N_{\Phi^1} + N_{\Phi^2} - d$ neurons. Additionally, for all $x \in \mathbb{R}^d$, we have that

$$R_\rho(P(\Phi^1, \Phi^2))(x) = (R_\rho(\Phi^1)(x), R_\rho(\Phi^2)(x)).$$

Proof. This can be again shown by direct computation. See (Gühring et al., 2019). $\square$

2.3.3 Standard Neural Networks

Here, we only want to briefly state an interesting fact, following from the definition of the standard networks, that motivates the treatment of any neural network architecture as a standard one. This generally allows us to omit the cumbersome nature of complete connectivity.

Corollary 2.30. Let ρ be the ReLU activation function, and Φ a neural network with d -dimensional input and $L = L(\Phi)$ layers, $N = N(\Phi)$ neurons and $M = M(\Phi)$ nonzero weights. Then there is a standard neural network Φ_{st} with d -dimensional input, L layers, at most $C_1 LN$ neurons and $C_2 \cdot (LN + M)$ nonzero weights, such that

$$R_\rho(\Phi)(x) = R_\rho(\Phi_{\text{st}})(x)$$

for all $x \in \mathbb{R}^d$ with $C_1, C_2 > 0$ being constants.

Proof. See (Gühring et al., 2019, p.9). $\square$

2.3.4 Approximation Theory

For all following theoretical results we will assume that the activation function ρ is the ReLU as given in Definition (2.23). Furthermore, define the set of functions bounded in Sobolev norm as

$$\mathcal{F}_{n,d,p,B} := \{f \in W^{n,p}((0,1)^d) \mid \|f\|_{W^{n,p}((0,1)^d)} \leq B\}.$$

In this subsection, we will briefly discuss the needed steps to find a bound for the active weights and layers used for a neural network, approximating an arbitrary function f of some regularity. We first construct a network, that realizes an interpolated multiplication operation. This quasimultiplication will be able to relate a partition of unity using polynomial functions with a neural network. For this, we need to take the interpolation error into account, which is closely linked to the regularity of the function, we want to approximate. For instance, if we have a function f in the Sobolev space $W^{2,2}((0,1)^d)$, then it is apparent, that at least two weak derivatives of f must exist. Therefore, this can be leveraged to get a Taylor expansion around some selected points.

This procedure will deduce a quantifiable error of the network approximation for any $W^{1,\infty}(\Omega)$ function.

This leaves us with two main objectives.

1. Bound the Sobolev function at critical equidistant points, using its Taylor expansion.
2. Relate a fixed error rate of the approximation polynomial and some neural network to the networks complexity.

These two objectives are pursued by both Yarotsky (Yarotsky, 2017) and Gühring et al (Gühring et al., 2019). Note, that Yarotsky only provides the special case $s = 0, B = 1$, since the proof only shows an estimation with a Taylor expansion regarding the L^∞ norm. This misses the crucial consideration of the weak derivative bound for Lipschitz functions.

Even Though we will use the statement of Gühring later on, which is the extension of Yarotsky to any choice of s, p and B , we will only give the idea of the proof for Gühring. It is similar to the proof of Yarotsky, but does include a lot of further theoretical setting, which will not be important for the actual results presented in this paper.

Proposition 2.31. Let $\varepsilon \in (0,1)$ and $f : [0,1] \rightarrow [0,1]; x \mapsto x^2$. Then there are constants $c_1, c_2, c_3 > 0$, such that there is a neural network $\Phi_\varepsilon^{\text{sq}}$ with 1-dimensional input and output, realizing

$$\|R_\rho(\Phi_\varepsilon^{\text{sq}}) - f\|_{L^\infty((0,1))} \leq \varepsilon,$$

and $R_\rho(\Phi_\varepsilon^{\text{sq}})(0) = 0$. This network has at most $c_1 \cdot \ln(1/\varepsilon)$ nonzero weights, at most $c_2 \cdot \ln(1/\varepsilon)$ layers, at most $c_3 \cdot \ln(1/\varepsilon)$ neurons.

Proof. Define the *tooth function* $g : [0,1] \rightarrow [0,1]$ by

$$g(x) = \begin{cases} 2x, & \text{for } x < 1/2, \\ 2(1-x), & \text{for } x \geq 1/2, \end{cases}$$

and its composition, the *sabertooth function*,

$$g_s(x) := \underbrace{g \circ g \circ \dots \circ g(x)}_{s\text{-times}}.$$

It has been proven in (Telgarsky, 2015), that g_s then has the following explicit representation

$$g_s(x) = \begin{cases} 2^s \left(x - \frac{2k}{2^s} \right), & \text{for } x \in \left[\frac{2k}{2^s}, \frac{2k+1}{2^s} \right], k = 0, \dots, 2^{s-1} - 1, \\ 2^s \left(\frac{2k}{2^s} - x \right), & \text{for } x \in \left[\frac{2k-1}{2^s}, \frac{2k}{2^s} \right], k = 1, \dots, 2^{s-1}. \end{cases}$$

Furthermore, from the definition of the ReLU function (refer to Definition (2.23)), we can also set $g(x) = 2\rho(x) - 4\rho(x - 1/2) + 2\rho(x - 1)$. This is the same as defining a neural network with 1-dimensional input and output via $\Phi = ((A_1, b_1), (A_2, b_2))$ with

$$A_1 = \begin{bmatrix} \text{Id}_{\mathbb{R}} \\ \text{Id}_{\mathbb{R}} \\ \text{Id}_{\mathbb{R}} \end{bmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ -1/2 \\ -1 \end{pmatrix}, \quad A_2 = \begin{bmatrix} 2 & -4 & 2 \end{bmatrix}, \quad b_2 = 0.$$

Then it is evident, using Lemma (2.27), that $g_m(x) = R_\rho(\overbrace{\Phi \odot \dots \odot \Phi}^{m\text{-times}})(x)$ for all $x \in [0, 1]$ represents a network with $\mathcal{O}(m)$ layers, active weights and neurons. Now, we set up a linear combination of these sabertooth functions as

$$\begin{aligned} f_m(x) &:= x - \sum_{s=1}^m \frac{g_s(x)}{2^{2s}} \\ &= \frac{2k+1}{2^m}x - \frac{k(k+1)}{2^{2m}}, \quad \text{for } x \in \left[\frac{k}{2^m}, \frac{k+1}{2^m} \right], k = 0, 1, \dots, 2^m - 1, \end{aligned}$$

and any $m \in \mathbb{N}$. In fact, if we define the dyadic intervals $I_k := [\frac{k}{2^m}, \frac{k+1}{2^m}]$ like above, then we can argue with the slope and the value separately. Let $m \in \mathbb{N}$ be fixed and $k = 0, 1, \dots, 2^m - 1$. If

$$x = \frac{k + \alpha}{2^m}, \quad \text{with } \alpha \in [0, 1),$$

we can assure $0 \leq \theta/2^{m-s} < 1$ for any $s \leq m$. Therefore, the slope is constant on each I_k . And since the slope for each segment of a fixed sabertooth function g_s is the same in magnitude, we write

$$\frac{d}{dx} f_m(x) = 1 - \sum_{s=1}^m \frac{g'_s(x)}{2^{2s}} = 1 - \sum_{s=1}^m \sigma_s(x) 2^{-s},$$

for any $x \in \overset{\circ}{I}_k$. Here, for $q_s = \lfloor k/2^m \rfloor$, the lower interval boundary, we define

$$\sigma_s(x) = \begin{cases} +1, & \text{if } q_s \text{ is even,} \\ -1, & \text{if } q_s \text{ is odd.} \end{cases}$$

Notice, that we can identify any natural number $k \leq m$ with a binary representation

$$k = \sum_{r=0}^{m-1} b_r 2^r, \quad \text{with } b_r \in \{0, 1\}.$$

This is useful, because we know with boolean arithmetics, that if the first digit is zero, the natural number it represent must be even. Thus write

$$q_s = \sum_{r=m-s}^{m-1} b_r 2^{r-(m-s)},$$

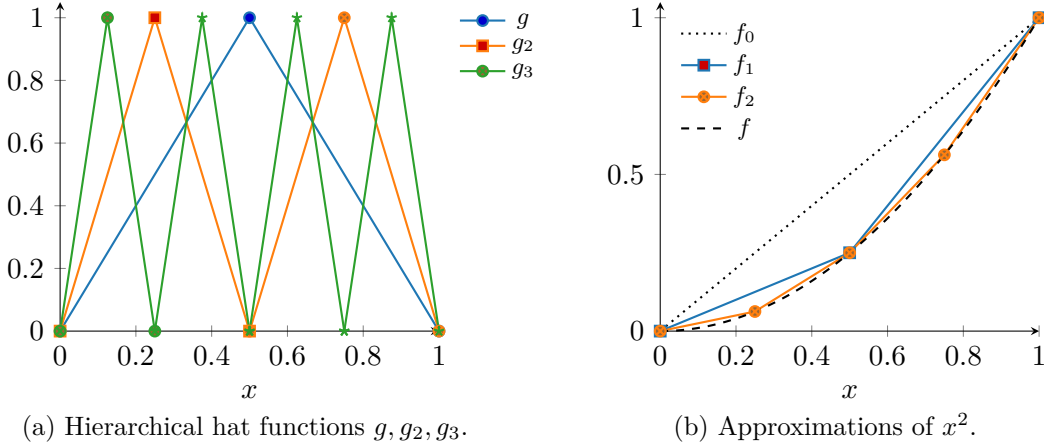


Figure 2.2: Sabertooth basis functions and their implementation.

which lets us denote the parity of q_s with b_{m-s} , the first digit. Now, writing $\varepsilon_s := 1/2(1 - \sigma_s) \in \{0, 1\}$ on any I_k , we get for the total slope for $x \in I_k$

$$\begin{aligned} \frac{d}{dx} f_m(x) &= 1 - \sum_{s=1}^m \sigma_s(x) 2^{-s} \\ &= 1 - \sum_{s=1}^m 2^{-s} + 2 \sum_{s=1}^m \varepsilon_s(x) 2^{-s} \\ &= 1 - (1 - 2^{-m}) + 2 \sum_{s=1}^m b_{m-s} 2^{-s} \\ &= 2^{-m} + 2 \sum_{r=0}^{m-1} b_r 2^{-m+r} = 2^{-m} + 2 \cdot 2^{-m} k. \end{aligned}$$

Further, for the shifts, because we know this function is piecewise affine, we can simply solve for $c_k \in \mathbb{R}$ on each I_k

$$f_m\left(\frac{k}{2^m}\right) = \frac{2k+1}{2^m} \cdot \frac{k}{2^m} + \alpha_k = \left(\frac{k}{2^m}\right)^2.$$

A visualization of the functions can be found in Figure (2.2), which is lifted from (Yarotsky, 2017).

Now, since we know that the function f_m is just an addition of at most m layered networks

$$g_m(x) = g \circ \dots \circ g(x) = R_\rho(\Phi \odot \dots \odot \Phi),$$

and the input, we can set up a neural network realizing f_m . This is done straight forward using a layer, which adds all layers together, while inverting each of the non-inputs. We call the subsequent neural network Φ_m^{sq} . This network then has $\mathcal{O}(m+1)$ layers, non-zero weights and neurons.

Finally, we can use the explicit definition of $f_m = R_\rho(\Phi_m^{\text{sq}})$ for some fixed $m \in \mathbb{N}$ to attain an L^∞ -approximation bound. For this, simply use a piece-wise extrema discussion on each I_k , for $k = 0, 1, \dots, 2^m - 1$. We derive

$$\frac{d}{dx} (f_m(x) - f(x)) = \frac{d}{dx} \left(\frac{2k+1}{2^m} x - \frac{k(k+1)}{2^m} - x^2 \right) = \frac{2k+1}{2^m} - 2x.$$

Setting this to zero, and inserting it in the antiderivative gives us for each I_k a k -independent bound of

$$\|R_\rho(\Phi_m^{\text{sq}}) - f\|_{L^\infty((0,1))} = \|f_m - f\|_{L^\infty((0,1))} = 2^{-2m-2}.$$

Hence, setting $m = \lceil \ln(1/\varepsilon) \rceil$, with $C, C' > 0$ constants, only dependent on the domain, we achieve the wanted bound from the statement for the neural network $\Phi_\varepsilon^{\text{sq}} := \Phi_m^{\text{sq}}$ with $\mathcal{O}(m) = \mathcal{O}(\ln(1/\varepsilon) + 1) \leq \mathcal{O}(2 \ln(1/\varepsilon))$ layers, neurons and active weights. $\square$

This statement enables us to express multiplication using a neural network. This can be shown by considering the polarization identity

$$xy = \frac{1}{2}((x+y)^2 - x^2 - y^2), \quad \text{for } x, y \in \mathbb{R}, \quad (2.3.1)$$

as given by the first binomial formula.

Proposition 2.32. Let $M > 0$ and $\varepsilon \in (0, 1)$. Then there is a ReLU neural network $\tilde{\times}$ with 2-dimensional input and 1-dimensional output, and constants $c_1 > 0$ and $c_2 = c_2(M) > 0$, such that for $\times : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}; (x, y) \mapsto xy$

1. $\|R_\rho(\tilde{\times}) - \times\|_{W^{0,\infty}((-M,M)^2)} \leq \varepsilon$,
2. if $x = 0$ or $y = 0$, then $R_\rho(\tilde{\times})(x, y) = 0$,
3. the number of layers, active weights and neurons of $\tilde{\times}$ is at most $c_1 \ln(1/\varepsilon) + c_2$.

Proof. Let $\delta := \varepsilon/(6M^2)$. Then we construct an approximation of the square function $f(x) = x^2$ for $x \in (0, 1)$ using Proposition (2.31) for the chosen δ . Indeed, we define Φ_δ^{sq} , such that

$$\|R_\rho(\Phi_\delta^{\text{sq}}) - f\|_{W^{0,\infty}((0,1)^1)} \leq \frac{\varepsilon}{6M^2},$$

which now of course has $\mathcal{O}(\ln(1/\delta))$ many layers, active weights and neurons. To achieve this we have to implement negative numbers as well. With $|x| = \rho(x) + \rho(-x)$, it can be shown, that it is indeed possible to expand any neural network with a positive domain to a negative domain using a neural network (by simply concatenating). Thus, we construct $\tilde{\times}$ via appending an addition layer on top of parallel square networks with scaled input, achieving

$$R_\rho(\tilde{\times})(x, y) = 2M^2 \left(R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|x+y|}{2M} \right) - R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|x|}{2M} \right) - R_\rho(\Phi_\delta^{\text{sq}}) \left(\frac{|y|}{2M} \right) \right).$$

We need the third statement of our Proposition, when putting together the necessary complexity of this network. This uses the constants in Proposition (2.31) as follows.

1. We have 2 layers for the implementation of the absolute value, 1 for the addition in the end, and $c_0 \ln(1/\delta)$ for the square function. Thus, $L_{\tilde{\times}} \leq c_0 \ln(1/\varepsilon) + c_0 \ln(6M^2) + 3$ in total.
2. We need $4 + 2 + 2$ active weights for the first matrix to distribute each (x, y) onto the parallel networks together with 6 active weights for the superposing of the input into the $|\cdot|$ -function. Another 3 are required for the last layer adding all outputs. Each of the different computations for the artificial scaled square function must be in parallel. Hence, we have $\omega_{\tilde{\times}} \leq 3c_0 \ln(1/\varepsilon) + 3c_0 \ln(6M^2) + 17$ in total.
3. Similar to (2.), we get $9 = (2 + 1) \cdot 3$ neurons for the first two and the last layer, thus giving us $N_{\tilde{\times}} \leq 3c_0 \ln(1/\varepsilon) + 3c_0 \ln(6M^2) + 9$ in total.

Note, that c_0 does not depend on M . By taking the adequate maximum, we get the desired bound on the complexity in (3.).

To make the true multiplication match the form of the implemented network, using (2.3.1), we may write for $x, y \in (-M, M)$

$$\begin{aligned} xy &= 4M^2 \left(\frac{x}{2M} \cdot \frac{y}{2M} \right) \\ &= 4M^2 \frac{1}{2} \left(\left(\frac{x}{2M} + \frac{y}{2M} \right)^2 - \left(\frac{x^2}{2M} \right)^2 - \left(\frac{y^2}{2M} \right)^2 \right) \\ &= 2M^2 \left(\left(\frac{|x+y|}{2M} \right)^2 - \left(\frac{|x|}{2M} \right)^2 - \left(\frac{|y|}{2M} \right)^2 \right). \end{aligned}$$

Now with the deviation from each of the square approximation we obtain,

$$\begin{aligned} \|R_\rho(\tilde{\times}) - \times\|_{L^\infty((-M,M)^2)} &\leq 2M^2 \left(3 \cdot \sup_{\substack{|x| \\ 2M} : x \in (-M,M)} |R_\rho(\Phi_\delta^{\text{sq}})(\tilde{x}) - \tilde{x}^2| \right) \\ &\leq 6M^2\delta = \varepsilon. \end{aligned}$$

We discuss a few things happening here for clarity. It is possible to simply use the reverse triangle equality to get all three error terms added, and then put them together. This can be done, since the $L^\infty((0,1)^2)$ norm detects only the maximal value of the error across all possible elements of the $(0,1)^2$ domain. Here the inner function is bijective, so these are just the same. Additionally, we use the statement

$$\|g \circ f\|_{L^\infty((0,1))} \leq \|g\|_{L^\infty((0,1))}.$$

This proves the statement (1.).

Statement (2.) on the other hand, is a direct consequence of the subliminal result in Proposition (2.31). $\square$

As a last step, before stating and proving the Yarotsky Theorem, we will discuss the construction and complexity of a network approximating any collection of equidistantly spaced function with local support in a higher dimensional domain $(0,1)^d$ for $1 \leq d < \infty$. Remark, that we set up these networks, such that their realizations form a partition of unity on the d -dimensional domain. Then this construction will be enough to get an approximation under a corresponding complexity discussion.

Lemma 2.33. For any $d, N \in \mathbb{N}$ there is a collection of functions

$$\Psi = \left\{ \phi_{\vec{m}} \mid \vec{m} \in \{0, \dots, N\}^d \right\},$$

with $\phi_{\vec{m}} : \mathbb{R}^d \rightarrow \mathbb{R}$ for all $\vec{m} = (m_1, \dots, m_d) \in \{0, \dots, N\}^d$ with the following properties.

1. $0 \leq \phi_{\vec{m}}(x) \leq 1$ for all $\phi_{\vec{m}} \in \Psi$ and every $x \in \mathbb{R}^d$,
2. $\sum_{\phi_{\vec{m}} \in \Psi} \phi_{\vec{m}}(x) = 1$ for every $x \in [0, 1]^d$,
3. $\text{supp}(\phi_{\vec{m}}) \subset B_{1/N, \|\cdot\|_{\ell^\infty}}(\vec{m}/N)$ for all $\phi_{\vec{m}} \in \Psi$,

4. There are constants $C, c \geq 1$ such that for each $\phi_{\vec{m}} \in \Psi$ there is a neural network $\Phi_{\vec{m}}$ with d -dimensional input and output, at most 3 layers, $C \cdot d$ active weights and neurons, such that

$$\prod_{l=1}^d [R_\rho(\Phi_{\vec{m}})]_l = \phi_{\vec{m}}$$

and $\| [R_\rho(\Phi_{\vec{m}})]_l \|_{L^\infty((0,1)^d)} \leq 1$ for all $l = 1, \dots, d$.

Proof. In this proof, we will construct adequate functions, which are easily transferable into ReLU neural networks. For this, define

$$\psi : \mathbb{R} \rightarrow [0, 1], \quad \psi(x) = \begin{cases} 1, & |x| < 1, \\ 0, & 2 < |x|, \\ 2 - |x| & 1 \leq |x| \leq 2, \end{cases}$$

and $\phi_{\vec{m}} : \mathbb{R}^d \rightarrow \mathbb{R}$ as the product of shifted and scaled versions of ψ , which inherit its local support, i.e.

$$\phi_{\vec{m}}(x) := \prod_{l=1}^d \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right),$$

for $\vec{m} = (m_1, \dots, m_d) \in \{0, \dots, N\}^d$ and $x = (x_1, \dots, x_d) \in \mathbb{R}^d$. This function satisfies the properties (1.), (2.) and (3.) directly. In fact, let $x_i \in \mathbb{R}$ arbitrary for some $i = 1, \dots, d$. Now we get that $\psi(3N(x_i - (m_i/N))) \in [-2, 2]$, since ψ has support of $[-2, 2]$. This gives us

$$x_i \in \left[\frac{m_i}{N} - \frac{2}{3N}, \frac{m_i}{N} + \frac{2}{3N} \right],$$

yielding the property (3.), as we took i arbitrarily. Simultaneously, we use the zero product law, and the fact, that the range of ψ is $[0, 1]$ to get property (1.). To prove property (2.) let $z_i := 3Nx_i$. Then we can rewrite for any N the one-dimensional function

$$S_i(x_i) := \sum_{m_i=0}^N \psi \left(3N \left(x_i - \frac{m_i}{N} \right) \right) \quad \text{into the function} \quad \tilde{S}_i(z_i) := \sum_{m_i=0}^N \psi(z_i - 3m_i).$$

In turn, this gives us the desired result of $\tilde{S}_i(z_i) = 1$ for any $z_i \in [0, 3N]$. This is due to the distance between shifts to the input being 3. This means going forwards or backwards by more than that lands one in the same situation. Hence, we only consider two cases without loss of generality. If $z_i \in [-1, 1]$, we directly get 1. For $z_i \in [1, 2]$, we get

$$\psi(z_i) + \psi(z_i - 3) = (2 - z_i) + (2 + (z_i - 3)) = 1.$$

We visualize this part of the proof in Figure (2.3). Now we can extrapolate this result to S_i by a simple scaling argument. Afterwards, we rearrange and use the factorization of the sum over products to get

$$\sum_{\vec{m} \in \{0, \dots, N\}^d} \phi_{\vec{m}}(x) = \sum_{m_1=0}^N \cdots \sum_{m_d=0}^N \prod_{l=1}^d \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right) = \prod_{l=1}^d \left(\sum_{m_l=0}^N \psi \left(3N \left(x_l - \frac{m_l}{N} \right) \right) \right) = 1.$$

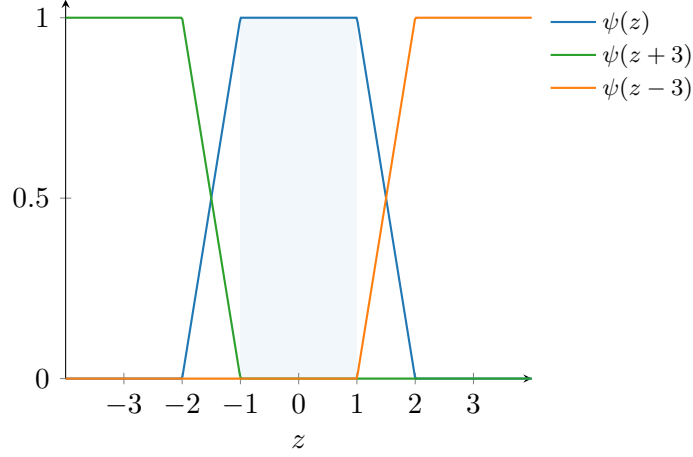


Figure 2.3: Shifted copies at step size 3.

Finally, it only remains to prove property (4.). For this we need to construct a neural network Φ_ψ , that realizes ψ . Let

$$A_1 := \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}, \quad b_1 := \begin{pmatrix} 2 \\ 1 \\ -1 \\ -2 \end{pmatrix}, \quad A_2 := [0 \mid 1 \quad -1 \quad -1 \quad 1], \quad b_2 := 0,$$

define the neural network $\Phi_\psi = ((A_1, b_1), (A_2, b_2))$. Then, by using the identity $|x| = \rho(x) + \rho(-x)$, it can be shown, that for all $x \in \mathbb{R}$

$$\begin{aligned} R_\rho(\Phi_\psi)(x) &= A_2(\rho(A_1 x + b_1)) + b_2 \\ &= A_2 \left(\begin{bmatrix} \rho(x+2) & \rho(x+1) & \rho(x-1) & \rho(x-2) \end{bmatrix}^T \right) \\ &= \rho(x+2) + \rho(x-2) - (\rho(x+1) + \rho(x-1)) \\ &= \psi(x). \end{aligned}$$

The complexity of this network is 2 layers, 12 active weights and 6 neurons. Let $\Phi_{\vec{m},l}$ be a neural network with d -dimensional input and one-dimensional output, such that

$$R_\rho(\Phi_{\vec{m},l})(x) = 3N \left(x_l - \frac{m_l}{N} \right), \quad \text{for all } x \in \mathbb{R}^d \text{ and } \vec{m} \in \{0, \dots, N\}^d.$$

Then the complexity of this network is 1 layer, 2 active weights and $d+1$ neurons. Finally, realizing that the composition of both these functions, i.e. the realization of the concatenation of networks, would yield each factor for the desired function. If we now parallelize this concatenation, we get property (4.) by definition. In fact, let

$$\Phi_{\vec{m}} := P(\Phi_\psi \odot \Phi_{\vec{m},l} \mid l = 1, \dots, d),$$

since then

$$\prod_{l=1}^d [R_\rho(\Phi_{\vec{m}})]_l(x) = \phi_{\vec{m}}(x), \quad \text{for all } x \in \mathbb{R}^d.$$

This network now has d -dimensional input and output and by Lemma (2.27) and Lemma (2.29), has 3 layers, at most $(12+2) \cdot 2 \cdot d = 28d$ active weights, and at most $9d$ neurons. $\square$

With these auxiliary results, we may finally state and proof the Yarotsky Theorem.

Theorem 2.34 (Yarotsky Theorem). *Let $d \in \mathbb{N}, n \in \mathbb{N}, B = 1$ and $f \in \mathcal{F}_{n,d,\infty,B}$ some arbitrary function. Then there is a constant $c = c(d, n) > 0$ with the following properties:*

For any $\varepsilon \in (0, 1)$, there is a neural network architecture $\mathcal{A}_\varepsilon = \mathcal{A}_\varepsilon(d, n, \varepsilon)$ with d -dimensional input and one-dimensional output such that, there is a neural network Φ_ε^f that has architecture $\mathcal{A}_\varepsilon$ and satisfies

$$\|f - R_\rho(\Phi_\varepsilon^f)\|_{W^{0,\infty}((0,1)^d)} \leq \varepsilon,$$

and

1. $L_{\mathcal{A}_\varepsilon} \leq c \cdot \ln(1/\varepsilon),$
2. $\omega_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/n} \ln(1/\varepsilon),$
3. $N_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/n} \ln(1/\varepsilon).$

Proof. Let $\phi_{\vec{m}} \in \Psi$ be as constructed in the proof of Lemma (2.33). We will divide the proof of this theorem into two parts. First, we will formulate an approximate function f_1 for f using the partition of unity and an adequate Taylor expansion. Secondly, we will find the subsequent approximation of that function f_1 by a neural network Φ_{f_1} using the $\tilde{\times}$ -multiplication of Proposition (2.32).

Step 1: Taylor Expansion

For any $\vec{m} \in \{0, \dots, N\}^d$, consider the $(n-1)$ -Taylor polynomial for the function f at $x_0 = \vec{m}/N$:

$$P_{\vec{m}}(x) = \sum_{\vec{n}: |\vec{n}| \leq n} \frac{D^{\vec{n}} f(x)}{\vec{n}!} \Big|_{x=\frac{\vec{m}}{N}} \left(x - \frac{\vec{m}}{N}\right)^{\vec{n}}, \quad \text{for } x \in \mathbb{R}^d,$$

with the conventions $\vec{n}! = \prod_{l=1}^d n_l!$ and $(x - \vec{m}/N)^{\vec{n}} = \prod_{l=1}^d (x_l - \frac{m_l}{N})^{n_l}$. With the partition of unity of Lemma (2.33), we can then define any function, as a linear combination of its point-values and the partition function. In particular, this allows us to define

$$f = \sum_{\vec{m} \in \{0, \dots, N\}^d} \phi_{\vec{m}} f, \quad \text{and} \quad f_1 = \sum_{\vec{m} \in \{0, \dots, N\}^d} \phi_{\vec{m}} P_{\vec{m}}.$$

Now we can utilize the properties given by Lemma (2.33), as follows; firstly use the $\|\phi_{\vec{m}}\|_{L_\infty((0,1)^d)} = 1$ property in Step (2.3.3). Then one can estimate the maximal number of partitions with 2^d in Step (2.3.4). Afterwards, we take the simple Lagrange remainder in Step (2.3.5). Lastly we argue with $f \in \mathcal{F}_{n,d,\infty,B=1}$ in Step (2.3.6).

$$\|f - f_1\|_{L_\infty((0,1)^d)} \leq \sum_{\vec{m} \in \{0, \dots, N\}^d} \|\phi_{\vec{m}} (f - P_{\vec{m}})\|_{L_\infty((0,1)^d)} \quad (2.3.2)$$

$$\leq \sum_{\vec{m}: |x_l - \frac{m_l}{N}| < \frac{1}{N}, \forall l=1, \dots, d} \|f - P_{\vec{m}}\|_{L_\infty((0,1)^d)} \quad (2.3.3)$$

$$\leq 2^d \max_{\vec{m}: |x_l - \frac{m_l}{N}| < \frac{1}{N}, \forall l=1, \dots, d} \|f - P_{\vec{m}}\|_{L_\infty((0,1)^d)} \quad (2.3.4)$$

$$\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^n \max_{\vec{n}: |\vec{n}|=n} \|D^{\vec{n}} f\|_{L_\infty((0,1)^d)} \quad (2.3.5)$$

$$\leq \frac{2^d d^n}{n!} \left(\frac{1}{N}\right)^{\vec{n}} \quad (2.3.6)$$

Now taking

$$N = \left\lceil \left(\frac{n!}{2^d d^n} \cdot \frac{\varepsilon}{2} \right)^{-\frac{1}{n}} \right\rceil$$

gives the approximation bound of

$$\|f - f_1\|_{L_\infty((0,1)^d)} \leq \frac{\varepsilon}{2}. \quad (2.3.7)$$

Step 2: Approximation by a Neural Network

The polynomial function f_1 can be written out to underline its construction, which only utilized the functions $\phi_{\vec{m}}$ and multiplications. In fact,

$$f_1(x) = \sum_{\vec{m} \in \{0, \dots, N\}^d} \sum_{\vec{n}: |\vec{n}| < n} a_{\vec{m}, \vec{n}} \underbrace{\left(\phi_{\vec{m}}(x) \left(x - \frac{\vec{m}}{N} \right)^{\vec{n}} \right)}_{=: f_{\vec{m}, \vec{n}}(x)}, \quad (2.3.8)$$

for all $x \in \mathbb{R}^d$. This expansion has at most $d^n(N+1)^d$ terms containing $f_{\vec{m}, \vec{n}}$, since

$$\left| \left\{ \vec{n} \in \{0, \dots, n-1\}^d : |\vec{n}| < n \right\} \right| \leq \left| \left\{ \vec{n} \in \{0, \dots, n-1\}^d \right\} \right| = d^n,$$

and obviously $|\{\vec{m} \in \{0, \dots, N\}^d\}| = (N+1)^d$. Note, that we take $M = d+n$ to define the network multiplication, i.e. $\tilde{\times} : (-M, M)^2 \rightarrow \mathbb{R}$. This is done, because if $x, y \in (-1, 1)$, then

$$|R_\rho(\tilde{\times})(x, y) - xy| \leq \delta \implies |R_\rho(\tilde{\times})(x, y)| \leq |y| + \delta,$$

for some fixed but not yet specified $\delta \in (0, 1)$. Recursively concatenating $\tilde{\times}$ for $(d+n-1)$ -times can then propagate the error to at most $M = (d+n)$ upon the last realization of $\tilde{\times}$.

As aforementioned, we can now infer a neural network, that has a comparable realization to the function $f_{\vec{m}, \vec{n}}$. This network will be a concatenation of the quasi-multiplication networks $\tilde{\times}$. It uses $(n-1)$ multiplications for the estimated realization of $\prod_{l=1}^d (x_l - (m_l/N))^{n_l}$, since we know that $\sum_{l=1}^d n_l < n$. Additionally, d multiplications for the estimated realization of the function $\phi_{\vec{m}}$ and one multiplication to put both together. Hence, we define a neural network $\Phi_{f_1}^{\vec{m}, \vec{n}}$ with d -dimensional input and one dimensional output via

$$\begin{aligned} R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}})(x) = R_\rho(\tilde{\times}) & \left([R_\rho(\Phi_{\vec{m}})]_1(x_1), R_\rho(\tilde{\times}) \left([R_\rho(\Phi_{\vec{m}})]_2(x_2), \dots \right. \right. \\ & R_\rho(\tilde{\times}) \left([R_\rho(\Phi_{\vec{m}})]_d(x_d), R_\rho(\tilde{\times}) \left(x_1 - \frac{m_1}{N}, \dots \right. \right. \\ & R_\rho(\tilde{\times}) \left(x_{d-1} - \frac{m_{d-1}}{N}, R_\rho(\tilde{\times}) \left(x_d - \frac{m_d}{N}, \dots \right. \right. \\ & \left. \left. \left. \left. \left. \left. R_\rho(\tilde{\times}) \left(x_d - \frac{m_d}{N}, x_d - \frac{m_d}{N} \right) \right) \right) \right) \right) \right) \right), \end{aligned}$$

for $x = (x_1, \dots, x_d) \in \mathbb{R}^d$. Here we used $\Phi_{\vec{m}}$ as in the proof of Lemma (2.33). This network contains at most $(d+n)$ concatenations of $\tilde{\times}$. Thus, by Lemma (2.33), Lemma (2.27), Proposition (2.32) and the constants $C, c_0, c_1(d, n) > 0$ as mentioned, we obtain

$$L_{\Phi_{f_1}^{\vec{m}, \vec{n}}} \leq 3c_0(d+n) \left(\ln \left(\frac{1}{\delta} \right) + (d+n)c_1 \right), \quad \omega_{\Phi_{f_1}^{\vec{m}, \vec{n}}} \leq C2(d+n)d \left(\ln \left(\frac{1}{\delta} \right) + 2(d+n)C \right), \quad (2.3.9)$$

for an a priori fixed precision level $\delta \in (0, 1)$. Note, that the number of neurons are of the same order as the number of active weights.

The error comparison of $R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}})$ and $f_{\vec{m}, \vec{n}}$ can then be done by counting the maximal depth of usage of the network $\tilde{\times}$, which is $(d + n)$ as shown above, and the estimate of

$$\left| \left(x - \frac{\vec{m}}{N} \right) \right| \leq 1, \quad |\psi(x)| \leq 1,$$

for all $x \in (0, 1)^d$. Then it follows directly, that

$$\|R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}}) - f_{\vec{m}, \vec{n}}\|_{L_\infty((0,1)^d)} \leq (d + n)\delta.$$

As mentioned below Equation (2.3.8), f_1 has at most $d^n(N + 1)^d$ terms. But since any $x \in \mathbb{R}^d$ can be only in the support of at most 2^d functions $\phi_{\vec{m}}$ by the second statement in Proposition (2.32), the sum going over all $\vec{m}$ can be reduced to only those, which are non-zero. We employ this argument in the upcoming Step (2.3.11). This improves the amount of summands from $(N + 1)^d$ to 2^d , which we will use in Step (2.3.12). Setting

$$\delta = \frac{\varepsilon}{2 \cdot 2^d d^n (d + n)}$$

allows us to finally assert, using the bound of Equation (2.3.9) in Step (2.3.13), that

$$\|R_\rho(\Phi_{f_1}) - f_1\|_{L_\infty((0,1)^d)} \leq \sum_{\vec{m} \in \{0, \dots, N\}^d} \sum_{\vec{n}: |\vec{n}| < n} |a_{\vec{m}, \vec{n}}| \|R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}}) - f_{\vec{m}, \vec{n}}\|_{L_\infty((0,1)^d)} \quad (2.3.10)$$

$$= \sum_{\vec{m}: x \in \text{supp}(\phi_{\vec{m}})} \sum_{\vec{n}: |\vec{n}| < n} |a_{\vec{m}, \vec{n}}| \|R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}}) - f_{\vec{m}, \vec{n}}\|_{L_\infty((0,1)^d)} \quad (2.3.11)$$

$$\leq 2^d d^n \max_{\vec{m}: x \in \text{supp}(\phi_{\vec{m}})} \max_{\vec{n}: |\vec{n}| < n} \|R_\rho(\Phi_{f_1}^{\vec{m}, \vec{n}}) - f_{\vec{m}, \vec{n}}\|_{L_\infty((0,1)^d)} \quad (2.3.12)$$

$$\leq 2^d d^n (d + n) \delta \quad (2.3.13)$$

$$= \frac{\varepsilon}{2}. \quad (2.3.14)$$

Given this δ , there are at most $d^n(N + 1)^d = \mathcal{O}(\varepsilon^{-d/n})$ parallel networks with one additive layer. Moreover, the fact that generally we have $c' \ln(1/\delta + c'') \leq c''' \ln(1/\delta)$ for $c', c'' > 0$ and $c''' \geq c' \ln(1/c'')$, gets us the final complexity of Φ_{f_1} by Equation (2.3.9). In fact,

$$L_{\Phi_{f_1}} \leq c \ln(1/\varepsilon), \quad \omega_{\Phi_{f_1}} \leq c \varepsilon^{-d/n} \ln(1/\varepsilon), \quad N_{\Phi_{f_1}} \leq c \varepsilon^{-d/n} \ln(1/\varepsilon),$$

with some constant $c = c(d, n) > 0$.

Using the triangle inequality, we can show that this network indeed satisfies the wanted precision bound with both errors as in the last Step (2.3.6) and the last Step (2.3.14), respectively:

$$\|f - R_\rho(\Phi_{f_1})\|_{L_\infty((0,1)^d)} \leq \|f - f_1\|_{L_\infty((0,1)^d)} + \|f_1 - R_\rho(\Phi_{f_1})\|_{L_\infty((0,1)^d)} \leq \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

□

We may extend Yarotsky to cover a more general case, where $f \in \mathcal{F}_{n,d,p,B}$ for arbitrarily $1 \leq p \leq \infty$ and $B > 0$ and the norm, to which we assert the precision, is an arbitrary Sobolev norm. As a matter of fact, we will denote, that we use Yarotsky, even if $B \neq 0$, to clearly separate the extension to the more general Sobolev-Slobodeckij norm, as long as the other prerequisites are met. This extension is presented by Gühring et al. (Gühring et al., 2019).

Theorem 2.35 (Gühring Theorem). *Let $d \in \mathbb{N}, n \in \mathbb{N}_{\geq 2}, B > 0, 1 \leq p \leq \infty$ and $0 \leq s \leq 1$ and $f \in \mathcal{F}_{n,d,p,B}$ some arbitrary function. Then there is a constant $c = c(d, n, p, B, s) > 0$ with the following properties:*

For any $\varepsilon \in (0, 1)$, there is a neural network architecture $\mathcal{A}_\varepsilon = \mathcal{A}_\varepsilon(d, n, p, B, s, \varepsilon)$ with d -dimensional input and one-dimensional output such that, there is a neural network Φ_ε^f that has architecture $\mathcal{A}_\varepsilon$ and satisfies

$$\|f - R_\rho(\Phi_\varepsilon^f)\|_{W^{s,p}((0,1)^d)} \leq \varepsilon,$$

and

1. $L_{\mathcal{A}_\varepsilon} \leq c \cdot \ln(1/\varepsilon),$
2. $\omega_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/(n-s)} \ln(1/\varepsilon),$
3. $N_{\mathcal{A}_\varepsilon} \leq c \cdot \varepsilon^{-d/(n-s)} \ln(1/\varepsilon).$

Proof Sketch. We will only sketch the general steps of the proof.

1. The first step is to expand the Proposition (2.31) to the $W^{1,\infty}(0,1)$ -norm. This is done by straight forward calculation.
2. This can be applied to Proposition (2.32) in the same way. Additionally, we need to bound $|R_\rho(\tilde{\times})|_{W^{1,\infty}((0,1)^2)}$, in order to estimate the collective error under the Taylor expanded function.
3. The crucial next step is to prepare the switch of norms using

$$\|f - f_N\|_{W^{s,p}((0,1)^d)} \leq C \left(\frac{1}{N}\right)^{n-s} \|f\|_{W^{n,p}((0,1)^d)},$$

for a partition of unity of size N . It is worth to note, that the factor in front influences the choice of δ and thus is responsible for the different approximation in regard to weights and neurons.

4. The last step, is the approximation of a sum of localized polynomials with a neural network. This requires control of the derivatives.

□

2.3.5 Statistical Learning

We have now introduced the framework for analytical discussion regarding complexity and potential of neural networks. It still remains to achieve a practical neural network Φ fitting the function in question for some fixed architecture $\mathcal{A}_\Phi$. This crucial step in the process of development is referred to as *module training* or *statistical learning*.

In order to quantify the "falseness" of a potential classification, we need a function, that compares a true solution to an output of the network. The following two definitions are based on (Ciampiconi et al., 2024), where the interested reader can find extensive terminology and currently used loss functions and their optimization techniques.

Definition 2.36 (Feature and Label Space). Let $\mathcal{X}$ be the so-called *feature space* and $\mathcal{Y}$ the *label space*. Let further

$$f : \mathcal{X} \rightarrow \mathcal{Y}$$

be the underlying *true classification function*, mapping each feature to a label.

The general goal of all machine learning problems is to closely approximate that classification function f .

Definition 2.37 (Loss Function). Let $\mathcal{X}$ be a feature space, $\mathcal{Y}$ a label space, and $f : \mathcal{X} \rightarrow \mathcal{Y}$ the underlying true classification function. Given some arbitrary function $\hat{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ and a sample set $\{x_i, f(x_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$ for some $N \in \mathbb{N}$, we define the general loss function as

$$\mathcal{L}(\hat{f}_\theta | \{x_1, \dots, x_N\}, \{f(x_1), \dots, f(x_N)\}) \in \mathbb{R}^+,$$

such that for f being the function from Definition (2.36)

$$\mathcal{L}(f | \{x_1, \dots, x_N\}, \{f(x_1), \dots, f(x_N)\}) = 0.$$

Definition 2.38 (Neural Network Weights). Let $d, L, N_L \in \mathbb{N}$ and

$$\Phi = ((A_1, b_1), \dots, (A_L, b_L))$$

be some neural network with input dimension d , L layers and output dimension N_L . Define the *masks of Φ* via

$$I_l := \{(i, j) : [A_l]_{ij} \neq 0\}, \quad J_l := \{i : [b_l]_i \neq 0\},$$

for each layer $1 \leq l \leq L$. We further set the *admissible weight space* as

$$\Theta(\Phi) := \prod_{l=1}^L (\mathbb{R}^{I_l} \times \mathbb{R}^{J_l}),$$

and its elements $\theta \in \Theta(\Phi)$ with

$$\theta = \left((\theta_{l,ij}^A)_{(i,j) \in I_l}, (\theta_{l,i}^b)_{i \in J_l} \right)_{l=1}^L.$$

Then $\Phi(\theta) = ((\hat{A}_1(\theta), \hat{b}_1(\theta)), \dots, (\hat{A}_L(\theta), \hat{b}_L(\theta)))$ can be uniquely defined for a choice of $\theta \in \Theta(\Phi)$ via the lifting

$$\hat{A}_l(\theta)_{ij} = \begin{cases} \theta_{l,ij}^A, & (i, j) \in I_l, \\ 0, & \text{otherwise,} \end{cases} \quad \hat{b}_l(\theta)_i = \begin{cases} \theta_{l,i}^b, & i \in J_l, \\ 0, & \text{otherwise.} \end{cases}$$

This allows us to classify realizations of a neural network uniquely by a map, when knowing its architecture and a choice of neural network weights $\theta \in \Theta(\Phi)$, via

$$R_\rho(\Phi(\theta)) : \mathbb{R}^d \rightarrow \mathbb{R}^{N_L}.$$

In particular, we know that if Φ has architecture $\mathcal{A}_\Phi$, then there is $e \in \Theta(\Phi)$, such that $\Phi(e) = \mathcal{A}_\Phi$.

This definition will be used to address the training and associated *freezing* of networks for online computation. Namely, it provides the liberty to separate already trained, or frozen, neural networks from ones, that are still being optimized. Furthermore, it does not hinder any results, since all approximation theoretic results are posed in an "existence of architecture" context anyway.

The natural next question is, how such optimization might look. To briefly address this, we provide the reader with a crude overview of the techniques, we will be using in our implementation. The main goal here, is to minimize the associated loss function for some neural network Φ with regard to some true classification function f , i.e.

$$\arg \min_{\theta \in \Theta(\mathcal{A}_\Phi)} \mathcal{L}(R_\rho(\Phi(\theta)) | \{x_1, \dots, x_N\}, \{f(x_1), \dots, f(x_N)\})$$

In the following, a brief introduction to the topics of batching, normalization, optimizers and schedulers is given.

Batching The term *batch* refers to a stack of a sample $\{x_i, f(x_i)\}_{i=1}^N \subset \mathcal{X} \times \mathcal{Y}$. This usually means, stacking along a new dimension. Conventionally this is the first dimension and the batches are chosen as large as possible, but such that a clustering of features is avoided.

Normalization In computer science the specific order of numbers is important, since scaling can affect the problem. This can be mitigated, by a priori normalizing the input data and the output data. For example, consider a one-dimensional sample $\{x_i, y_i\}_{i=1}^N \subset \mathbb{R} \times \mathbb{R}$. Then the *min-max normalization* for that sample is given via

$$x_i \mapsto \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1], \quad y_i \mapsto \frac{y_i - y_{\min}}{y_{\max} - y_{\min}} \in [0, 1],$$

where we set up the maximum and minimum via

$$x_{\max} = \max_{i=1, \dots, N} x_i, \quad x_{\min} = \min_{i=1, \dots, N} x_i,$$

and for $y_{\min}, y_{\max}$ accordingly. It is important to remember to denormalize the solution for evaluation and comparative analysis with the deterministic inverse operation. For our later example, we will switch this into a multidimensional setting.

Optimizer The choice of an optimization method is crucial for the practical performance of neural networks. The original gradient descent scheme with stochastic approximation dates back to Robbins and Monro (Robbins & Monro, 1951), and became widely adopted in the form of backpropagation (Rumelhart et al., 1986). A first improvement was the introduction of momentum terms (Qian, 1999), followed by adaptive step-size methods such as Adagrad (Duchi et al., 2011) and RMSProp (Tieleman & Hinton, 2012). The most popular adaptive optimizer in modern deep learning is Adam (Kingma & Ba, 2015), which combines momentum with adaptive learning rates, and its decoupled variant AdamW for improved regularization (Loshchilov & Hutter, 2019). In our implementations we will mainly rely on AdamW.

Scheduler Learning rate schedulers control the step-size of the optimizer during training. While constant learning rates are sufficient for convergence in theory, in practice carefully designed schedules can significantly improve both training stability and generalization. One well-established approach is the use of *cosine annealing with warm restarts* (SGDR) introduced by Loshchilov and Hutter (Loshchilov & Hutter, 2017), which periodically reduces and resets the learning rate. Another popular technique is the class of cyclical learning rates (Smith, 2017). In this work, we adopt cosine warm restarts due to their good empirical performance on PDE-related problems.

Chapter 3

Formulation of the General Problem

Let $\Theta \subset \mathbb{R}^p$ and $\Theta' \subset \mathbb{R}^q$ be compact and denote the geometric and physical parameter spaces respectively. Additionally, let $\Omega \subset \mathbb{R}^d$ be some Lipschitz domain and $[0, T]$ be a time interval for some $T > 0$ of a parametric time-dependent PDE. For $\mu \in \Theta$ and $\nu \in \Theta'$

$$u^{\mu, \nu} := u(\mu, \nu), \quad \text{with sufficient regularity,}$$

is called the (strong) solution of the PPDE, with the choice of the function space depending on the specified problem, if

$$\begin{cases} \frac{\partial}{\partial t} u^{\mu, \nu}(x, t) + \mathcal{L}(\mu, \nu) u^{\mu, \nu}(x, t) + \mathcal{N}(u^{\mu, \nu}(x, t), \mu, \nu) = f(x, t; \mu, \nu), & (x, t) \in \Omega \times [0, T], \\ \mathcal{B}(\mu, \nu) u^{\mu, \nu}(x, t) = g_N(x, t; \mu, \nu) & (x, t) \in \partial\Omega \times [0, T], \\ u^{\mu, \nu}(x, t) = u_0^{\mu, \nu}(x) & (x, t) \in \Omega \times \{0\}, \end{cases}$$

where $\mathcal{L}$ represents a linear, and $\mathcal{N}$ represents a non-linear integro-differential operator. Using calculus of variation or distribution theory, one can then formulate a weak form of the problem and find a general space. For example, if the problem has a weak form, the space commonly is Sobolev (for elliptic problems with coercive properties), Bochner (for parabolic problems with coercive properties on the stationary part) or discontinuous Galerkin spaces (for hyperbolic problems with enough regularity). Such a space $\mathcal{H}$ is generally some Banach space, most often a subspace of L^2 .

This weak solution will have less regularity imposed than the strong formulation. Such weak form, then sometimes allows us to retrieve existence and uniqueness results. Quite famously, there are counter examples; the Navier-Stokes Equation in three dimensions poses one such result. For this PDE, a weak form exists, and numerical methods have been developed, which can provide a solution in special cases, but existence remains unproven in general. As mentioned before, we will omit the transformation of the strong PDE formulation into a weak one, and simply assume that this can be done.

Say we (can) discretize the problem in the spacial domain Ω with N_h degrees of freedom. For this, let $\mathcal{H}_h \subset \mathcal{H}$ a Hilbert space with $\mathcal{H}_h \cong \mathbb{R}^{N_h}$, an inner product $\langle \cdot, \cdot \rangle_{\mathcal{H}_h}$ and a corresponding norm $\|\cdot\|_{\mathcal{H}_h}$. Then the resulting full order model solution can be identified by

$$t \mapsto u_h(\cdot, t; \mu, \nu) = \sum_{j=1}^{N_h} [u_h^{\mu, \nu}(t)]_j \varphi_j(\mu) \in C^1([0, T]; \mathcal{H}_h) \quad (3.0.1)$$

with $\{\varphi_j(\mu)\}_{j=1}^{N_h}$ a basis of $\mathcal{H}_h$ for each $\mu \in \Theta$ and $u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}$ for each $t \in [0, T]$. Then it is

assumed, that the coefficients are a solution to a problem of the following formulation.

$$\begin{cases} M(\mu) \frac{\partial}{\partial t} u_h^{\mu, \nu}(t) + A(\mu, \nu) u_h^{\mu, \nu}(t) + N(u_h^{\mu, \nu}(t), \mu, \nu) = f(t; \mu, \nu) & t \in [0, T], \\ u_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0, \end{cases} \quad (3.0.2)$$

where we assume

- $M : \Theta \rightarrow \mathbb{R}^{N_h \times N_h}$ to send the geometrical parameter to a mass matrix,
- $A : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h \times N_h}$ to send the set to a symmetric positive definite stiffness matrix,
- $N : \mathbb{R}^{N_h} \times \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ to be some non-linear term,
- $f : [0, T] \times \mathbb{R}^{N_h} \times \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ to be the corresponding source term of the equation.

Additionally, let $u_0 : \Theta \times \Theta' \rightarrow \mathbb{R}^{N_h}$ be the initial datum. The mass matrix is defined for each $\mu \in \Theta$ as follows

$$M(\mu) = \begin{bmatrix} \langle \varphi_1(\mu), \varphi_1(\mu) \rangle_{\mathcal{H}_h} & \cdots & \langle \varphi_1(\mu), \varphi_{N_h}(\mu) \rangle_{\mathcal{H}_h} \\ \vdots & \ddots & \vdots \\ \langle \varphi_{N_h}(\mu), \varphi_1(\mu) \rangle_{\mathcal{H}_h} & \cdots & \langle \varphi_{N_h}(\mu), \varphi_{N_h}(\mu) \rangle_{\mathcal{H}_h} \end{bmatrix}.$$

This allows us to equip $\mathcal{S} \subset \mathbb{R}^{N_h}$, the space of the coefficients of any solution in $\mathcal{H}_h$, with the following norm

$$\|u_h^{\mu, \nu, t}\| := \sqrt{(u_h^{\mu, \nu, t})^T M(\mu) u_h^{\mu, \nu, t}} \quad (3.0.3)$$

for $\mu \in \Theta, \nu \in \Theta', t \in [0, T]$ and $u_h^{\mu, \nu, t} \in \mathcal{S}$. In fact, the norm above used on the coefficient representation is equivalent to the $\mathcal{H}_h$ -norm, i.e. $\|u_h^{\mu, \nu, t}\| = \|u_h(\cdot, t; \mu, \nu)\|_{\mathcal{H}_h}$.

Note that it is indeed possible to achieve this discretization without skipping to a weak formulation, e.g. using finite differences or finite volume schemes. For the interested reader, an explicit discretization using finite volume schemes can be seen in (Haasdonk & Ohlberger, 2008).

The formulation in Equation (3.0.2) establishes smoothness with regard to time. This is called *method of lines approach* (Hesthaven et al., 2022, p. 278). To solve this resulted ODE, we will implicitly further discretize by using time steps $\{t_k\}_{k=1}^{N_t} := \mathcal{T}$ to gather data.

Definition 3.1 (Solution Data). We denote with

$$u_h^{\mu, \nu, t} := u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}, \quad \text{for } \mu \in \Theta, \nu \in \Theta' \text{ and all } t \in \mathcal{T},$$

a finite dimensional *solution snapshot* and

$$u_h^{\mu, \nu} := [u_h^{\mu, \nu}(t)]_{t \in \mathcal{T}} \in \mathbb{R}^{N_h \times N_t}, \quad \text{for } \mu \in \Theta \text{ and } \nu \in \Theta',$$

with $N_t := |\mathcal{T}|$ a finite dimensional *solution trajectory* of the discretized parametric PDE. Furthermore, let $\mathcal{P}_1 \subset \Theta$, $\mathcal{P}_2 \subset \Theta'$ and $\mathcal{P} = \mathcal{P}_1 \times \mathcal{P}_2$ be some finite parameter data with sizes $N_{s_1}, N_{s_2} \in \mathbb{N} \cup \{\infty\}$ and $N_s = N_{s_1} N_{s_2}$, respectively.

We have stated the problem framework, to know which class of PDEs, we will be considering. However, to justify the use of L^2 norms, we assume that $\mathcal{H}_h$ can be embedded in L^2 . Then we can compare the finite dimensional solution $u_h \in \mathcal{H}_h$ and the approximation of that solution $\hat{u}_h \in \mathcal{H}_h$ (of reduced order) using ROM methods with L^2 estimates.

From this point forward, we want to consistently rely on the following few assumptions, which regulate the discrete problem above.

Assumption (1) justifies a nuanced approach regarding the geometric parameter μ and the physical parameter ν , but can be relaxed under some constraints. Note, that the exact size of α and β is not needed for any of the upcoming proofs. However, the general separation of problems with slow and fast KnW decay is important in the context of literature.

Assumption (2) is required for the derivation of any convergence result to a true parameter-to-solution map, when sampling an ever-growing number of examples.

Assumption (3) imposes regularity on the mentioned parameter-to-solution map. It ensures the existence of a true solution of the discrete problem for any specified parameter choice. We observe, that this does not translate into the solvability for the upstream general problem.

Assumption (4) gives us the ability to later meaningfully use an Autoencoder to achieve a reduction. Its application is justified with results from (Franco et al., 2023).

Assumption 1. Let $\mathcal{S} := \{u_h^{\mu,\nu,t} \in \mathbb{R}^{N_h} \mid (\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]\}$ denote the *total solution manifold* and $\mathcal{S}_{\mu,t} := \{u_h^{\mu,\nu,t} \in \mathbb{R}^{N_h} \mid \nu \in \Theta'\}$ denote the *solution submanifold* for a fixed set $(\mu, t) \in \Theta \times [0, T]$, then we assume that

1. the linear KnW of $\mathcal{S}$ decays slowly, i.e.

$$d_N(\mathcal{S}) \leq CN^{-\alpha},$$

where $0 < \alpha \leq 1$ and $C > 0$ some constant.

2. the linear KnW of $\mathcal{S}_{\mu,t}$ decays quickly for all $\mu \in \Theta$ and $t \in [0, T]$, i.e.

$$\sup_{(\mu,t) \in \Theta \times [0,T]} d_N(\mathcal{S}_{\mu,t}) \leq C'N^{-\beta},$$

where $\beta > 1$ and $C' > 0$ some constant.

This is mostly satisfied for problems in which the transport dominated, or non-linear part is chained to the parameter μ , and any coercive or elliptic operator is most heavily influence by ν .

Assumption 2. Let Θ, Θ' and $[0, T]$ for $T > 0$ as above. We assume that all $N_s \in \mathbb{N}_{\geq 2}$ parameter data points as in (3.1) are sampled independent and identically distributed in the joint parameter space $\Theta \times \Theta'$. For the time variable a uniform grid $\mathcal{T} = \{\Delta t, 2\Delta t, \dots, N_t\Delta t\}$ is used, where $N_t \in \mathbb{N}_{\geq 2}$, N_{s_1} and N_{s_2} are the respective number of random samples in the parameter spaces Θ and Θ' , $N_s = N_{s_1}N_{s_2} \in \mathbb{N}$ is the total sample size, and $\Delta t = T/N_t$.

We remark, that the equidistant spacing of the time samples is a matter of convenience. It could be loosened under some constraints.

Assumption 3. Let $\mathcal{G} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}$ be the parameter-to-solution map, mapping $(\mu, \nu, t) \mapsto u_h^{\mu,\nu,t}$. Then, we assume that

1. $m = \operatorname{ess\,inf}_{(\mu,\nu,t) \in \Theta \times \Theta' \times \mathcal{T}} \|u_h(\mu, \nu, t)\| > 0$, $M = \operatorname{ess\,sup}_{(\mu,\nu,t) \in \Theta \times \Theta' \times \mathcal{T}} \|u_h(\mu, \nu, t)\| < \infty$,
2. $\mathcal{G}$ is Lipschitz-continuous with the constant $L > 0$.

There is another subliminal assumption inside the well-posed definition of such a parameter-to-solution map; the solution $u_h(\cdot, \cdot; \mu, \nu)$ exists and is unique. This is meant, at least, in an L^2 -embedded sense, since the time-derivative can be relaxed using a variational argument.

Assumption 4. Let $N \in \mathbb{N}$ be fixed and $n \in \mathbb{N}$ such that $n \leq 2(p+q)+3$. We assume that for s, s' sufficiently large, there are two maps $\psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^N$ and $\psi'_* : \mathbb{R}^N \rightarrow \mathbb{R}^n$ which attain a nonlinear embedding without loss of the reduced solution manifold, as given in Definition (2.18), i.e. for $\mathcal{S}_N := \{q(\mu, \nu, t) = \mathbb{A}^T u(\mu, \nu, t) \mid (\mu, \nu, t) \in \Theta \times \Theta' \times \mathcal{T}\}$, where $\mathbb{A} \in \mathbb{R}^{N_h \times N}$ is the POD matrix,

$$\delta_n(\mathcal{S}_N) = 0.$$

From here on, every introduced Reduced Order Model will pursue the following universal (and vague) goal.

Goal 1. Find

$$\hat{\mathcal{G}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \hat{u}_h^{\mu, \nu, t},$$

such that for all $\mu \in \Theta$ and $\nu \in \Theta'$

$$\sup_{t \in [0, 1]} \|u_h^{\mu, \nu, t} - \hat{u}_h^{\mu, \nu, t}\| = \sup_{t \in [0, 1]} \|u_h(\cdot, t; \mu, \nu) - \hat{u}_h(\cdot, t; \mu, \nu)\|_{\mathcal{H}_h}$$

is small.

We recall, that $\mathcal{H}_h \subset \mathcal{H}$ is a finite dimensional approximation space of our weak problem. As mentioned before, we discretized time-steps. Therefore, we have something akin to

$$\begin{cases} M(\mu) \hat{\partial}_t u_h^{\mu, \nu}(t) + A(\mu, \nu) u_h^{\mu, \nu}(t) + N(u_h^{\mu, \nu}(t), \mu, \nu) = f(t; \mu, \nu) & t \in \mathcal{T}, \\ u_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0, \end{cases} \quad (3.0.4)$$

for $\hat{\partial}_t$ a discrete time-stepping operator and $u_h^{\mu, \nu}(t) \in \mathbb{R}^{N_h}$ for $t \in [t_k, t_{k+1})$. For some $k = 0, \dots, N_t-1$ the piecewise constant coefficient vector for the true solution is then given by

$$u_h(\cdot, t; \mu, \nu) = \sum_{j=1}^{N_h} [u_h^{\mu, \nu}(t)]_j \varphi_j(\mu) \in \mathcal{H}_h, \text{ for each } t \in [t_k, t_{k+1}) \text{ and } (\mu, \nu) \in \Theta \times \Theta'.$$

3.1 Intrusive Reduced Order Models

In this section, we will introduce some *intrusive* methods for the dimensionality reduction of the finite dimensional parametric PDE. The name "intrusive" means to symbolize the "intrusion" of the method into the formulation of the PDE and the specifications of its operators. There are methods for linear and non-linear problems respectively. The more simple and lightweight reduction techniques can leverage an abstract linear variational problem to achieve a (Petrov-)Galerkin projection. The methods for non-linear problems, so problems with $N \neq 0$ according to Equation (3.0.2), utilize the specifications of the non-linear operator to achieve reduction.

Remark 3.2 (Historic Literature). The development of intrusive reduced order models (ROMs) dates back to the late 1970s and 1980s, when methods for projection onto low-dimensional trial spaces were systematically introduced in the context of parametrized PDEs. Early formulations of the *reduced basis (RB) method* can be found in the control and approximation theory literature, e.g. in the pioneering works of Courant and Hilbert on variational methods (Courant & Hilbert, 1953), and in the first computational implementations by Noor and coworkers in structural mechanics (Noor, 1980). These ideas were later rigorously developed into the modern RB methodology by

Patera in the mid-1980s (Patera, 1984), and subsequently advanced by Maday, Rovas, Prud'homme and others in the 1990s and 2000s (Maday et al., 2002; Prud'homme et al., 2002).

In parallel, the POD emerged as a computational tool for obtaining optimal low-rank approximations of dynamical systems, building on statistical formulations dating back to Karhunen and Loève. The introduction of POD-Galerkin for intrusive model reduction was consolidated in applied mechanics and fluid dynamics in the 1990s and early 2000s (see (Holmes et al., 1996; Kunisch & Volkwein, 2001)).

As nonlinear problems gained prominence, the cost of evaluating nonlinear terms prompted the development of *hyper-reduction techniques*, such as the Empirical Interpolation Method (EIM) (Barrault et al., 2004) and its discrete variants (DEIM) (Chaturantabut & Sorensen, 2010), which remain core components of modern intrusive ROM frameworks.

We will start with the framework of projection-based methods, that necessitate a problem with $N = 0$. All the following methods will be discussed broadly, as the thesis will focus on the later data-driven ROMs.

Definition 3.3 (Reduced Basis). Let $\mathcal{H}_n \subset \text{span}(u_h(\cdot, t; \mu, \nu) \in \mathcal{H}_h \mid (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T})$ be an n -dimensional subspace of $\mathcal{H}_h$, then if $\{\hat{\varphi}_j\}_{j=1}^n$ is an orthonormalized basis of $\mathcal{H}_n$, we call this the *reduced basis*.

Henceforth, the goal of all *reduced basis methods* is to find a reduced basis $\{\hat{\varphi}_j(\mu)\}_{j=1}^n$ and $\hat{u}_h^{\mu, \nu}(t) \in \mathbb{R}^n$ for $n \ll N_h$, such that

$$\hat{u}_h(\cdot, t; \mu, \nu) = \sum_{j=1}^n [\hat{u}_h^{\mu, \nu}(t)]_j \hat{\varphi}_j(\mu) \in \mathcal{H}_n \subset \mathcal{H}_h.$$

Note, that we aim to vaguely achieve

$$\hat{u}_h(\cdot, t; \mu, \nu) \approx u_h(\cdot, t; \mu, \nu) \quad \text{for all } \mu \in \Theta, \nu \in \Theta' \text{ and } t \in [0, T].$$

In the terminology the Reduced Basis (RB) methods are a setup for many ROMs, and often appear for so-called projection-based ROMs. These itself are intrusive ROMs. Projection-based ROMs identify a reduced basis, and then solve the reduced problem for the coefficients of this basis. Two prominent examples of such RB methods in the context of projection-based ROMs can be found in the following. Both will contain intrinsically an *offline* and an *online* part in the algorithm, whose separation we will mostly neglect. However, the offline procedure commonly refers to the part of the algorithm, that can be collected a priori to its purposed deployment. Here, this is the construction of a reduced basis via samples. On the other hand, online refers to its counterpart; the many-query element of the procedure, such as the evaluation of the best-guess parameter-to-solution map. This terminology is used frequently throughout the literature as it instinctively makes sense in a software development environment.

3.1.1 Galerkin Proper Orthogonal Decomposition

As a prerequisite for Galerkin orthogonality, we assume for this subsection that the weak form giving rise to the equation at hand derives from an abstract bilinear form in the stationary part. This is justified, since the non-linear summand is assumed to be zero. The source for this subsection is (Quarteroni et al., 2015, p. 137).

The POD as introduced in Definition (2.13) is one of the most common methods for a dimensional reduction. We can use the system above to solve for the solution using a Galerkin projection for the

following identity.

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^n [\hat{u}_h^{\mu, \nu}(t)]_j \xi_j(x), \quad (3.1.1)$$

where $x \in \Omega$, $t \in [0, T]$, $(\mu, \nu) \in \Theta \times \Theta'$ and ξ_j for $1 \leq j \leq n$ are the POD modes, of Definition (2.13) with $\mathbb{G} := \mathbb{I}_{N_h}$. Then $\hat{u}_h^{\mu, \nu}(t) \in \mathbb{R}^n$ is the coefficient vector, that solves the reduced equation

$$\begin{cases} \mathbb{A}^T M(\mu) \mathbb{A} \hat{\partial}_t \hat{u}_h^{\mu, \nu}(t) + \mathbb{A}^T A(\mu, \nu) \mathbb{A} \hat{u}_h^{\mu, \nu}(t) = \mathbb{A}^T f(t; \mu, \nu) & t \in \mathcal{T}, \\ \hat{u}_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0. \end{cases} \quad (3.1.2)$$

Note, that in this construction $\mathbb{A}$ is generally *not* $M(\mu)$ -orthogonal. However, if we can set up the problem with a μ -independent mass matrix $\mathbb{G}$, then such a POD matrix would retain $\mathbb{G}$ -orthonormality. This would then conclude in a simpler variant of the problem in Equation (3.1.2) due to $\mathbb{A}^T \mathbb{G} \mathbb{A} = \mathbb{I}_n$. The parameter-to-solution map can be recovered by the following Algorithm (2), where the steps 1–4 are executed *offline* and 5–6 are called for each *online* computation.

Algorithm 2 POD-Galerkin

Require: $(\mu_j, \nu_j)_{j=1}^{N_s}$ data sample and $n < N_h$ reduced dimension

Ensure: approximated parameter-to-solution-map $(\mu, \nu) \mapsto \hat{u}_h(\cdot, \cdot; \mu, \nu)$

- 1: Compute $\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}$ from Equation (3.0.4) (with $N = 0$)
 - 2: $(\xi_j, \sigma_j)_{j=1}^n \leftarrow \text{POD}(u_h^{\mu_j, \nu_j, t_{k_m}}, n)$
 - 3: $\mathbb{A} \leftarrow \text{stack}(\xi_j, j = 1, \dots, n)$
 - 4: Build reduced Equation (3.1.2)
 - 5: For each (μ, ν) solve Equation (3.1.2)
 - 6: Recover $\hat{u}_h(\cdot, \cdot; \mu, \nu)$ via Equation (3.1.1)
-

3.1.2 Greedy Proper Orthogonal Decomposition

An introduction to this method in the stationary and well-posed case can be found in (Siena et al., 2022). As before, we assume $N = 0$ in Equation (3.0.4) and the inheritance from a weak formulation. Additionally, let $\eta : \Theta \times \Theta' \rightarrow \mathbb{R}$ be a function estimating the error of projection under a specific parameter set, i.e.

$$\sup_{t \in \mathcal{T}} \|u_h(\mu, \nu, t) - \hat{u}_h(\mu, \nu, t)\| \leq \eta(\mu, \nu),$$

for $(\mu, \nu) \in \mathcal{P}$. Purely using POD-Galerkin can encounter major pitfalls. Namely, if the range of $\mathcal{P}$ is extensive, the calculation of an eigenvalue decomposition for the correlation matrix might get too cumbersome to provide all areas of interest in the set. Moreover, to obtain the snapshots for fixed parameters, a complete time-trajectory must be computed, which entails a lot of overhead of otherwise useless information. To overcome this the mixed POD-Greedy approach was proposed in (Grepl & Patera, 2005). This method was later improved in (Haasdonk & Ohlberger, 2008). The method takes a POD over the solution trajectories (or, in the improved version, the error trajectories), while using the Greedy approach to handle the parameter samples. It follows a similar workflow, as the POD-Galerkin, since the only thing, that changes is the choice and number of basis functions. In fact, the size of the basis is chosen according to the tolerance in the offline part of the algorithm.

Let

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^N [\hat{u}_h^{\mu, \nu}(t)]_j \xi_j(x), \quad (3.1.3)$$

with $x \in \Omega$, $t \in [0, 1]$, $(\mu, \nu) \in \Theta \times \Theta'$ identify the reduced solution of dimension N , where ξ_j are the basis functions obtained from the offline procedure in Algorithm (3) for $1 \leq j \leq N$. Then $\hat{u}_h^{\mu, \nu}(t) \in \mathbb{R}^N$ is the coefficient vector, that solves the reduced equation

$$\begin{cases} \mathbb{V}_{\text{rb}}^T M(\mu) \mathbb{V}_{\text{rb}} \hat{\partial}_t \hat{u}_h^{\mu, \nu}(t) + \mathbb{V}_{\text{rb}}^T A(\mu, \nu) \mathbb{V}_{\text{rb}} \hat{u}_h^{\mu, \nu}(t) = \mathbb{V}_{\text{rb}}^T f(t; \mu, \nu) & t \in \mathcal{T}, \\ \hat{u}_h^{\mu, \nu}(0) = u_0(\mu, \nu) & t = 0. \end{cases} \quad (3.1.4)$$

Note again, that $\mathbb{V}_{\text{rb}}$ is generally *not* $M(\mu)$ -orthonormal. The complexity of the offline computation for this Algorithm (3) is additive, and not multiplicative, in the number of training data $\mathcal{P}$. This can be an advantage, when compared to the pure POD-Galerkin approach.

Algorithm 3 POD-Greedy-Galerkin

Require: tol , $\mathcal{Z} = 0$, $N = 0$, $(\mu_j, \nu_j)_{j=1}^{N_s}$, N_1 and N_2

Ensure: approximated parameter-to-solution-map $(\mu, \nu) \mapsto \hat{u}_h(\cdot, \cdot; \mu, \nu)$

```

1: for  $j = 1, \dots, N_s$  do
2:   Compute  $\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}$  from Equation (3.0.4) (with  $N = 0$ )
3:    $(\zeta_j, \lambda_j)_{j=1}^{N_1} \leftarrow \text{POD}(\{u_h^{\mu_j, \nu_j, t_{k_m}}\}_{m=1}^{N_t}, N_1)$ 
4:    $\mathcal{Z} \leftarrow \{\mathcal{Z}, \{\zeta_1, \dots, \zeta_{N_1}\}\}$ 
5:    $N \leftarrow N + N_2$ 
6:   Compress:  $(\xi_j, \sigma_j)_{j=1}^n \leftarrow \text{POD}(\mathcal{Z}, N)$ 
7:    $(\mu_{j+1}, \nu_{j+1}) \leftarrow \arg \max_{(\mu, \nu) \in \mathcal{P}} \eta(\mu, \nu)$ 
8:   if  $\eta(\mu_{j+1}, \nu_{j+1}) > \text{tol}$  then
9:      $j \leftarrow j + 1$ 
10:  else
11:    break
12:  end if
13: end for
14:  $\mathbb{V}_{\text{rb}} \leftarrow \text{stack}(\xi_1, \dots, \xi_N)$ 
15: Build reduced Equation (3.1.4)
16: For each  $(\mu, \nu)$  solve Equation (3.1.4)
17: Recover  $\hat{u}_h(\cdot, \cdot; \mu, \nu)$  via Equation (3.1.3)
    
```

3.1.3 Non-linear Reduced Order Models

In this subsection, we want to briefly introduce relevant ROMs, that handle situations in which $N \neq 0$ according to Equation (3.0.2). For discrete problem derived from Finite Element Methods, such situations can be found, in case the associated weak form of the PPDE has a non-affine representation in the respective space in regard to the parameters. Each of these methods, try to overcome the issue of N_h -dimensional workflow in the N operator. This is straightforward, if one

would use a RB-method in order to substitute the Equation (3.0.2) in the same way as we have done in projection-based ROMs above. In such case, we would get

$$\begin{cases} M_{\text{red}}(\mu) \hat{\partial}_t u_{\text{red}}^{\mu, \nu}(t) + A_{\text{red}}(\mu, \nu) u_{\text{red}}^{\mu, \nu}(t) + V^T N(V u_{\text{red}}^{\mu, \nu}(t), \mu, \nu) = f_{\text{red}}(t; \mu, \nu) & t \in \mathcal{T}, \\ u_{\text{red}}^{\mu, \nu}(0) = V^T u_0(\mu, \nu) & t = 0, \end{cases}$$

with $\hat{u}_h^{\mu, \nu}(t) \approx V u_{\text{red}}^{\mu, \nu}(t)$ for each $t \in [0, T]$ and some reduction operator V . Hence, we would still rely on the high dimensional evaluation of the non-linear term. In the following, we will list some of the intrusive ROMs to overcome such evaluating, and give a brief description of their workflow.

Empirical Interpolation Method (EIM) In essence, the EIM leverages on interpolation functionals, which are defined iteratively. We minimize with regard to $\inf_{z \in W_{M-1}} \|N(\cdot, \mu, \nu) - z\|_{L^\infty(\Omega)}$, where W_{M-1} is the span of the last iteration, which appends all former ones. The start of this iteration is given by setting $W_1 := \text{span}(\xi_1)$ with $\xi_1 := N(u_h^{\mu_1, \nu_1, t_1}, \mu, \nu)$ for some solution snapshot. This constitutes finding a space, in which the non-affine (non-linear) operator N is approximated well by interpolation. After sufficient steps, we stop the creation of this space and take the wanted amount $m < N_h$ of interpolation functionals spanning that space. Then, we approximate with $U = [\xi_j]_{j=1}^m$ and $P \in \mathbb{R}^{N_h \times m}$ being the selection matrix for the non-linear term under a less costly interpolant

$$N(\hat{u}_h^{\mu, \nu}, \mu, \nu) \approx U(P^T U)^{-1} P^T N(\hat{u}_h^{\mu, \nu, t}, \mu, \nu).$$

In fact, the expensive $U(P^T U)^{-1}$ term can be computed offline, while only $m \ll N_h$ entries of $N(\hat{u}_h^{\mu, \nu}, \mu, \nu)$ need to be assembled online. The EIM has been introduced in (Barrault et al., 2004), but a more fitting definition for our framework can be found in (Quarteroni et al., 2015).

Gappy-POD Residual Minimization (GNAT) For simplicity and consistency with the literature's notation, we will denote with $R : \mathbb{R}^{N_h} \rightarrow \mathbb{R}^{N_h}$ given by

$$R(u) := M(\mu) \partial_t u + A(\mu, \nu) u + N(u, \mu, \nu) - f(t; \mu, \nu)$$

the residual of the problem in Equation (3.0.2). The GNAT aims at reducing the dimensionality of an arbitrary residual realization by using a Petrov-Galerkin projection. Thus, it limits the domain of the residual to a less complex, intrinsically low dimensional, subspace of $\mathbb{R}^{N_h}$, by using gappy POD and sampling. After this, it solves a least-squares problem with a Gauss-Newton method, i.e. it finds

$$\min_{u_{\text{red}}} \|P^T R(V u_{\text{red}})\|^2,$$

with $P \in \mathbb{R}^{N_h \times m}$ a basis selector and V the reduction matrix. For this refer to (Carlberg et al., 2013). However, the origin of the method can be found in (Carlberg et al., 2011).

3.2 Data-driven Reduced Order Models

In contrast to the introduced intrusive ROMs above, the *data-driven* alternatives aim to circumvent the direct use of the governing equations. As such, they allow a more general treatment.

Remark 3.4 (Historic Literature). The origins of data-driven ROMs can be traced back to techniques in fluid dynamics and control, where the objective was to extract coherent structures or dominant modes from experimental measurements without explicit reliance on the full Navier-Stokes equations. A first influential step was the development of the POD for experimental data, introduced

in the turbulence community by Lumley (Lumley, 1967), and further popularized in the coherent structure analysis of Sirovich (Sirovich, 1987). While the POD itself does not provide a dynamical law for the online phase, it laid the foundation for representing complex systems with only few empirical modes.

A decisive breakthrough was the *Dynamic Mode Decomposition* (DMD) (Schmid, 2010), which can be interpreted as a data-driven approximation of the infinite-dimensional Koopman operator (Rowley et al., 2009). DMD represents temporal evolution as a best-fit linear operator acting on snapshot data, thereby yielding both modal decompositions and a surrogate model for prediction. This was one of the first systematic attempts to construct a fully non-intrusive ROM.

In parallel, regression-based approaches have been developed to learn reduced operators directly from data, often under the name of *operator inference* (Peherstorfer et al., 2016), or in the form of interpolation-based reduced models, e.g. POD with interpolation (PODI) as in Barrault et al. (Barrault et al., 2004). These methods demonstrate that the reduced dynamics can be inferred without Galerkin projection, either by statistical regression or by interpolation in parameter space.

For this section we intentionally do not give an explicit training algorithm for any of the methods presented, since it can be inferred with an optimizer of the readers choice from the discussion in Subsection (2.3.5). With readability in mind, we thus only state the architecture of the network at hand and its loss function. Most importantly, we assume that some choice of basis for $\mathcal{H}_h$ is already done a priori to the training of the network and independent of $\mu \in \Theta$. Thus, we assume $\mathbb{G} := M(\mu)$ for all $\mu \in \Theta$ as our mass matrix. Under Equation (3.0.3) this also implies a μ -independent norm $\|\cdot\|$, which in fact allow us to use the POD matrix, if constructed with $\mathbb{G}$, as a $\mathbb{G}$ -orthogonal projector onto the span of $\mathbb{A}$ in an L^2 sense. Note, that this could be relaxed, if a general equivalent norm would be known without imposing a μ -independent basis.

For clarity, we formulate the following as an assumption, since the reduction systems presented can choose these dimensional numbers somewhat freely. They should however retain this hierarchy, to ensure comparability. Note, that the number of systems can make the dimensionality of the architectures confusing. The reader is therefore encouraged to revisit this assumption, whenever the general setting appears to be unclear.

Assumption 5 (Dimension Hierarchy). Let for all following data-driven Reduced Order Models, the following natural number hierarchy be fixed

$$n < N' < N \ll N_A \leq N_h.$$

Note, that " $\ll$ " is an unspecified relation, meaning to symbolize a "large" difference.

A general Deep Learning based ROM (DL-ROM) first appeared in architecture and training in the paper (Fresca et al., 2020). To properly approximate the potentially non-linear solution manifold of the high fidelity FOM, an Autoencoder (AE) is deployed to map the degrees of freedom to a coefficient dimension, from where some other neural network architecture, like the proposed Deep Feed-Forward Neural Network (DFNN), is used to learn the latent dynamics of the parametric PDE. This concludes in the following approximation of the FOM coefficient solution $u_h^{\mu,\nu,t} \in \mathbb{R}^{N_h}$ for each $\mu \in \Theta$, $\nu \in \Theta'$ and $t \in \mathcal{T}$

$$\hat{u}_h^{\mu,\nu,t} = (R_\rho(\Psi_h) \circ R_\rho(\Phi_n))(\mu, \nu, t) \in \mathbb{R}^{N_h}, \quad (3.2.1)$$

where we set $\Psi_h = f_h^D$ for the decoder architecture with input dimension n and output dimension N_h (and $\Psi'_n = f_n^E$ the corresponding encoder architecture) of a convolutional AE. Then Ψ'_n denotes the projection onto the reduced trial solution manifold, and we let $\Phi_n = \Phi_n^{DF}$ to be the DFNN

architecture of input dimension $p + q + 1$ and output dimension n , mapping the dynamics of the parametric PDE onto the reduced trial solution manifold.

The subsequent training is then performed via a loss function of a convex combination of both, the projection error using the encoder and the dynamics error considering both the decoder and the feed forward network. For this optimization problem, we define the trainable weights $\theta = (\theta_D, \theta_E, \theta_{DF}) \in \Theta(\Psi_h) \times \Theta(\Psi'_n) \times \Theta(\Phi_n)$ and the loss function

$$\begin{aligned} \mathcal{L}_{\text{DL-ROM}} \Big(R_\rho(P(\Psi_h \odot \Phi_n, \Psi'_n)(\theta)) | \{ \mu_i, \nu_j, k\Delta t \}_{i,j,k}, \{ u_h^{\mu_i, \nu_j, k\Delta t} \}_{i,j,k} \Big) = \\ \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} \frac{\omega_h}{2} \| u_h^{\mu_i, \nu_j, k\Delta t} - (R_\rho(\Psi_h(\theta_D)) \circ R_\rho(\Phi_n(\theta_{DF})))(\mu_i, \nu_j, k\Delta t) \|^2 \\ + \frac{1 - \omega_h}{2} | R_\rho(\Psi'_n(\theta_E))(u_h^{\mu_i, \nu_j, k\Delta t}) - R_\rho(\Phi_n^{\text{DF}}(\theta_{DF}))(\mu_i, \nu_j, k\Delta t) |^2. \end{aligned} \quad (3.2.2)$$

3.2.1 Proper Orthogonal Decomposition Deep Learning ROM

The main idea of the POD-DL-ROM, in contrast to the DL-ROM, is the additional employment of an outer a priori reduction of the training data using a POD. We base the following definition on (Fresca & Manzoni, 2022). Note, that we sway somewhat from the source, since we are trying to achieve an estimate, which is close to the high-fidelity solution in an L^2 -sense by applying the mass matrix $\mathbb{G}$ from the start of the section. Such a reduction yields great benefits, such as the scalability in N_h and reduction in computational cost for the offline training. This is done by employing a POD (see Algorithm (1)) on the stacked solution trajectories

$$U := \left[u_h^{\mu_1, \nu_1} | \dots | u_h^{\mu_{N_{s1}}, \nu_1} | \dots | u_h^{\mu_{N_{s1}}, \nu_{N_{s2}}} \right]^T \in \mathbb{R}^{N_h \times N_{\text{data}}},$$

of Definition (3.1), resulting in the POD matrix $\mathbb{A}_P \in \mathbb{R}^{N_h \times N}$ for $N < N_h$ from Assumption (5). Then we can define the reduced solution snapshots by computing

$$u_{\text{red}}^{\mu, \nu, t} := \mathbb{A}_P^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^N \quad \text{for all } (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}.$$

By $\mathbb{G}$ -orthonormality of $\mathbb{A}_P$, we can thus define the FOM approximate coefficient solution $\hat{u}_h^{\mu, \nu, t} \in \mathbb{R}^{N_h}$ for each $\mu \in \Theta$, $\nu \in \Theta'$ and $t \in \mathcal{T}$

$$\hat{u}_h^{\mu, \nu, t} = \mathbb{A}_P \cdot (R_\rho(\Psi_N) \circ R_\rho(\Phi_n))(\mu, \nu, t) \in \mathbb{R}^{N_h}, \quad (3.2.3)$$

where we set $\Psi_N = f_N^D$ for the decoder architecture with input dimension n and output dimension N (and $\Psi'_n = f_n^E$ the corresponding encoder architecture) of a convolutional AE. Then f_n^E denotes the projection onto the reduced trial solution manifold, and we let $\Phi_n = \Phi_n^{\text{DF}}$ be the DFNN architecture of input dimension $p + q + 1$ and output dimension n same as before.

The loss function has now the attribute of being fed only at most N -dimensional training data, and only evaluating at most N -dimensional euclidean norms, hence greatly increasing performance. The justification of using the euclidean norm here comes directly from the Proposition (2.12), where the averaging over all deterministic POD projection errors can be collectively ignored for training, without misrepresenting the solution as a bare vector in euclidean space.

Hence, we can define the optimization problem. Let the neural network weights $\theta = (\theta_D, \theta_E, \theta_{DF}) \in$

$\Theta(\Psi_N) \times \Theta(\Psi'_n) \times \Theta(\Phi_n)$ and the loss function be defined by

$$\begin{aligned} \mathcal{L}_{\text{POD-DL-ROM}} \Big(R_\rho(P(\Psi_N \odot \Phi_n, \Psi'_n)(\theta)) | \{ \mu_i, \nu_j, k\Delta t \}_{i,j,k}, \{ u_{\text{red}}^{\mu_i, \nu_j, k\Delta t} \}_{i,j,k} \Big) = \\ \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} \left(\frac{\omega_h}{2} | u_{\text{red}}^{\mu_i, \nu_j, k\Delta t} - (R_\rho(\Psi_N(\theta_D)) \circ R_\rho(\Phi_n(\theta_{DF}))) (\mu_i, \nu_j, k\Delta t) |^2 \right. \\ \left. + \frac{1 - \omega_h}{2} | R_\rho(\Psi'_n(\theta_E))(u_{\text{red}}^{\mu_i, \nu_j, k\Delta t}) - R_\rho(\Phi_n(\theta_{DF}))(\mu_i, \nu_j, k\Delta t) |^2 \right). \quad (3.2.4) \end{aligned}$$

3.2.2 Deep Orthogonal Decomposition based ROMs

A big advantage of the POD-DL-ROM is the fact, that under the assumption of a static basis $\{\xi_j\}_{j=1}^{N_h} \subset \mathcal{H}_h$ of $\mathcal{H}_h$ and its associated mass matrix $\mathbb{G}$ and $\mathcal{H}_h$ sensitive norm $\|\cdot\|$, we may reduce the high dimensional optimization problem to a more manageable computational complexity. However, this so far ignored the particular problem in sight. Assumption (1) guarantees a significant reduction cost, when employing a (μ, t) -independent linear projector. To circumvent this Kolmogorov N -width barrier for $N \ll N_h$, one can instead of using a POD for the choice of reduced basis, invoke a Neural Network based linear projector, that fits a basis superposition dependent on μ and t . This projector can thus produce a further reduction to $N' < N$ under a similar energy loss.

The following depicts an adaptive RB method.

Definition 3.5 (Deep Orthogonal Decomposition). Recollect the dimensional hierarchy of Assumption (5). Employ a POD (according to Algorithm (1)) with $\mathbb{G}$ given as above, using the stacked solution trajectories

$$U := \left[u_h^{\mu_1, \nu_1} | \dots | u_h^{\mu_{N_{s1}}, \nu_1} | \dots | u_h^{\mu_{N_{s1}}, \nu_{N_{s2}}} \right]^T \in \mathbb{R}^{N_h \times N_{\text{data}}}.$$

This results in the POD matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$. Now set up a Neural Network architecture $\Phi_{\tilde{\mathbb{V}}}$ with input dimension $p + 1$ and output dimension $N_A \times N'$, such that

$$\Phi_{\tilde{\mathbb{V}}} = \text{ORTH} \circ \text{STACK} \circ (P((R_1 \odot s), \dots, (R_N \odot s))),$$

where P is denoting the network parallelization operator as given in Definition (2.28). Here, we have

1. s is a Deep Feed Forward Neural Network architecture with input dimension $p + 1$ and output dimension l ,
2. $R_1, \dots, R_N$ are a collection of Deep Feed Forward Neural Network architectures with input dimensions l and output dimensions N_A ,
3. $\text{STACK} : \mathbb{R}^{N_A \cdot N'} \rightarrow \mathbb{R}^{N_A \times N'}$ is a canonical stacking operator,
4. $\text{ORTH} : \mathbb{R}^{N_A \times N'} \rightarrow \mathbb{R}^{N_A \times N'}$ is an orthonormalization unit, that orthonormalizes any given matrix $W \in \mathbb{R}^{N_A \times N'}$ into $\tilde{W} \in \mathbb{R}^{N_A \times N'}$ such that $\text{span}(W) = \text{span}(\tilde{W})$. This is most often done using a QR decomposition (Golub & Van Loan, 2013).

The realization of the network $\Phi_{\tilde{\mathbb{V}}}$ for some $\mu \in \Theta$, $t \in [0, T]$ and any chosen neural network weight $\theta_{\text{DOD}} \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ according to Definition (2.38) will be abbreviated with

$$\tilde{\mathbb{V}}_{\mu, t}(\theta_{\text{DOD}}) := R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}))(\mu, t). \quad (3.2.5)$$

For any choice of weights, this is called the *inner module of the Deep Orthogonal Decomposition*, or in short the inner DOD module.

The *Deep Orthogonal Decomposition* can hence be achieved by upward projection under the POD matrix $\mathbb{A}$ of the inner DOD module. In other words, a realization of the underlying DOD neural network for a choice of weights $\theta_{\text{DOD}} \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ is attained for $\mu \in \Theta$ and $t \in [0, T]$ by

$$\mathbb{V}_{\mu,t}(\theta_{\text{DOD}}) := \mathbb{A} \cdot \tilde{\mathbb{V}}_{\mu,t}(\theta_{\text{DOD}}) \in \mathbb{R}^{N_h \times N'}. \quad (3.2.6)$$

This is visualized in Figure (3.1).

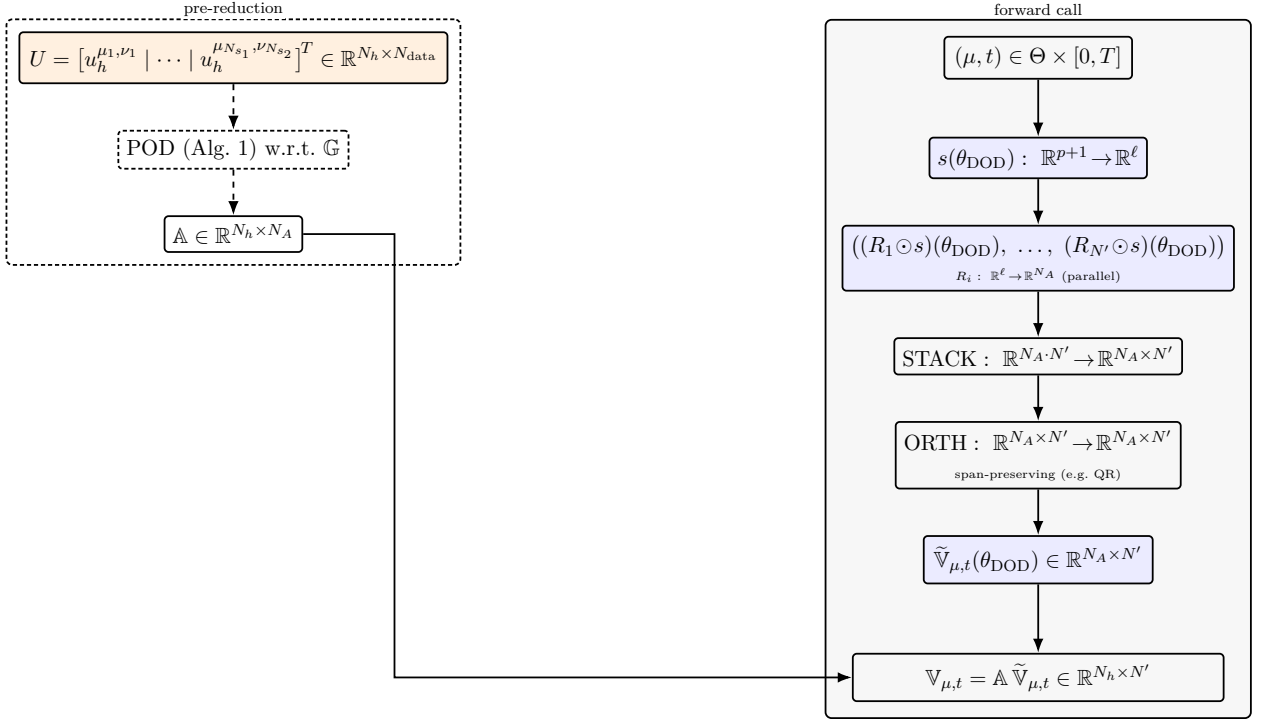


Figure 3.1: Realization of the Deep Orthogonal Decomposition for given $\theta_{\text{DOD}} \in \Theta(\Phi_{\tilde{\mathbb{V}}})$.

Nonetheless, this does not warrant a "good" DOD; just a defined one. For this one needs to assert a qualitative loss function and some data, which we will collect by pre-reduction in the same manner as before, i.e. let

$$u_{\text{pre-red}}^{\mu, \nu, t} := \mathbb{A}^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^{N_A} \quad \text{for all } (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}.$$

Then, we define the optimization problem as follows. Let $\theta = \theta_{\text{DOD}} \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ be the trainable weights, then we optimize using the following loss function

$$\begin{aligned} \mathcal{L}_{\text{DOD}} \left(R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta)) | \{ \mu_i, \nu_j, k \Delta t \}_{i,j,k}, \{ u_{\text{pre-red}}^{\mu_i, \nu_j, k \Delta t} \}_{i,j,k} \right) \\ = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} | u_{\text{pre-red}}^{\mu_i, \nu_j, k \Delta t} - \tilde{\mathbb{V}}_{\mu_i, k \Delta t}(\theta_{\text{DOD}}) \tilde{\mathbb{V}}_{\mu_i, k \Delta t}^T(\theta_{\text{DOD}}) u_{\text{pre-red}}^{\mu_i, \nu_j, k \Delta t} |^2. \end{aligned} \quad (3.2.7)$$

Note, that this is indeed again an N_A -dimensional loss function, thanks to the $\mathbb{G}$ -orthonormality of the POD matrix and Proposition (2.12). We will denote by $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ the optimized neural network weights.

Notice, that we have not yet performed any actual search for the approximate parameter-to-solution map $\mathcal{G}$. After all, the frozen DOD is a linear orthogonal projector, and not an estimation of $\mathcal{G}$. It can be seen as an analog to the POD.

In this paper, we propose two novel data-driven ROMs, based on the DOD, to find an approximate solution-to-parameter map. These are the *Deep Orthogonal Decomposition with Deep Feed-Forward Neural Network* (DOD+DFNN) and the *Deep Orthogonal Decomposition based Deep Learning ROM* (DOD-DL-ROM). The former indeed symbolizes a general approach hence including the latter, but, with some abuse of notation, we admit the DOD+DFNN a naive direct Deep Feed Forward architecture for the determination of the coefficient, while the DOD-DL-ROM is an adaptation of the DL-ROM architecture introduced in the header of the section.

DOD+DFNN

Let $\Phi_{N'} := \Phi_{N'}^{\text{DF}}$ be some Deep Feed-Forward Neural Network architecture with input dimension $p + q + 1$ and output dimension N' . Then the approximate parameter-to-solution map of the *DOD+DFNN* can be given for each $(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]$ via

$$\hat{u}_h^{\mu, \nu, t} = \mathbb{A} \cdot R_\rho(\Phi_{\tilde{\mathbb{V}}})(\mu, t) \cdot R_\rho(\Phi_{N'})(\mu, \nu, t) \in \mathbb{R}^{N_h}. \quad (3.2.8)$$

The training is done in series and not in parallel. We first find the optimal DOD parameter $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ by solving the optimization problem in Definition (3.5). This freezes the DOD, and we set $\mathbb{V}_{\mu, t} := \mathbb{V}_{\mu, t}(\theta_{\text{DOD}}^*)$ for $(\mu, t) \in \Theta \times [0, T]$.

Now, we can use the frozen DOD matrix to reduce the training data

$$u_{\text{red}^2}^{\mu, \nu, t} := \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^{N'} \quad \text{for all } (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}.$$

Let the optimization problem be given for any choice of neural network weights $\theta = \theta_{\text{DF}} \in \Theta(\Phi_{N'})$ by the loss function

$$\begin{aligned} \mathcal{L}_{\text{DOD+DNN}} & \left(R_\rho(\Phi_{N'}(\theta_{\text{DF}})) | \{ \mu_i, \nu_j, k\Delta t \}_{i,j,k}, \{ u_{\text{red}^2}^{\mu_i, \nu_j, k\Delta t} \}_{i,j,k} \right) \\ &= \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} |u_{\text{red}^2}^{\mu_i, \nu_j, k\Delta t} - R_\rho(\Phi_{N'}(\theta_{\text{DF}}))(\mu_i, \nu_j, k\Delta t)|^2. \end{aligned} \quad (3.2.9)$$

This is a rather obvious extension of (Franco et al., 2024), but works surprisingly well. It will be the basic benchmark to overcome for the following algorithms.

DOD-DL-ROM

Let $\Psi_{N'} := f_{N'}^{\text{D}}$ be a Decoder architecture with input dimension n and output dimension N' (and $\Psi_n' := f_n^{\text{E}}$ the corresponding Encoder architecture), and $\Phi_n := \Phi_n^{\text{DF}}$ be some Deep Feed-Forward Neural Network architecture with input dimension $p + q + 1$ and output dimension n . Then the approximate parameter-to-solution map of the *DOD-DL-ROM* can be given for each $(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]$ via

$$\hat{u}_h^{\mu, \nu, t} = \mathbb{A} \cdot R_\rho(\Phi_{\tilde{\mathbb{V}}})(\mu, t) \cdot (R_\rho(\Psi_{N'}) \circ R_\rho(\Phi_n))(\mu, \nu, t) \in \mathbb{R}^{N_h}. \quad (3.2.10)$$

Same as for the DOD+DNN, the training is done in series. We find the optimal DOD parameter $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ by solving the optimization problem in Definition (3.5). This again freezes the DOD,

and we set $\mathbb{V}_{\mu,t} := \mathbb{V}_{\mu,t}(\theta_{\text{DOD}}^*)$ for $(\mu, t) \in \Theta \times [0, T]$. One then observes, that the training data can be again reduced with

$$u_{\text{red}^2}^{\mu,\nu,t} := \mathbb{V}_{\mu,t}^T \mathbb{G} u_h^{\mu,\nu,t} \in \mathbb{R}^{N'} \quad \text{for all } (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}.$$

Let the optimization problem be given for any choice of neural network weights $\theta = (\theta_{\text{DOD}}, \theta_{\text{D}}, \theta_{\text{DF}}) \in \Theta(\Phi_{\tilde{\mathbb{V}}}) \times \Theta(f_N^{\text{D}}) \times \Theta(\Phi_n^{\text{DF}})$ by the loss function

$$\begin{aligned} \mathcal{L}_{\text{DOD-DL-ROM}} \left(R_\rho(P(\Psi_{N'} \odot \Phi_n, \Psi'_n)(\theta)) | \{\mu_i, \nu_j, k\Delta t\}_{i,j,k}, \{u_{\text{red}^2}^{\mu_i, \nu_j, k\Delta t}\}_{i,j,k} \right) = \\ \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} \left(\frac{\omega_h}{2} |u_{\text{red}^2}^{\mu_i, \nu_j, k\Delta t} - (R_\rho(\Psi_{N'}(\theta_{\text{D}})) \circ R_\rho(\Phi_n(\theta_{\text{DF}}))) (\mu_i, \nu_j, k\Delta t)|^2 \right. \\ \left. + \frac{1 - \omega_h}{2} |R_\rho(\Psi'_n(\theta_{\text{E}}))(u_{\text{red}^2}^{\mu_i, \nu_j, k\Delta t}) - R_\rho(\Phi_n^{\text{DF}}(\theta_{\text{DF}}))(\mu_i, \nu_j, k\Delta t)|^2 \right). \quad (3.2.11) \end{aligned}$$

Notice, that this reduction allows us to elect an algorithm, which uses only the n - and N' -dimensional euclidean norm, therefore enabling even less computational complexity for each training epoch, under the assumption of an already optimized DOD.

3.2.3 A CoLoRA inspired approach

In this subsection, we will introduce a novel architecture based on the Continuous Low Rank Adaptation (CoLoRA) (Berman & Peherstorfer, 2024). It will use the stationary version of the Deep Orthogonal Decomposition and a supplementing Dense Forward Neural Network, defined in (Franco et al., 2024). This will be done to achieve a solution approximation of the stationary limit case of the problem. This "first guess" is then continuously adapted during the online phase using CoLoRA layers. Notice, that such a procedure only makes sense in a case, where the problem has a meaningful stationary limit case.

For this, let for all $\mu \in \Theta$ and $\nu \in \Theta'$

$$\begin{cases} A(\mu, \nu) u_{h,0}^{\mu,\nu} + N(u_{h,0}^{\mu,\nu}, \mu, \nu) = 0 \\ u_{h,0}^{\mu,\nu} = u_D^{\mu,\nu} \quad \text{on some boundary} \end{cases} \quad (3.2.12)$$

define a fully discretized stationary problem. Note, that the boundary term could also be of some other type like Neumann or Robin. We assume, that this problem is closely related to the time-dependent version, in a sense that

$$\lim_{t \rightarrow T} u_h^{\mu,\nu,t} = u_{h,0}^{\mu,\nu}, \quad \text{for all } (\mu, \nu) \in \Theta \times \Theta'.$$

Then we define the stationary model using the DOD Definition (3.5) with $[0, T] = \{T\}$. The pre-reduction here is taken as a POD $\mathbb{A}_0 \in \mathbb{R}^{N_h \times N_A}$ of the stationary snapshot matrix

$$U_0 = \left[u_{h,0}^{\mu_1, \nu_1} | \dots | u_{h,0}^{\mu_{N_{s1}}, \nu_1} | \dots | u_{h,0}^{\mu_{N_{s1}}, \nu_{N_{s2}}} \right]^T \in \mathbb{R}^{N_h \times N_s}.$$

We can then produce some stationary reduced data $u_{\text{pre-red},0}^{\mu,\nu} := \mathbb{A}_0^T \mathbb{G} u_{h,0}^{\mu,\nu} \in \mathbb{R}^{N_A}$ for $\mu \in \mathcal{P}_1$ and $\nu \in \mathcal{P}_2$, where $u_{h,0}^{\mu,\nu} \in \mathbb{R}^{N_h}$ are the solutions of Equation (3.2.12). This data is used to find the optimal neural network weights $\theta_{\text{sDOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}}^{\text{stat}})$ for the stationary DOD, thus freezing it. We then denote with $\tilde{\mathbb{V}}_\mu := R_\rho(\Phi_{\tilde{\mathbb{V}}}^{\text{stat}}(\theta_{\text{sDOD}}^*))(\mu)$ any realization of the inner stationary DOD module

as given in (Franco et al., 2024). As a next step, we define a DFNN $\Phi_{N'}^{\text{stat}}$ in an analogous way to the DOD+DFNN subsection above, to attain for any input choice $(\mu, \nu) \in \Theta \times \Theta'$ the total approximation for a stationary solution of the form

$$\hat{u}_{h,0}^{\mu,\nu} = \mathbb{A}_0 \cdot \tilde{\mathbb{V}}_\mu \cdot R_\rho(\Phi_{N'}^{\text{stat}})(\mu, \nu) \in \mathbb{R}^{N_h}.$$

Then let the neural network weights $\theta_{\text{sDFNN}} \in \Theta(\Phi_{N'}^{\text{stat}})$ and the loss function of the corresponding optimization problem be given in an analogous way to the DOD+DFNN. Solving it, will freeze the stationary DFNN as well. This lets us write the frozen stationary approximation for the optimal weights $\theta_{\text{sDFNN}}^* \in \Theta(\Phi_{N'}^{\text{stat}})$ as

$$\hat{u}_{\text{pre-red},0}^{\mu,\nu} := \tilde{\mathbb{V}}_\mu \cdot R_\rho(\Phi_{N'}^{\text{stat}}(\theta_{\text{sDFNN}}^*))(\mu, \nu) \in \mathbb{R}^{N_A},$$

for any $(\mu, \nu) \in \Theta \times \Theta'$. Following this, we now implement a CoLoRA neural network architecture, which defines neural networks in terms of CoLoRA layers.

Definition 3.6 (CoLoRA network). Let $N_A, L, R \in \mathbb{N}$ with $R \ll N_A$, then a *CoLoRA-layered neural network* Φ_α with input dimensional N_A and L layers is defined via

$$\Phi_\alpha = ((W_1, A_1, B_1, b_1), \dots, (W_L, A_L, B_L, b_L)),$$

such that $N_0 = N_A$ and $N_1, \dots, N_L \in \mathbb{N}$, where W_l is a $(N_l \times \sum_{k=0}^{l-1} N_k)$ -, A_l a $(N_l \times R)$ -, B_l a $(R \times \sum_{k=0}^{l-1} N_k)$ -dimensional matrix, and $b_l \in \mathbb{R}^{N_l}$ for each $l = 1, \dots, L$. We then denote with

$$R_\rho(\Phi_\alpha) : \Theta' \times [0, T] \times \mathbb{R}^{N_A} \rightarrow \mathbb{R}^{N_L}$$

be a *realization* of Φ_α with activation function ρ and latent state α for some $\rho : \mathbb{R} \rightarrow \mathbb{R}$ and $\alpha = [\alpha_l]_{l=1}^L$, with $\alpha_l : \Theta' \times [0, T] \rightarrow \mathbb{R}$ some continuous map for all $l = 1, \dots, L$, as the result of the following scheme for $\nu \in \Theta', t \in [0, T]$ and $x \in \mathbb{R}^{N_A}$

$$\begin{aligned} x_0 &:= x, \\ x_l &:= \rho(W_l x_{l-1} + \alpha_l(\nu, t) A_l B_l x_{l-1} + b_l), \quad \text{for } l = 1, \dots, L-1, \\ x_L &:= W_L x_{L-1} + \alpha_L(\nu, t) A_L B_L x_{L-1} + b_L. \end{aligned}$$

Finally, with this definition, we can set up the total workflow of the proposed method. For the latent state α , we use a simple realization of an underlying neural network, a DFNN, where we set up a fixed layer size for each of them. This strategy is simply lifted from (Berman & Peherstorfer, 2024), but as the authors have mentioned there too, this can be replaced by some other continuous mapping.

The approximate solution to the time-dependent problem in Equation (3.0.4) is given by the realization of a CoLoRA network Φ_α , and a lift using the time-trajectory sensitive POD, i.e.

$$\hat{u}_h^{\mu,\nu,t} = \mathbb{A} \cdot R_\rho(\Phi_\alpha)(\nu, t, \hat{u}_{\text{pre-red},0}^{\mu,\nu}) \in \mathbb{R}^{N_h}.$$

Since, the trainable network of this method is defined again on the span of the N_A -dimensional POD matrix, we can set up the reduced training data as

$$u_{\text{pre-red}}^{\mu,\nu,t} := \mathbb{A}^T \mathbb{G} u_h^{\mu,\nu,t} \in \mathbb{R}^{N_A} \quad \text{for all } (\mu, \nu, t) \in \mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}.$$

Let the optimization problem be given for any choice of neural network weights $\theta = \theta_\alpha \in \Theta(\Phi_\alpha)$ by the loss function

$$\begin{aligned} \mathcal{L}_{\text{CoLoRA}} \left(R_\rho(\Phi_\alpha(\theta)) | \{ \mu_i, \nu_j, k \Delta t \}_{i,j,k}, \{ u_{\text{pre-red}}^{\mu_i, \nu_j, k \Delta t} \}_{i,j,k} \right) \\ = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} |u_{\text{pre-red}}^{\mu_i, \nu_j, k \Delta t} - R_\rho(\Phi_\alpha(\theta_\alpha))(\nu_j, k \Delta t, \hat{u}_{\text{pre-red},0}^{\mu_i, \nu_j})|^2. \end{aligned} \quad (3.2.13)$$

Remark 3.7 (Heuristic Discussion). The idea of this adapted approach is that many partial differential equations, such as parabolic ones, often have an intrinsically linked stationary limit. Further, specifically parabolic equations tend to gain smoothness in growing time. Hence, the continuous adaptation of an initial approximate solution might make sense. Additionally, to counter the KnW barrier, we employ the DOD method for a cheap reduction in case of a hybrid parameter problem.

Chapter 4

Approximation Theory for Reduced Order Models

This chapter provides the main results of this thesis based on the data-driven ROMs introduced in Chapter (3). As such, we repeat that in order to execute the reduction for the norm $\|\cdot\|$ defined in that chapter, we require a mass matrix, that is independent of $\mu \in \Theta$, since otherwise the orthonormality with regard to it cannot be computed under the generalized POD Algorithm (1). However, we note, that the paper (Brivio et al., 2023) we base most of this chapter on, and several other works dealing with a similar setup, instead neglect the representation in the underlying Hilbert space and use a simple euclidean norm for the coefficient vectors. For our situation though, we let

$$\hat{u}_h(x, t; \mu, \nu) = \sum_{j=1}^{N_h} [\hat{u}_h^{\mu, \nu}(t)]_j \varphi_j(x), \quad (4.0.1)$$

for all $x \in \Omega$, $t \in [0, T]$ and parameters $(\mu, \nu) \in \Theta \times \Theta'$, which simultaneously specifies $[\mathbb{G}]_{ij} = \langle \varphi_i, \varphi_j \rangle_{\mathcal{H}_h} = [M(\mu)]_{ij}$ as the mass matrix. The norm for this chapter will be, if not stated otherwise, the norm from Equation (3.0.3), so here

$$\|u\| := \sqrt{u^T \mathbb{G} u},$$

for all $u \in \mathbb{R}^{N_h}$. Furthermore, the solution vector $\hat{u}_h^{\mu, \nu}(t) = \hat{u}_h^{\mu, \nu, t} \in \mathbb{R}^{N_h}$ is set to be constant between the time instants in our model, i.e. for all $t \in [(k-1)\Delta t, k\Delta t)$ for some $k = 1, \dots, N_t$. As a goal of this chapter, we will provide upper bounds on the complexity of the networks making up the data-driven ROMs, where we especially outline a comparison between the POD-DL-ROM and the newly introduced DOD-DL-ROM.

In order to analyze any error occurring from a data-based approximation, we need to set up a probability space, in whose sense a convergence under an increased data set can be viewed. This is to be regarded as a clarification of Assumption (2).

Definition 4.1 (Sampling). Let $(\mathcal{P}, \mathcal{B}(\mathcal{P}))$ be a Borel-measurable parameter space and $(\Omega_{\mathbb{P}}, \mathcal{F}, \mathbb{P})$ some underlying probability space. Then we refer to an *independent sampling of size N* , if for each $i \in \{1, \dots, N\}$ the random variable mapping attaining a sample (μ_i, ν_i)

$$X_i : (\Omega_{\mathbb{P}}, \mathcal{F}, \mathbb{P}) \rightarrow (\mathcal{P}, \mathcal{B}(\mathcal{P}))$$

is measurable, its push forward $\mathbb{P}^{X_i}$ admits the same distribution $\mathbb{P}^X$, i.e.

$$\mathbb{P}[X_i \in A] = \mathbb{P}^{X_i}[A] = \mathbb{P}^X[A],$$

and the samples are independent. We further name it *independent and identically distributed sampling*, if the distribution $\mathbb{P}^X$ is the uniform distribution on $\mathcal{P}$.

With this in mind, we can view sampling as the realization of a random variable, that provides an element of the sample space. Therefore, any mention of a probability space or a probability measure $\mathbb{P}$ refers to the sampling procedure in the probability space as given in Definition (4.1).

The upcoming proposition is a necessary result for the following chapters, as it provides us with a practical bound on the needed number of samples.

Proposition 4.2. Let $f \in L^2(\Theta \times \Theta' \times [0, T])$. Under Assumption (2), it is true, that

$$\mathbb{E} \left| \int_{\Theta \times \Theta' \times [0, T]} f(\mu, \nu, t) d(\mu, \nu, t) - \frac{|\Theta \times \Theta' \times [0, T]|}{N_{\text{data}}} \sum_{i=1}^{N_{s1}} \sum_{j=1}^{N_{s2}} \sum_{k=1}^{N_t} f(\mu_i, \nu_j, k\Delta t) \right| \leq \mathcal{O}(N_s^{-1/4} + N_t^{-1/2}).$$

Proof. See the Appendix of (Brivio et al., 2023). $\square$

4.1 Error Decomposition Formulas

To adequately start analyzing the ROMs in question, one needs to recognize the different aspects of "error" inherent to their deployment. Namely, we can subdivide these separate instances of inaccuracy occurring in the process of both, the POD-based data-driven ROM and the DOD-based data-driven ROM, into an error caused by the imperfect recollection of the parameter spaces Θ and Θ' , an error caused by the imperfect projection given by the DOD or POD respectively and an error of imperfect attribution of latent dynamics on that projection space by a neural network. This separation can be achieved similarly for both types of algorithms. Note, that the general handling of "DOD-based"(DOD+DNN) or "POD-based"(POD+DNN) refers to any architecture, relying on a projection by DOD or by POD and using a neural network of any sort to fit the coefficients for the modes.

Before we start with the formulation and proof of these decompositions, we want to predefine some elements of the corresponding theorems, to make them less cumbersome to read. The visual portrayal of this workflow is given by Figure (4.1).

Firstly let in general n, N', N, N_A and N_h be the fixed natural number hierarchy from Assumption (5). Let the approximate POD-based parameter-to-solution map be defined as follows

$$\mathcal{G}_{\text{POD+DNN}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \mathbb{A}_P \cdot \hat{q}_P(\mu, \nu, t),$$

where we let $\mathbb{A}_P \in \mathbb{R}^{N_h \times N}$ be a POD computed by Algorithm (1) for the dimension N with the mass matrix $\mathbb{G}$, and

$$\hat{q}_P := R_\rho(\Phi_N^P(\theta_{\text{PDNN}})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^N$$

be the realization of some neural network Φ_N^P with input dimension $p+q+1$ and output dimension N , and any neural network weights $\theta_{\text{PDNN}} \in \Theta(\Phi_N^P)$. And let similarly the approximate DOD-based parameter-to-solution map be defined as follows

$$\mathcal{G}_{\text{DOD+DNN}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \mathbb{V}_{\mu, t} \cdot \hat{q}_D(\mu, \nu, t),$$

where we let

$$\mathbb{V}_{\mu, t} := \mathbb{A} \cdot \tilde{\mathbb{V}}_{\mu, t}(\theta_{\text{DOD}}) \in \mathbb{R}^{N_h \times N'}, \quad \text{for } (\mu, t) \in \Theta \times [0, T],$$

and any neural network weights $\theta_{\text{DOD}} \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ according to Definition (3.5) and the used POD matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$, and

$$\hat{q}_D := R_\rho(\Phi_{N'}^D(\theta_{\text{DDNN}})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N'}$$

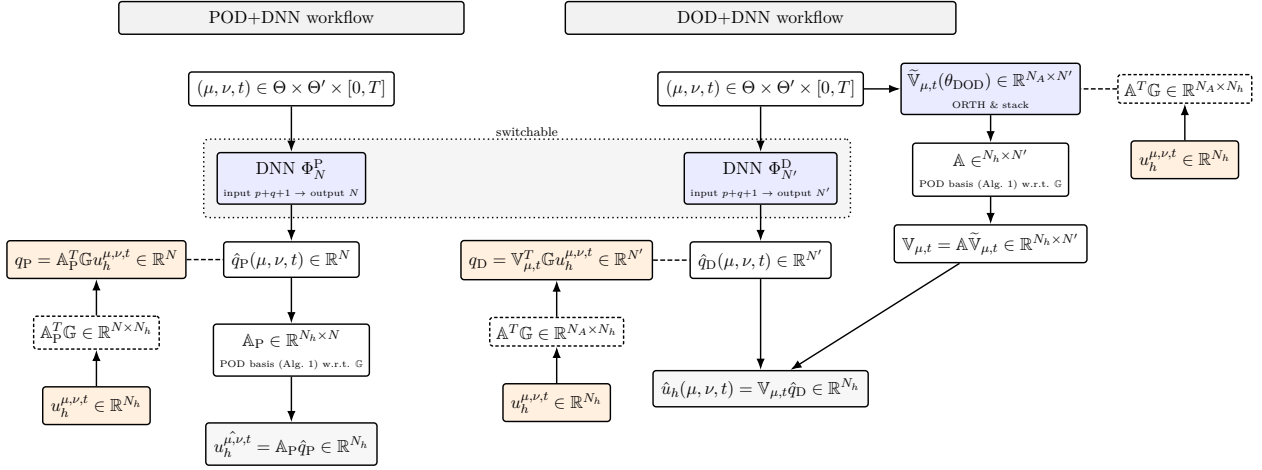


Figure 4.1: Dimensional workflow for POD+DNN (left) and DOD+DNN (right). Solid arrows show the inference; dashed lines denote training.

be the realization of some neural network $\Phi_{N'}^D$, with input dimension $p + q + 1$ and output dimension N' , and any neural network weights $\theta_{\text{DDNN}} \in \Theta(\Phi_{N'}^D)$.

Moreover, we assume for this section to have after reordering and with $N_{\text{data}} = N_{s_1} N_{s_2} N_t$

$$\{(\mu_j, \nu_j, t_j), u_j\}_{j=1}^{N_{\text{data}}} := \left\{(\mu_i, \nu_j, k\Delta t), u_h^{\mu_i, \nu_j, k\Delta t}\right\}_{(i,j,k)=(1,1,1)}^{(N_{s_1}, N_{s_2}, N_t)} \subset (\Theta \times \Theta' \times [0, T]) \times \mathbb{R}^{N_h}$$

a set of training data conforming to the label and feature notation in Definition (2.36), where the labels have been sampled in accordance to Assumption (2).

For both algorithms, the relative error can be given as

$$\mathcal{E}_R := \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|u_h^{\mu, \nu, t} - \hat{u}_h^{\mu, \nu, t}\|^2}{\|u_h^{\mu, \nu, t}\|^2} d(\mu, \nu, t) \right)^{1/2}, \quad (4.1.1)$$

where $\hat{u}_h^{\mu, \nu, t} \in \mathbb{R}^{N_h}$ can be given either with regard to the approximate parameter-to-solution map of the DOD+DNN or the POD+DNN.

Theorem 4.3 (POD+DNN Decomposition). *Define the preliminaries such as in the introduction to this Section (4.1). Then under the Assumptions (2) and (3), we have for the relative error of the POD+DNN*

$$\mathcal{E}_R^{\text{POD+DNN}} \leq \mathcal{E}_S + \mathcal{E}_{\text{POD}} + \mathcal{E}_{\text{NN}},$$

where

1. The sampling error

$$\mathcal{E}_S = \mathcal{E}_S(\mathcal{G}, (\mu_i, \nu_i, t_i)_{i=1}^{N_{\text{data}}}, N)$$

satisfies $\mathcal{E}_S \rightarrow 0$ as $N_s, N_t \rightarrow \infty$, with expectation

$$\mathbb{E}[\mathcal{E}_S] = \mathcal{O}(N_s^{-1/4} + N_t^{-1}).$$

2. The POD projection error

$$\mathcal{E}_{POD} = \mathcal{E}_{POD} \left(\mathcal{G}, (\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}, N \right)$$

satisfies $\mathcal{E}_{POD} \rightarrow \mathcal{E}_{POD,\infty}$ almost surely as $N_s, N_t \rightarrow \infty$, where

$$\mathcal{E}_{POD,\infty} = \mathcal{E}_{POD,\infty}(\mathcal{G}, N)$$

is independent of the data snapshots and proportional to the linear KnW of the manifold $\mathcal{S} = \{u_h^{\mu,\nu,t} \mid (\mu, \nu, t) \in \Theta' \times [0, T]\}$.

3. The neural network approximation error

$$\mathcal{E}_{NN} = \mathcal{E}_{NN} \left(\mathcal{G}, N, \theta_{PDNN}, \Phi_N^P \right)$$

is arbitrarily small depending on the size and choice of weights of the neural network Φ_N^P .

Proof. Let

$$\|\cdot\|_{L_\omega^2} := \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|\cdot\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \right)^{1/2},$$

define a map $\|\cdot\|_{L_\omega^2} : L^2(\Theta \times \Theta' \times [0, T]; \mathbb{R}^{N_h}) \rightarrow \mathbb{R}^+$. Then this is a norm. We coarsely proof this, by remarking the inheritance of the triangle inequality from the $\|\cdot\|$ under the usage of the Minkowski inequality, i.e. for arbitrary $v, w \in L^2(\Theta \times \Theta' \times [0, T]; \mathbb{R}^{N_h})$

$$\begin{aligned} \|v + w\|_{L_\omega^2}^2 &= \int_{\Theta \times \Theta' \times [0, T]} \frac{\|v(\mu, \nu, t) + w(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \\ &\leq \int_{\Theta \times \Theta' \times [0, T]} \frac{\|v(\mu, \nu, t)\|^2 + \|w(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) = \|v\|_{L_\omega^2}^2 + \|w\|_{L_\omega^2}^2, \end{aligned}$$

and the absolute homogeneity from simple qualities of the integral. Finally, let $\|v\|_{L_\omega^2} = 0$ for some $v \in L^2(\Theta \times \Theta' \times [0, T])$, then assuming $v \neq 0$ yields with Assumption (3)

$$\|v\|_{L_\omega^2}^2 \geq M^{-2} \int_{\Theta \times \Theta' \times [0, T]} \|v(\mu, \nu, t)\|^2 d(\mu, \nu, t) > 0,$$

hence reaching a contradiction. With $\|\cdot\|_{L_\omega^2}$ being a norm, we have

$$\mathcal{E}_R = \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \right)^{1/2} \quad (4.1.2)$$

$$\leq \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t}\|_{L_\omega^2} + \|\mathbb{A}_P \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t} - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|_{L_\omega^2}. \quad (4.1.3)$$

Let $q_P(\mu, \nu, t) := \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t} \in \mathbb{R}^N$ be a reduced solution according to the POD basis. Then the natural definition of the neural network approximation error is

$$\mathcal{E}_{NN} = \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|\mathbb{A}_P q_P(\mu, \nu, t) - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \right)^{1/2}. \quad (4.1.4)$$

This marks the second part in (4.1.3). This is only dependent on the ability of the neural network to identifying the best choice on a given linear sub-manifold of the solution manifold. The first

part is concerned with the error generated from the best *possible* approximation of such a linear sub-manifold.

For this, we can bound the norm $\|\cdot\|_{L_\omega^2} \leq m^{-1}\|\cdot\|$, where $m > 0$ is the constant stemming from Assumption (3). Further we remind ourselves of the definition of the discrete correlation matrix $K = |\Theta \times \Theta \times \mathcal{T}| N_{data}^{-1} U G U^T$ and its eigenvalues σ_k^2 in (2.13). Using $\sqrt{a+b} \leq \sqrt{a} + \sqrt{b}$ for all $a, b \geq 0$, we get

$$\begin{aligned}
 & \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|_{L_\omega^2} \\
 & \leq m^{-1} \left(\int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) \right)^{1/2} \\
 & \leq m^{-1} \left(\int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 + \sum_{k>N} \sigma_k^2 \right)^{1/2} \\
 & \leq m^{-1} \left(\left| \int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 \right| + \sum_{k>N} \sigma_k^2 \right)^{1/2} \\
 & \leq m^{-1} \underbrace{\left| \int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) - \sum_{k>N} \sigma_k^2 \right|^{1/2}}_{=: \mathcal{E}_S} + m^{-1} \underbrace{\sqrt{\sum_{k>N} \sigma_k^2}}_{=: \mathcal{E}_{POD}}.
 \end{aligned}$$

This concludes the estimate up to the facts about the convergence of the errors. Recalling Proposition (2.14), we can rewrite the sampling error as follows

$$\mathcal{E}_S = m^{-1} \left| \int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) - \frac{|\Theta \times \Theta' \times [0,T]|}{N_{data}} \sum_{j=1}^{N_{data}} \|u_j - \mathbb{A}_P \mathbb{A}_P^T G u_j\|^2 \right|^{1/2}$$

Furthermore, we define, using the compactness postulated in Assumption (2) and the boundedness in Assumption (3), the L^2 -error map via

$$f(\mu, \nu, t) := \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T G u_h^{\mu,\nu,t}\|^2 \leq M^2 \|I_{N_h} - \mathbb{A}_P \mathbb{A}_P^T G\|_{\text{op}}^2 = M^2 \|I_{N_A} - \mathbb{A}_P\|_{\text{op}} < \infty.$$

Thus, with $f \in L^2(\Theta \times \Theta' \times [0, T])$, we know that by Proposition (4.2), $\mathbb{E}[\mathcal{E}_S] \leq \mathcal{O}(N_s^{-1/4} + N_t^{-1/2})$.

By the strong law of large numbers, we can thus conclude, that $\mathcal{E}_S \rightarrow 0$ almost surely, as $N_t, N_s \rightarrow \infty$, and further by Proposition (2.16), we have that $\mathcal{E}_{POD} \rightarrow \mathcal{E}_{POD,\infty}$ as $N_t, N_s \rightarrow \infty$, where

$$\mathcal{E}_{POD,\infty} = m^{-1} \sqrt{\sum_{k>N} \sigma_{k,\infty}^2}. \quad (4.1.5)$$

□

Even optimistically, the upper error bound decomposition of Theorem (4.3) suggest for an ever-increasing number of data points $N_s, N_t \rightarrow \infty$, and an ever-increasing complexity for the Deep Neural Network underlying the realization of $\hat{q}_P$, an error of order of the Kolmogorov N -width of the solution manifold, since the DNN cannot overcome the unavoidable loss to even the perfect POD choice. Intuitively, the specific proportionality constant will be larger if the quotient between "small" regions of the solution space to the "large" regions of the solution space is big.

This observation can be reformulated into the following theorem.

Theorem 4.4. *Under the assumptions of Theorem (4.3), we have that*

$$\mathcal{E}_R^{POD+DNN} \geq \frac{m}{M} \mathcal{E}_{POD,\infty},$$

where $\mathcal{E}_{POD,\infty} := m^{-1} \sqrt{\sum_{k>N} \sigma_{k,\infty}^2}$.

Proof. We can simply start from the definition and argue, that under Proposition (2.14) the projection coefficients are chosen at best sub-optimal, i.e.

$$\mathcal{E}_R^2 = \int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \geq \int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t}\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t),$$

where $\mathbb{A}_P \in \mathbb{R}^{N_h \times N}$ is the POD matrix as given by Definition (2.13). Then starting from the other direction for a POD matrix $\mathbb{A}_\infty$ under the continuous correlation matrix, we get under the definition of the proof of Theorem (4.3)

$$(\mathcal{E}_{POD,\infty})^2 = m^{-2} \sum_{k>N} \sigma_{k,\infty}^2 \quad (4.1.6)$$

$$= m^{-2} \int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_\infty \mathbb{A}_\infty^T \mathbb{G} u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) \quad (4.1.7)$$

$$\leq m^{-2} \int_{\Theta \times \Theta' \times [0,T]} \|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) \quad (4.1.8)$$

$$= m^{-2} \int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{A}_P \mathbb{A}_P^T \mathbb{G} u_h^{\mu,\nu,t}\|^2}{\|u_h^{\mu,\nu,t}\|^2} \|u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) \quad (4.1.9)$$

$$\leq \frac{M^2}{m^2} (\mathcal{E}_R)^2, \quad (4.1.10)$$

where we use the Proposition (2.14) for the continuous POD matrix in Step (4.1.7), the optimality of the same proposition for Step (4.1.8) and the first equation of this proof in Step (4.1.10) with the rather obvious use of the bound on $\|u_h^{\mu,\nu,t}\|$ from Assumption (3). $\square$

We have now discussed some general relative error results for any POD+DNN for some given neural network weights. As a next step, one can do the same for the DOD+DNN. First, we need an auxiliary statement, that precedents the similarity of the definition of the DOD projection error to the POD projection error, since the former depends on the architecture of the inner DOD module and thus must be at least sub-optimal to some degree.

The key idea here being, that one can employ Horniks Theorem to ensure existence of a neural network architecture and corresponding neural network of such architecture, such that it finds $s(\mu, t)$ minimizing the objective functional

$$(\mu, t, \mathbb{V}) \mapsto \int_{\Theta'} \|u_h^{\mu,\nu,t} - \mathbb{V}_{\mu,t} \mathbb{V}_{\mu,t}^T \mathbb{G} u_h^{\mu,\nu,t}\|^2 d(\nu),$$

for any $(\mu, t) \in \Theta \times [0, T]$. Under triangle equality for the distance to the optimal minimizer of the objective functional, this can be separated into the mistake of fitting the network to the minimizer, which is arbitrarily small, and the mistake of the minimizer, that is nothing but the Kolmogorov N' -width under fixed (μ, t) .

Remark 4.5. Generally we can *not* employ Yarotsky Theorem (2.34) here, since even though the objective functional, as described above, can be shown to be Lipschitz continuous under our current assumptions, as given above, its minimizer with regard to the matrix entry does not necessarily inherit any regularity and Yarotsky does require Sobolev-type smoothness. However, we will set up the proof for the following lemma in such a way, that Yarotsky can be directly interchanged, if a higher regularity for the optimal DOD is imposed.

Before we start, we would like to define the aforementioned objective functional and its minimizer, since further assumptions on the PDE can expose the needed further regularity.

The source for the following proposition and its framework is generally (Franco et al., 2024), even though we have specified the proofs in accordance to their proposed definition of the DOD. It now also reflects the pre-reduction loss separately.

Definition 4.6 (Optimal DOD). Let $N', N_A, N_h \in \mathbb{N}$ be fixed by Assumption (5). Assuming we have $\mu \in \Theta, t \in [0, T]$ and $\mathbb{V} = \mathbb{A}\tilde{\mathbb{V}} \in [-1, 1]^{N_h \times N'}$, with $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ fixed being $\mathbb{G}$ -orthonormal and hence $\|\mathbb{A}\|_{\text{op}} = 1$ by Lemma (2.6), and $\tilde{\mathbb{V}} \in [-1, 1]^{N_A \times N'}$, then we define $J : \Theta \times [0, T] \times [-1, 1]^{N_h \times N'} \rightarrow \mathbb{R}$ the *objective DOD functional* via

$$J(\mu, t, \mathbb{V}) := \int_{\mathcal{X}} \|u_h^{\mu, \nu, t} - \mathbb{V}\mathbb{V}^T \mathbb{G}u_h^{\mu, \nu, t}\|^2 d(\nu),$$

for any compact set $\mathcal{X} \subset \Theta'$ with $\dim(\mathcal{X}) = q$. Then we call a Borel measurable map $s : \Theta \times [0, T] \rightarrow [-1, 1]^{N_A \times N'}$ producing w.l.o.g. $\mathbb{G}$ -orthonormal matrices given by

$$s : (\mu, t) \mapsto \underset{\mathbb{V} \in [-1, 1]^{N_A \times N'}}{\operatorname{argmin}} J(\mu, t, \mathbb{A}\mathbb{V}),$$

the *optimal DOD* for $\mathcal{X}$, if it is uniquely defined.

Under the already given assumptions, we can pose its first existence and an important smoothness result.

Lemma 4.7. Let $J : \Theta \times [0, T] \times [-1, 1]^{N_h \times N'} \rightarrow \mathbb{R}$ be an objective DOD functional for some $\mathcal{X} \subset \Theta'$ with $\dim(\mathcal{X}) = q$ as in Definition (4.6), then under Assumption (3), we have that J is Lipschitz, and the optimal DOD s exists.

Proof. Let $\mu, \mu' \in \Theta, t, t' \in [0, T]$ and $\mathbb{V}, \mathbb{V}' \in [-1, 1]^{N_h \times N'}$ arbitrary. For the following calculation, we compose the preliminary identity for $u := u_h^{\mu, \nu, t}$ and $u' := u_h^{\mu', \nu, t'}$, and the projectors $P := \mathbb{V}\mathbb{V}^T \mathbb{G}$, $P' := \mathbb{V}'(\mathbb{V}')^T \mathbb{G}$,

$$\begin{aligned} (\mathbb{I} - P)u - (\mathbb{I} - P')u' &= u - Pu - u' + P'u' \\ &= u - Pu - u' + P'u' + Pu' - Pu' \\ &= u - u' - Pu + Pu' + P'u' - Pu' \\ &= (\mathbb{I} - P)(u - u') + (P' - P)u'. \end{aligned} \tag{4.1.11}$$

Then we can calculate with our auxiliary Equation (4.1.11),

$$\begin{aligned}
 |J(\mu, t, \mathbb{V}) - J(\mu', t', \mathbb{V}')| &= \int_{\mathcal{X}} \left| \|u - Pu\|^2 - \|u' - P'u'\|^2 \right| d(\nu) \\
 &= \int_{\mathcal{X}} \left| \langle (u - Pu) + (u' - P'u'), (u - Pu) - (u' - P'u') \rangle \right| d(\nu) \\
 &\leq \int_{\mathcal{X}} \left(\|u - Pu\| + \|u' - P'u'\| \right) \| (u - Pu) - (u' - P'u') \| d(\nu) \\
 &= \int_{\mathcal{X}} \left(\|u - Pu\| + \|u' - P'u'\| \right) \| (\mathbb{I} - P)(u - u') + (P' - P)u' \| d(\nu) \\
 &= \int_{\mathcal{X}} \left(\|u - Pu\| + \|u' - P'u'\| \right) \left(\|(\mathbb{I} - P)(u - u')\| + \|(P' - P)u'\| \right) d(\nu) \\
 &\leq \int_{\mathcal{X}} \left(\|u\| + \|u'\| \right) \|u - u'\| d(\nu) + \int_{\mathcal{X}} \left(\|u\| + \|u'\| \right) \|P' - P\|_{\text{op}} \|u'\| d(\nu) \\
 &\leq |\mathcal{X}| (2M) L(|\mu - \mu'| + |t - t'|) + |\mathcal{X}| (2M^2) \|P' - P\|_{\text{op}} \\
 &\leq |\mathcal{X}| (2ML(|\mu - \mu'| + |t - t'|) + 2M^2 \|\mathbb{V} - \mathbb{V}'\|_{\text{op}}) \\
 &\leq |\mathcal{X}| (L + 2M) (|\mu - \mu'| + |t - t'| + \|\mathbb{V} - \mathbb{V}'\|_{\text{op}}).
 \end{aligned}$$

The key justification here, is the bound on $\|P - P'\|_{\text{op}}$, that can be achieved by considering

$$\begin{aligned}
 \|P - P'\|_{\text{op}} &= \|(\mathbb{V}' - \mathbb{V})(\mathbb{V}')^T \mathbb{G} - \mathbb{V}((\mathbb{V}')^T - \mathbb{V}^T) \mathbb{G}\|_{\text{op}} \\
 &\leq \|\mathbb{V}' - \mathbb{V}\|_{\text{op}} \cdot \|(\mathbb{V}')^T \mathbb{G}\|_{\text{op}} + \|\mathbb{V}\|_{\text{op}} \cdot \|((\mathbb{V}')^T - \mathbb{V}^T) \mathbb{G}\|_{\text{op}} \\
 &\leq 2\|\mathbb{V} - \mathbb{V}'\|_{\text{op}}.
 \end{aligned}$$

Here, we denote with $|\cdot|_{\text{op}}$ the standard (euclidean) spectral norm. Hence, by considerations made in the introductory Section (2.1), we have $\|\mathbb{V}\|_{\text{op}} = \|\mathbb{V}^T \mathbb{G}\| = 1$ for any $\mathbb{G}$ -orthonormal matrix $\mathbb{V} \in \mathbb{R}^{N_h \times N'}$.

To prove existence of the optimal DOD $s : \Theta \times [0, T] \rightarrow [-1, 1]^{N_A \times N'}$, we remark, that the space $[-1, 1]^{N_A \times N'}$ is a compact one, and the objective DOD functional J is continuous, which allows us to use the Kuratowski-Ryll-Nardzewski Theorem, found in (Kuratowski & Ryll-Nardzewski, 1965). The specifications of how this theorem can be applied to this specific situation can be found in (Franco et al., 2024, Appendix B.), whereas we will refrain from an extensive discussion, since we assume a stronger quality later on anyway. $\square$

Proposition 4.8 (Deep Orthogonal Decomposition Bound). For each geometric parameter $\mu \in \Theta$, let $\mathcal{S}_{\mu, t}^{\text{pre-red}} := \{\mathbb{A}^T \mathbb{G} u_h^{\mu, \nu, t}\}_{\nu \in \Theta'}$ be the (μ, t) -invariant pre-reduced solution submanifold, where $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ is a $\mathbb{G}$ -orthonormal matrix. Then, under the Assumptions (2) and (3), for every $\epsilon > 0$ there is a neural network architecture $\Phi_{\tilde{\mathbb{V}}}$ and neural network weights $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$ for a DOD according to Definition (3.5), denoted by $\mathbb{V}_{\mu, t} := \mathbb{V}_{\mu, t}(\theta_{\text{DOD}}^*) \in \mathbb{R}^{N_h \times N'}$ for each $(\mu, t) \in \Theta \times [0, T]$ and a constant $c'_{\mathbb{A}} > 0$ only depending on $\mathbb{A}, N_A$ and the parameter space, such that

$$\int_{\Theta \times \Theta' \times [0, T]} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\mu, \nu, t) \leq \epsilon + c'_{\mathbb{A}} + \int_{\Theta \times [0, T]} d_{N'}(\mathcal{S}_{\mu, t}^{\text{pre-red}})^2 d(\mu, t).$$

Proof. Let $N', N_A, N_h \in \mathbb{N}$ be fixed by Assumption (5). We set J , the objective DOD functional for Θ' , and s , the optimal DOD for Θ' , according to Definition (4.6), whereas the map

$$s : (\mu, t) \mapsto \underset{\tilde{\mathbb{V}} \in [-1, 1]^{N_A \times N'}}{\text{argmin}} J(\mu, t, \mathbb{A} \tilde{\mathbb{V}}),$$

is bounded (given by continuity of J and compactness of its image). Notice, that by the reasoning of Lemma (2.6), we know that for $\mathbb{V} = \mathbb{A}\tilde{\mathbb{V}}$ being $\mathbb{G}$ -orthonormal, its operator norm is 1, and hence after rescaling each entry can be inferred to be w.l.o.g. in $[-1, 1]$. This is due to the norm equivalence on this finite dimensional matrix space. Continuing the reasoning from before, we have $s \in L^2(\Theta \times [0, T]; \mathbb{R}^{N_A \times N'})$, i.e.

$$\|s\|_{L^2(\Theta \times [0, T]; \mathbb{R}^{N_A \times N'})}^2 := \int_{\Theta \times [0, T]} |s(\mu, t)|^2 d(\mu, t) < \infty.$$

Define $[s]_{ij}$ as the canonically lifted entry in position $(i, j) \in \{1, \dots, N_A\} \times \{1, \dots, N'\}$.

This means that by Hornik Theorem (2.24), there is a neural network architecture $[\tilde{\mathbb{V}}_0]_{ij}$ with $(p+1)$ -dimensional input and one-dimensional output, and a choice of neural network weights $\theta_{ij}^* \in \Theta([\tilde{\mathbb{V}}]_{ij})$ for that architecture, satisfying

$$\|[s]_{ij} - R_\rho([\tilde{\mathbb{V}}_0]_{ij}(\theta_{ij}^*))\|_{L^2(\Theta \times [0, T]; \mathbb{R})}^2 \leq \epsilon,$$

for each $(i, j) \in \{1, \dots, N_A\} \times \{1, \dots, N'\}$. We then write $[\tilde{\mathbb{V}}_0]_{ij} := [\tilde{\mathbb{V}}_0]_{ij}(\theta_{ij}^*)$ for each of these networks. Thus, by using network parallelization as in Definition (2.28), we can set the optimal DOD neural network as

$$\tilde{\mathbb{V}}_0 := P\left(P([\tilde{\mathbb{V}}_0]_{11}, \dots, [\tilde{\mathbb{V}}_0]_{1N'}), \dots, P([\tilde{\mathbb{V}}_0]_{N_A 1}, \dots, [\tilde{\mathbb{V}}_0]_{N_A N'})\right),$$

with a joint set of weights $\theta_{\Sigma ij}^* \in \Theta(\tilde{\mathbb{V}}_0)$. These can technically be computed using the canonical lift and the Definition (2.28) of parallelized networks. The realization of this network then attains

$$\|s - R_\rho(\tilde{\mathbb{V}}_0)\|_{L^2(\Theta \times [0, T]; \mathbb{R}^{N_A \times N'})}^2 \leq \epsilon.$$

To now ensure, that the provided realization of the almost DOD neural network are indeed in the pre-image of J , we need to ensure, that there is a network, which indeed attains the accuracy, but is also entrywise $[-1, 1]$. For this, let $\zeta = ((A_1, b_1), (A_2, b_2), (A_3, b_3))$ be given via

$$\begin{aligned} A_1 &:= [\mathbb{I}_{\mathbb{R}^{N_A \times N'}}], & b_1 &:= (\mathbb{I}_{\mathbb{R}^{N_A \times N'}}), & A_2 &:= [-\mathbb{I}_{\mathbb{R}^{N_A \times N'}}], & b_2 &:= (\mathbb{I}_{\mathbb{R}^{N_A \times N'}}), \\ A_3 &:= [-\mathbb{I}_{\mathbb{R}^{N_A \times N'}}], & b_3 &:= (\mathbb{I}_{\mathbb{R}^{N_A \times N'}}) \end{aligned}$$

The realization of this network is equivalent to the entry-wise operator of $x \mapsto 1 - \rho(2 - \rho(\rho(x+1))) \in [-1, 1]$ for $x \in \mathbb{R}$ on all entries of a $(N_A \times N')$ -dimensional matrix. Note, that to make this neural network truly adhere to Definition (2.19), which needs the dimensions to be natural numbers, we would have to reinterpret any matrix as a canonically stacked vector, but we omit this for the sake of readability, and since it does not affect the number of layers or weights.

Further because the Lipschitz constant of this map is 1, we can infer with the definition

$$\tilde{\mathbb{V}}_{\mu, t} := \left(R_\rho(\zeta) \circ R_\rho(\tilde{\mathbb{V}}_0(\theta_{\Sigma ij}^*))\right)(\mu, t), \quad \text{for } (\mu, t) \in \Theta \times [0, T],$$

a new network, called the inner DOD module, with a final architecture $\Phi_{\tilde{\mathbb{V}}}$ and weights $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$. Note, that these weights can be computed using the canonical lift, and the Definition (2.25) of network concatenations. The complexity of it only differs from $\tilde{\mathbb{V}}_0$ by constants, such that

$$\int_{\Theta \times [0, T]} \|\mathbb{V}_{\mu, t} - \mathbb{A}s(\mu, t)\|^2 d(\mu, t) \leq \int_{\Theta \times [0, T]} |R_\rho(\tilde{\mathbb{V}}_0)(\mu, t) - s(\mu, t)|^2 d(\mu, t) \leq \epsilon,$$

by the 1-Lipschitz property of $R_\rho(\zeta)$ and the argument of $\|\mathbb{A}x\| = |x|$ for any $x \in \mathbb{R}^{N_h}$. Hence, as a matrix, the DOD is indeed in the pre-image of J . We further with the Proposition (2.12) for the switch to the euclidean norm with

$$c'_A := \int_{\Theta \times [0, T]} \left(\sup_{\nu \in \Theta'} \underbrace{\|u_h^{\mu, \nu, t} - \mathbb{A} \mathbb{A}^T \mathbb{G} u_h^{\mu, \nu, t}\|}_{=c_A}^2 \right) d(\mu, t).$$

Then we can calculate with $u_{\text{pre-red}}^{\mu, \nu, t} := \mathbb{A}^T \mathbb{G} u_h^{\mu, \nu, t}$

$$\begin{aligned} & \int_{\Theta \times \Theta' \times [0, T]} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\mu, \nu, t) \\ & \leq \int_{\Theta \times [0, T]} |J(\mu, t, \mathbb{V}_{\mu, t}) - J(\mu, t, \mathbb{A} s(\mu, t))| d(\mu, t) + \int_{\Theta \times [0, T]} J(\mu, t, \mathbb{A} s(\mu, t)) d(\mu, t) \\ & \leq \epsilon + \int_{\Theta \times [0, T]} \left(\min_{\mathbb{V} \in \mathbb{R}^{N_A \times N'}} J(\mu, t, \mathbb{A} \mathbb{V}) \right) d(\mu, t) \\ & \leq \epsilon + c'_A + \int_{\Theta \times [0, T]} \left(\min_{\tilde{\mathbb{V}} \in \mathbb{R}^{N_A \times N'}} \sup_{\nu \in \Theta'} |u_{\text{pre-red}}^{\mu, \nu, t} - \tilde{\mathbb{V}} \tilde{\mathbb{V}}^T u_{\text{pre-red}}^{\mu, \nu, t}|^2 \right) d(\mu, t). \quad (4.1.12) \end{aligned}$$

□

Remark 4.9. We have structured the Proposition (4.8) with the L^1 -integral in mind, but we may completely omit the integral with regard to $d(\mu, t)$, by formally setting up a one element space, and set the objective DOD functional for $\mathcal{P}_2$ for the statement and proof as

$$\hat{J}(\mu, t, \mathbb{V}) := \frac{|\Theta'|}{N_{s_2}} \sum_{\nu \in \mathcal{P}_2} \|u_h^{\mu, \nu, t} - \mathbb{V} \mathbb{V}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2.$$

Then the entire proof can be done in the same way, since the Lipschitz property of $\hat{J}$ in Lemma (4.7) can be transferred, and culminates in the following version of Proposition (4.8) for $\mathbb{V}_{\mu, t} := \mathbb{V}_{\mu, t}(\theta_{\text{DOD}}^*)$, when discarding the very last step in Equation (4.1.12)

$$\frac{|\Theta'|}{N_{s_2}} \sum_{\nu \in \mathcal{P}_2} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 \leq \epsilon + c_A + \frac{|\Theta'|}{N_{s_2}} \min_{\tilde{\mathbb{V}} \in \mathbb{R}^{N_A \times N'}} \sum_{\nu \in \mathcal{P}_2} |u_{\text{pre-red}}^{\mu, \nu, t} - \tilde{\mathbb{V}} \tilde{\mathbb{V}}^T u_{\text{pre-red}}^{\mu, \nu, t}|^2,$$

for any choice of $(\mu, t) \in \Theta \times [0, T]$. Note, that if $N_A = N_h$, we have $c_A = 0$ by the definition of the POD as a canonical $\mathbb{G}$ -orthonormalization. In this case, the right-hand-side sum would be over the non-reduced average of KnW. This would simply constitute a case, where we completely circumvent a pre-reduction.

With that in mind, one can now build an analog to the relative error on the POD+DNN (i.e. Theorem (4.3)), since we now have a similar statement as the one of Proposition (2.14), in the form of Remark (4.9). In fact, we can even leverage Proposition (2.14) to define

$$\sigma(\mu, t)_1^2 \geq \dots \geq \sigma(\mu, t)_n^2 \quad \text{for any } n \leq r, \quad (4.1.13)$$

as the eigenvalues of a virtual POD (see Definition (2.13)) with the snapshot matrix

$$U := \left[u_{\text{pre-red}}^{\mu, \nu_1, t} \cdots u_{\text{pre-red}}^{\mu, \nu_{N_{s_2}}, t} \right]^T \in \mathbb{R}^{N_A \times N_{s_2}},$$

for each $(\mu, t) \in \Theta \times [0, T]$, and a subsequently defined discrete correlation matrix

$$K(\mu, t) := \frac{|\Theta'|}{N_{s_2}} U U^T,$$

that fulfill

$$\frac{|\Theta'|}{N_{s_2}} \min_{\tilde{V} \in \mathbb{R}^{N_A \times n}} \sum_{\nu \in \mathcal{P}_2} |u_{\text{pre-red}}^{\mu, \nu, t} - \tilde{V} \tilde{V}^T u_{\text{pre-red}}^{\mu, \nu, t}|^2 = \sum_{k > n} \sigma(\mu, t)_k^2,$$

for the $n \leq r$ from above. In the same manner, we can define a virtual infinite limit POD with the respective infinite correlation matrix and their eigenvalues

$$\sigma(\mu, t)_{1,\infty}^2 \geq \dots \geq \sigma(\mu, t)_{n,\infty}^2 \quad \text{for any } n \leq r, \quad (4.1.14)$$

that fulfill

$$\min_{\tilde{V} \in \mathbb{R}^{N_A \times n}} \int_{\nu \in \Theta'} |u_{\text{pre-red}}^{\mu, \nu, t} - \tilde{V} \tilde{V}^T u_{\text{pre-red}}^{\mu, \nu, t}|^2 d(\nu) = \sum_{k > n} \sigma(\mu, t)_{k,\infty}^2.$$

Theorem 4.10. *Define the preliminaries such as in the introduction to this Section (4.1). Further, set $\mathbb{V}_{\mu,t} := \mathbb{V}_{\mu,t}(\theta_{DOD}^*)$ for the choice of weights $\theta_{DOD}^* \in \Theta(\Phi_{\tilde{V}})$ as in Proposition (4.8). Then, under the Assumptions (2) and (3), we have for the relative error of the DOD+DNN*

$$\mathcal{E}_R^{DOD+DNN} \leq \mathcal{E}_S + \mathcal{E}_A + \mathcal{E}_{DOD} + \mathcal{E}_{NN}, \quad (4.1.15)$$

where

1. *The sampling error*

$$\mathcal{E}_S = \mathcal{E}_S(\mathcal{G}, (\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}, N')$$

satisfies $\mathcal{E}_S \rightarrow 0$ as $N_{s_2} \rightarrow \infty$, with expectation

$$\mathbb{E}_\nu[\mathcal{E}_S] \leq \mathcal{O}(N_{s_2}^{-1/4}).$$

2. *The ambient error*

$$\mathcal{E}_A = \mathcal{E}_A(\mathcal{G}, (\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}, N_A)$$

being equal to zero, if $N_A = N_h$.

3. *The DOD projection error*

$$\mathcal{E}_{DOD} = \mathcal{E}_{DOD}(\mathcal{G}, (\mu_i, \nu_i, t_i)_{i=1}^{N_{data}}, N')$$

satisfies $\mathcal{E}_{DOD} \rightarrow \mathcal{E}_{DOD,\infty} + \varrho$ almost surely as $N_{s_2} \rightarrow \infty$ for any $\varrho = \varrho(\Phi_{\tilde{V}}) > 0$ arbitrarily small depending on the size and choice of weights of the inner DOD module, where

$$\mathcal{E}_{DOD,\infty} = \mathcal{E}_{DOD,\infty}(\mathcal{G}, N')$$

is independent of the data snapshots and of order of the maximal linear KnW of the solution submanifold $\mathcal{S}_{\mu,t} = \{u_h^{\mu, \nu, t} \mid \nu \in \Theta'\}$ for any $(\mu, t) \in \Theta \times [0, T]$.

4. The neural network approximation error

$$\mathcal{E}_{NN} = \mathcal{E}_{NN}(\mathcal{G}, N', \Phi_{N'}^D)$$

is arbitrarily small depending on the size and choice of weights of the neural network $\Phi_{N'}^D$.

Proof. In its core, this proof inherits its structure and arguments from the one of Theorem (4.3), we will therefore only shorten the parts in which they are interchangeable. Let $\|\cdot\|_{L_\omega^2} : L^2(\Theta \times \Theta' \times [0, T]; \mathbb{R}^{N_h}) \rightarrow \mathbb{R}^+$ be the norm as given before, and define

$$q_D(\mu, \nu, t) := \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^{N'},$$

as the reduced solution regarding the DOD projection. Then we can apply the triangle inequality like before, to achieve

$$\mathcal{E}_R^{\text{DOD}+\text{DNN}} \leq \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|_{L_\omega^2} + \|\mathbb{V}_{\mu, t} q_D(\mu, \nu, t) - \mathbb{V}_{\mu, t} \hat{q}_D(\mu, \nu, t)\|_{L_\omega^2}.$$

In turn, from here we can already infer the error stemming from the latent dynamics identification under the neural network realization $\hat{q}_D$, i.e.

$$\mathcal{E}_{NN} = \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|\mathbb{V}_{\mu, t} q_D(\mu, \nu, t) - \mathbb{V}_{\mu, t} \hat{q}_D(\mu, \nu, t)\|^2}{\|u_h^{\mu, \nu, t}\|^2} d(\mu, \nu, t) \right)^{1/2}. \quad (4.1.16)$$

Moreover, we can extrapolate a similar procedure of splitting the remaining term into the respective errors as in the proof of Theorem (4.3), where we already assume for the fixed $N' \in \mathbb{N}$, the quasi-optimal choice of weights $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\mathbb{V}})$ for the underlying DOD neural network in accordance to Remark (4.9) for some fixed $\delta > 0$, the definition of $c_{\mathbb{A}} > 0$ and its subsequently defined virtual discrete correlation eigenvalues $\sigma(\mu, t)_k$ in Equation (4.1.13)

$$\begin{aligned} & \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|_{L_\omega^2} \\ & \leq m^{-1} \left(\int_{\Theta \times [0, T]} \int_{\Theta'} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\nu) d(\mu, t) \right)^{1/2} \\ & = m^{-1} \left(\int_{\Theta \times [0, T]} \int_{\Theta'} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\nu) - \sum_{k > N'} \sigma(\mu, t)_k^2 + \sum_{k > N'} \sigma(\mu, t)_k^2 d(\mu, t) \right)^{1/2} \\ & \leq m^{-1} \underbrace{\left| \int_{\Theta \times [0, T]} \int_{\Theta'} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\nu) - \frac{|\Theta'|}{N_{s_2}} \sum_{\nu \in \mathcal{P}_2} \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\mu, t) \right|}_{=:\mathcal{E}_S}^{1/2} \\ & \quad + m^{-1} \underbrace{\left(\int_{\Theta \times [0, T]} \|u_h^{\mu, \nu, t} - \mathbb{A}^T \mathbb{A} \mathbb{G} u_h^{\mu, \nu, t}\|^2 d(\mu, t) \right)^{1/2}}_{=:\mathcal{E}_A} \\ & \quad + m^{-1} \underbrace{|\Theta \times [0, T]|^{1/2} \delta^{1/2} + m^{-1} |\Theta \times [0, T]|^{1/2} \sup_{(\mu, t) \in \Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)_k^2}}_{=:\mathcal{E}_{\text{DOD}}}. \end{aligned}$$

Here, we can easily see, that the supremum chosen in the last step is actually an exaggeration since the mean

$$\mathcal{E}'_{\text{DOD}} := m^{-1} \left(\int_{\Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)^2} d(\mu, t) \right)^{1/2} + \frac{\delta^{1/2} |\Theta \times [0, T]|^{1/2}}{m}$$

is a more direct bound, also satisfying the upcoming convergence. However, we state this in regard to the former definition to stay consistent with the later discussion surrounding Assumption (1), that is formulated in maximal terms. From here, deploying the same strategy as in the proof of Theorem (4.3). We can assert, that $\mathcal{E}_S \rightarrow 0$ almost surely by the uniform strong law of large numbers (Newey & McFadden, 1994). In fact, we calculate

$$\hat{f}(\nu; \mu, t) := \|u_h^{\mu, \nu, t} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}\|^2 \leq M^2 \|\mathbb{I}_{N_h} - \mathbb{V}_{\mu, t} \mathbb{V}_{\mu, t}^T \mathbb{G}\|_{\text{op}}^2 \leq 2M^2 < \infty,$$

uniformly bounded and continuous in $(\mu, t) \in \Theta \times [0, T]$ by Lemma (4.7). Note, that Assumption (2) gives compactness of $\Theta \times [0, T]$. With this, we can use the mentioned uniform law, to get a bound on the supremum, i.e.

$$\sup_{(\mu, t) \in \Theta \times [0, T]} \left| \int_{\Theta'} \hat{f}(\nu; \mu, t) d(\nu) - \frac{|\Theta'|}{N_{s_2}} \sum_{\nu \in \mathcal{P}_2} f(\nu; \mu, t) \right| \xrightarrow{\text{a.s.}} 0.$$

And since this is an upper bound for the L^2 -integral, we get the wanted result. Further, we know with that boundedness of the underlying that $\mathbb{E}_\nu[\mathcal{E}_S] \leq \mathcal{O}(N_{s_2}^{-1/4})$ as a special case of Proposition (4.2). Remark, that $\mathbb{E}_\nu$ means to denote the integral with regard to the samples in Θ' .

Moreover, we have from Proposition (2.16), that for any $(\mu, t) \in \Theta \times [0, T]$

$$\hat{J}(\mu, t, \mathbb{A}s(\mu, t)) = \sum_{k > N'} \sigma(\mu, t)_k^2 \xrightarrow{N_{s_2} \rightarrow \infty} \sum_{k > N'} \sigma(\mu, t)_{k, \infty}^2,$$

where $\hat{J} : \Theta \times [0, T] \times [-1, 1]^{N_A \times N'}$ is the objective DOD functional, $s \in L^2(\Theta \times [0, T]; [-1, 1]^{N_A \times N'})$ the optimal DOD and $\mathbb{A} \in \mathbb{R}^{N_h \times N'}$ is the pre-reduction POD matrix, from Remark (4.9). Remark, that this allows us to assert, that indeed via an argument of restriction the map $(\mu, t) \mapsto \hat{J}(\mu, t, \mathbb{A}s(\mu, t))$ is continuous and bounded by Lemma (4.7). Hence,

$$(\mu, t) \mapsto \sqrt{\sum_{k > N'} \sigma(\mu, t)_k^2}$$

is continuous as well. This allows for the same use of the uniform strong law of large numbers (Newey & McFadden, 1994), resulting in

$$\begin{aligned} \mathcal{E}_{\text{DOD}} &= \frac{|\Theta \times [0, T]|^{1/2}}{m} \sup_{(\mu, t) \in \Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)_k^2} + \frac{\delta^{1/2} |\Theta \times [0, T]|^{1/2}}{m} \\ &\xrightarrow{N_{s_2} \rightarrow \infty} \frac{|\Theta \times [0, T]|^{1/2}}{m} \sup_{(\mu, t) \in \Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)_{k, \infty}^2} + \frac{\delta^{1/2} |\Theta \times [0, T]|^{1/2}}{m} =: \mathcal{E}_{\text{DOD}, \infty} + \varrho(\Phi_{\tilde{\mathbb{V}}}). \end{aligned}$$

Here, the $\varrho(\Phi_{\tilde{\mathbb{V}}})$ can be arbitrarily improved for a more complex choice of the inner DOD module architecture. This dependency on the complexity is suggested by Yarotsky Theorem (2.34), since the L^∞ error correlates with the $\mathcal{H}_h$ representation error. This is due to the compactness of the time-parameter space, that is given by Assumption (2). But that correlation is only truly provable under additional assumptions as argued in Remark (4.5).

Finally, we know that by Remark (4.9) $c_{\mathbb{A}} = 0$ implying $\mathcal{E}_{\mathbb{A}} = 0$, if $N_A = N_h$, which thus concludes the proof. $\square$

To connect this line of thought to the prior discussion for the POD+DNN, we can now pose a similar lower bound on the error for the DOD+DNN technique. For practical reasoning, the reader should keep in mind, that the result is closely connected to the complexity of the neural network of the inner DOD module, even if the following theorem omits such an appreciation, since the connection given through Yarotsky Theorem (2.34) is not immediate under the preliminaries alone.

Theorem 4.11. *Under the assumptions of Theorem (4.10), we have that*

$$\mathcal{E}_R^{DOD+DNN} \geq \frac{m}{M} \mathcal{E}'_{DOD,\infty},$$

where we set

$$\mathcal{E}'_{DOD,\infty} := m^{-1} \int_{\Theta \times [0,T]} \sqrt{\sum_{k>N'} \sigma(\mu, t)_{k,\infty}^2} d(\mu, t),$$

and $\sigma(\mu, t)_{k,\infty}$ the eigenvalues for a non-reduced continuous correlation matrix.

Proof. The proof will again follow a similar structure as the one of Theorem (4.4). In fact, we can again infer the suboptimal choice of the projection coefficients under any neural network realization

$$\int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{V}_{\mu,t} \hat{q}_D(\mu, \nu, t)\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t) \geq \int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{V}_{\mu,t} \mathbb{V}_{\mu,t}^T \mathbb{G} u_h^{\mu,\nu,t}\|^2}{\|u_h^{\mu,\nu,t}\|^2} d(\mu, \nu, t),$$

where $\mathbb{V}_{\mu,t} \in \mathbb{R}^{N_h \times N'}$ is the DOD matrix realization for any $(\mu, t) \in \Theta \times [0, T]$ as given by Definition (3.5). Then again starting from the other direction for a virtual POD matrix $\mathbb{A}_{\mu,t}^\infty \in \mathbb{R}^{N_h \times N'}$ for each $(\mu, t) \in \Theta \times [0, T]$ under the continuous correlation matrix, obtained as a limit of

$$U := \left[u_h^{\mu,\nu_1,t} \mid \dots \mid u_h^{\mu,\nu_{N_{s_2}},t} \right]^T \in \mathbb{R}^{N_h \times N_{s_2}}$$

for $N_{s_2} \rightarrow \infty$, which exists and is well-defined via the Proposition (2.14) and has eigenvalues $\sigma(\mu, t)_{1,\infty}^2 \geq \dots \geq \sigma(\mu, t)_{N',\infty}^2$, we get using the definition of the proof of Theorem (4.10)

$$(\mathcal{E}'_{DOD,\infty})^2 = m^{-2} \int_{\Theta \times [0,T]} \sum_{k>N'} \sigma(\mu, t)_{k,\infty}^2 d(\mu, t) \quad (4.1.17)$$

$$= m^{-2} \int_{\Theta \times [0,T]} \int_{\Theta'} \|u_h^{\mu,\nu,t} - \mathbb{A}_{\mu,t}^\infty (\mathbb{A}_{\mu,t}^\infty)^T \mathbb{G} u_h^{\mu,\nu,t}\|^2 d(\nu) d(\mu, t) \quad (4.1.18)$$

$$\leq m^{-2} \int_{\Theta \times [0,T]} \int_{\Theta'} \|u_h^{\mu,\nu,t} - \mathbb{V}_{\mu,t} \mathbb{V}_{\mu,t}^T \mathbb{G} u_h^{\mu,\nu,t}\|^2 d(\nu) d(\mu, t) \quad (4.1.19)$$

$$= m^{-2} \int_{\Theta \times \Theta' \times [0,T]} \frac{\|u_h^{\mu,\nu,t} - \mathbb{V}_{\mu,t} \mathbb{V}_{\mu,t}^T \mathbb{G} u_h^{\mu,\nu,t}\|^2}{\|u_h^{\mu,\nu,t}\|^2} \|u_h^{\mu,\nu,t}\|^2 d(\mu, \nu, t) \quad (4.1.20)$$

$$\leq \frac{M^2}{m^2} (\mathcal{E}_R)^2, \quad (4.1.21)$$

where we use the Proposition (2.14) for the continuous POD matrix in Step (4.1.18), the optimality of the same proposition for Step (4.1.19) and the first equation of this proof in Step (4.1.21) with the rather obvious use of the bound on $\|u_h^{\mu,\nu,t}\|$ from Assumption (3). Notice, that this statement works independent of the pre-reduction problem, since $\mathbb{A}_{\mu,t}^\infty$ is just the optimal $(N_h \cdot N')$ -dimensional matrix-based projector, and subsequently does not suffer from the pre-reduction. $\square$

Note, that this result barely differs from Theorem (4.4) for the POD+DNN ROM lower bound, and that we use a different version for the DOD error, than we have for the proof of Theorem (4.10), because the average over the Kolmogorov N' -widths in $\Theta \times [0, T]$ has more comparability to the Kolmogorov N -width decay of the whole solution manifold as a lower bound.

Of course, the optimality step in the proof suggests the use of a "good" DOD matrix and is hence closely related to the complexity of the inner DOD module, but this explains the lack of any mention of a choice for the neural network weights; this simply works for any matrix.

4.2 Upper Complexity Bounds

In this section, we will specify under the Assumption (2), (3), (4) and (5) the upper complexity bounds for our three main algorithms: the POD-DL-ROM, the DOD+DFNN and the DOD-DL-ROM, as introduced in Section (3.2).

However, to achieve actual complexity bounds for the latter two ROMs, we will necessarily need to impose one further assumption for the regularity of the optimal objective functional as discussed in Remark (4.5). This will be inserted in an upcoming part, before the discussion of the DOD-DL-ROM upper complexity bound as it is optional for the assertion of the complexity bound of the main Theorem (4.13). Indeed, if we omit any new assumptions, we may achieve the same Probably-Almost-Correct bound for the relative error of the DOD-DL-ROM, however without any information about the maximal complexity of the inner DOD module.

For the following discussion, we want to again set some preliminaries to shorten the theorem's layout. Firstly, we assume the setup of the previous Section (4.1), where we recall defining a sample to be used for all subsequent algorithms both POD-based and DOD-based neural networks, respectively marked POD+DNN and DOD+DNN. In fact, set up N and N' as follows

$$N = \arg \min_{n \in \mathbb{N}} \left(\sum_{k > n} \sigma_k^2 \leq \frac{m^2}{9} \varepsilon^2 \right), \quad N' = \arg \min_{n \in \mathbb{N}} \left(\sup_{(\mu, t) \in \Theta \times [0, T]} \sum_{k > n} \sigma(\mu, t)_k^2 \leq \frac{m^2}{16|\Theta \times [0, T]|} \varepsilon^2 \right) \quad (4.2.1)$$

with $\varepsilon > 0$ yet unspecified, and σ_k^2 according to Definition (2.13) and $\sigma(\mu, t)_k^2$ to Equation (4.1.13). Note, that under Assumption (1) the hierarchy of Assumption (5) is still preserved, if the choice of ε is small enough and N_{data} is large enough.

From this starting point, refine the definition of the approximate parameter-to-solution map for the POD+DNN into

$$\mathcal{G}_{\text{POD-DL-ROM}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \mathbb{A}_P \cdot (\psi_N \circ \phi_n)(\mu, \nu, t),$$

the POD-DL-ROM under weights $\theta_{\text{POD-DL-ROM}} = (\theta_D, \theta_{\text{DF}})$ with

$$\phi_n := R_\rho(\Phi_n(\theta_{\text{DF}})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n, \quad \psi_N := R_\rho(\Psi_N(\theta_D)) : \mathbb{R}^n \rightarrow \mathbb{R}^N,$$

from the Subsection (3.2.1). Note, that $\mathbb{A}_P \neq \mathbb{A}$ in general, since like before $\mathbb{A}_P \in \mathbb{R}^{N_h \times N}$ and $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$. Similarly, refine the approximate parameter-to-solution map for the DOD+DNN into

$$\mathcal{G}_{\text{DOD+DFNN}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \mathbb{V}_{\mu, t} \cdot \phi_{N'}(\mu, \nu, t),$$

the DOD+DFNN under weights $\theta_{\text{DOD+DFNN}} = (\theta_{\text{DOD}}, \theta_{\text{DF}_2})$ with

$$\mathbb{V}_{\mu, t} := \mathbb{A} \cdot R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}))(\mu, t) \in \mathbb{R}^{N_h \times N'}, \quad \text{for each } (\mu, t) \in \Theta \times [0, T], \text{ and} \\ \phi_{N'} := R_\rho(\Phi_{N'}(\theta_{\text{DF}_2})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N'}$$

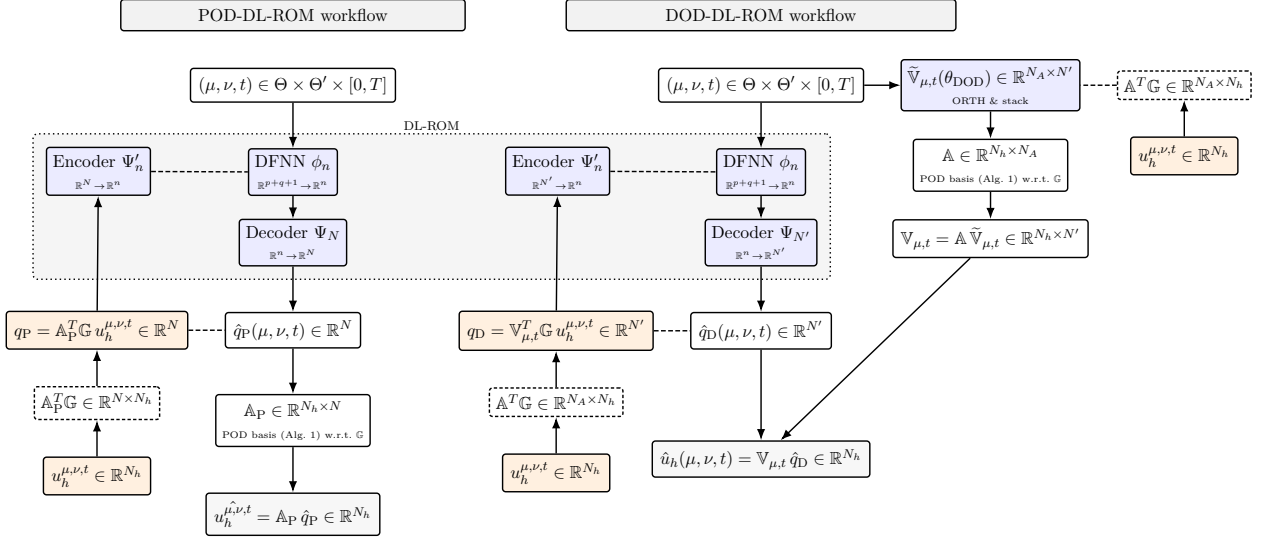


Figure 4.2: POD-DL-ROM (left) and DOD-DL-ROM (right) with autoencoder training. Solid arrows show inference; dashed lines denote training.

from the Subsection (3.2.2) and the DOD Definition (3.5), and

$$\mathcal{G}_{\text{DOD-DL-ROM}} : \Theta \times \Theta' \times [0, T] \rightarrow \mathbb{R}^{N_h}; \quad (\mu, \nu, t) \mapsto \mathbb{V}_{\mu, t} \cdot (\psi_{N'} \circ \phi_n)(\mu, \nu, t),$$

the DOD-DL-ROM under weights $\theta_{\text{DOD-DL-ROM}} = (\theta_{\text{DOD}}, \theta_{\text{D}_2}, \theta_{\text{DF}_3})$ with

$$\begin{aligned} \mathbb{V}_{\mu, t} &:= \mathbb{A} \cdot R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}))(\mu, t) \in \mathbb{R}^{N_h \times N'}, \quad \text{for each } (\mu, t) \in \Theta \times [0, T], \text{ and} \\ \psi_{N'} &:= R_\rho(\Psi_{N'}(\theta_{\text{D}_2})) : \mathbb{R}^n \rightarrow \mathbb{R}^{N'}, \quad \phi_n := R_\rho(\Phi_n(\theta_{\text{DF}_3})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n, \end{aligned}$$

from the Subsection (3.2.2) and the DOD Definition (3.5). In analogy to the previous Section (4.1), we visualize this with Figure (4.2). Note, that we omit the DOD+DFNN, since it is build in structure exactly like the DOD+DNN.

Furthermore, let $\chi_* \in C^s(\mathbb{R}^n; \mathbb{R}^{N'})$ and $\chi'_* \in C^{s'}(\mathbb{R}^{N'}; \mathbb{R}^n)$ respectively be the maps attaining the perfect embedding Assumption (4), with $s \gg s' \geq 2$. Since $N' < N$ in general, the perfect embedding is implied for N as well; let $\psi_* \in C^s(\mathbb{R}^n; \mathbb{R}^N)$ and $\psi'_* \in C^{s'}(\mathbb{R}^N; \mathbb{R}^n)$ be the maps attaining the equivalent for N .

Finally, set constants

$$C'_1 := \sup_{|\alpha| \leq s'} \sup_{v \in \mathbb{R}^{N'}} |D^\alpha \chi_*(v)|, \quad C'_2 := \sup_{|\alpha| \leq s} \sup_{w \in \mathbb{R}^n} |D^\alpha \chi'_*(w)|,$$

and C_1, C_2 analogously for ψ_* and ψ'_* .

Theorem 4.12. *Let the preliminaries of the POD-DL-ROM be given as in the introduction of this Section (4.2). Then there is $N_{\text{data}} = N_{\text{data}}(\delta, \varepsilon)$, a constant $c = c(\Theta, \Theta', T, L, C_1, C_2, p, q, n, s, s')$, an architecture and a choice of neural network weights for that architecture, such that the POD-DL-ROM approximate parameter-to solution map $\mathbb{A}_P \cdot \hat{q}_P := \mathbb{A}_P \cdot (\psi_N \circ \phi_n) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N_h}$ attains*

$$\mathbb{P} \left[\mathcal{E}_R^{\text{POD-DL-ROM}} < \varepsilon \right] > 1 - \delta.$$

Further, the complexity of the underlying architecture is at most

- $L_{\Psi_N} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Psi_N} = cN\varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights,

for the Decoder, and

- $L_{\Phi_n} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Phi_n} = cn\varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights,

for the Deep Feedforward Neural Network.

Proof. We can directly bound $\mathcal{E}_S = \mathcal{E}_S(N_{s_1}, N_{s_2}, N_t)$ defined in the proof of Theorem (4.3) independently of N , under the Assumption (2) by the strong law of large numbers, i.e. for all $1 > \delta > 0$ and $\varepsilon > 0$ there are N_{s_1}, N_{s_2}, N_t , such that

$$\mathbb{P}[\mathcal{E}_S((N_{s_1}, N_{s_2}, N_t)) < \varepsilon/3] > 1 - \delta. \quad (4.2.2)$$

This result provides the "probability" part of the PAC (probably almost correct) bound, which we want to show. In other words, provided this, the other two error types can be bounded without regard to the probability space underlying the sampling procedure. Bounding $\mathcal{E}_{\text{NN}}$ and $\mathcal{E}_{\text{POD}}$ by $\varepsilon/3$ respectively will then provide the wanted result. For this, by choosing N as in Equation (4.2.1), we simply rewrite

$$\mathcal{E}_{\text{POD}} := m^{-1} \sqrt{\sum_{k>N} \sigma_k^2} \leq \frac{\varepsilon}{3}. \quad (4.2.3)$$

Let $q_P(\mu, \nu, t) := \mathbb{A}_P^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^N$ be a reduced solution according to the POD basis. To now bound the neural network approximation error, we will need to simply rewrite the error in terms of a form accommodating a use of Gühring or Yarotsky Theorem;

$$\begin{aligned} \mathcal{E}_{\text{NN}} &= \left(\int_{\Theta \times \Theta' \times [0, T]} \frac{\|\mathbb{A}_P q_P(\mu, \nu, t) - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2}{\|u_h^{\mu, \nu, t}\|^2} d(\mu, \nu, t) \right)^{1/2} \\ &\leq m^{-1} \left(\int_{\Theta \times \Theta' \times [0, T]} \|\mathbb{A}_P q_P(\mu, \nu, t) - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2 d(\mu, \nu, t) \right)^{1/2} \\ &\leq m^{-1} \left(|\Theta \times \Theta' \times [0, T]| \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} \|\mathbb{A}_P q_P(\mu, \nu, t) - \mathbb{A}_P \hat{q}_P(\mu, \nu, t)\|^2 \right)^{1/2} \\ &\leq m^{-1} \left(|\Theta \times \Theta' \times [0, T]| \cdot \|\mathbb{A}_P\|_{\text{op}}^2 \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} \|q_P(\mu, \nu, t) - \hat{q}_P(\mu, \nu, t)\|^2 \right)^{1/2} \\ &= m^{-1} |\Theta \times \Theta' \times [0, T]|^{1/2} \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} \|q_P(\mu, \nu, t) - \hat{q}_P(\mu, \nu, t)\|. \end{aligned} \quad (4.2.4)$$

We split the latter error into both parts, since $\hat{q}_P = \psi_N \circ \phi_n$ per definition. This means employing two types of error estimations in the matter of finding the respective optimal maps for the feed forward estimation on the latent dynamics space via $\phi_* : \mathbb{R}^{(p+q+1)} \rightarrow \mathbb{R}^n$ and the decoder lift via $\psi_* : \mathbb{R}^n \rightarrow \mathbb{R}^N$.

- Let $\mathcal{S}_N := \{q_P(\mu, \nu, t) = \mathbb{A}_P^T \mathbb{G} u_h^{\mu, \nu, t} \in \mathbb{R}^N \mid (\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]\}$, then the image of the optimal embedding $\mathcal{V}_n = \psi'_*(\mathcal{S}_N) \in \mathbb{R}^n$ is such that $\text{diam}(\mathcal{V}_n) \leq C_1 L \cdot \text{diam}(\Theta \times \Theta' \times [0, T])$,

given the Assumption (3) and the preliminaries. In fact,

$$\sup_{w, w' \in \mathcal{V}_n} |w - w'| = \sup_{v, v' \in \mathcal{S}_N} |\psi'_*(v) - \psi'_*(v')| \quad (4.2.5)$$

$$\leq C_1 \sup_{v, v' \in \mathcal{S}_N} |v - v'| \quad (4.2.6)$$

$$\leq C_1 \sup_{(\mu, \nu, t), (\mu', \nu', t') \in \Theta \times \Theta' \times [0, T]} |\mathbb{A}_P^T \mathbb{G}(u_h^{\mu, \nu, t} - u_h^{\mu', \nu', t'})| \quad (4.2.7)$$

$$\leq C_1 \sup_{(\mu, \nu, t), (\mu', \nu', t') \in \Theta \times \Theta' \times [0, T]} |\mathbb{A}_P^T \mathbb{G}|_{\text{op}} |u_h^{\mu, \nu, t} - u_h^{\mu', \nu', t'}| \quad (4.2.8)$$

$$\leq C_1 L \cdot \text{diam}(\Theta \times \Theta' \times [0, T]). \quad (4.2.9)$$

Here, we have used the definition of $\mathcal{V}_n$ as the image of ψ'_* in Step (4.2.5), the fact, that

$$C_1 \geq \sup_{v \in \mathcal{S}_N} |D^\alpha \psi'_*|$$

for $|\alpha| = 1$ and its implication under the mean value theorem (Rudin, 1976, p. 107) in Step (4.2.6) and the fact, that $|\mathbb{A}_P^T \mathbb{G}|_{\text{op}} = \|\mathbb{A}_P\|_{\text{op}} = 1$ by $\mathbb{G}$ -orthonormality of $\mathbb{A}_P$, together with the Lipschitz-quality of $\mathcal{G}$ from Assumption (3) in Step (4.2.8). This argument allows us to find that $\psi_* \in \mathcal{F}_{s, n, \infty, C_1 L \cdot \text{diam}(\Theta \times \Theta' \times [0, T])}$, after unilateral prior transformation $x \mapsto x/|x| \in (0, 1)^n$ of any input, for the target space of the optimal function required by Theorem G hring (2.35). Thus, there is a neural network architecture Ψ_N and neural network weights $\theta_D \in \Theta(\Psi_N)$ fitting the network realization to that optimal function, such that its realization $\psi_N = R_\rho(\Psi_N(\theta_D)) : \mathbb{R}^n \rightarrow \mathbb{R}^N$ attains

$$\|\psi_* - \psi_N\|_{W^{1, \infty}((0, 1)^d)} \leq \frac{m}{6} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon.$$

This architecture has at most $L_{\Psi_N} = c \log(\varepsilon^{-1})$ layers and $\omega_{\Psi_N} = cN \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights.

For the sake of completeness, we mention that one would build N -many entry-wise neural networks in parallel and hence attain for each $\omega_{[\psi_N]_1} = c \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights under G hring, then employ the Lemma (2.29) for the parallelization of these. Note, that this does not influence the order of the layer number. A similar approach was already argued in the proof of Proposition (4.8). This statement is equivalent to

$$\sup_{v \in \mathcal{V}_N} \|\psi_*(v) - \psi_N(v)\| < \frac{m}{6} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon, \quad (4.2.10)$$

$$\text{ess sup}_{v, v' \in \mathcal{V}_N} \frac{|(\psi_* - \psi_N)(v) - (\psi_* - \psi_N)(v')|}{|v - v'|} < \frac{m}{6} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon, \quad (4.2.11)$$

by definition of the derivative and the fact, that Sobolev norms are the sum of both the equations above. Additionally, this implies that, ψ_N is Lipschitz with constant $C_3 := C_2 + m/6 \cdot |\Theta \times \Theta' \times [0, T]|^{-1/2}$, since

$$\frac{|(\psi_* - \psi_N)(v) - (\psi_* - \psi_N)(v')|}{|v - v'|} \geq \frac{|\psi_N(v) - \psi_N(v')|}{|v - v'|} - \frac{|\psi_*(v) - \psi_*(v')|}{|v - v'|}$$

by inverse triangle inequality for any $v, v' \in \mathcal{V}_N$ and subsequently employing the C_2 -bound for ψ_* from the preliminaries of this Section (4.2).

- Now let $\phi_*(\mu, \nu, t) = \psi'_*(q_P(\mu, \nu, t))$ for $(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]$, which is Lipschitz continuous, with constant LC_1 . In fact, we have for any pair $(\mu, \nu, t), (\mu', \nu', t') \in \Theta \times \Theta' \times [0, T]$,

$$\begin{aligned} |\phi_*(\mu, \nu, t) - \phi_*(\mu', \nu', t')| &= |\psi'_*(\mathbb{A}_P^T \mathbb{G} u_h^{\mu, \nu, t}) - \psi'_*(\mathbb{A}_P^T \mathbb{G} u_h^{\mu', \nu', t'})| \\ &\leq C_1 |\mathbb{A}_P^T \mathbb{G} (u_h^{\mu, \nu, t} - u_h^{\mu', \nu', t'})| \\ &\leq C_1 L \cdot (|\mu - \mu'| + |\nu - \nu'| + |t - t'|), \end{aligned}$$

by the very same arguments as in the calculation for the diameter of $\mathcal{V}_n$ above. The Lipschitz continuity of this map then allows us to employ Theorem (2.11), which guarantees that $\phi_* \in W^{1, \infty}(\mathcal{S}_N)$, since $\text{diam}(\mathcal{S}_N) \leq L \cdot \text{diam}(\Theta \times \Theta' \times [0, T])$ by the same arguments as in the $\mathcal{V}_n$ -case. Hence, after entry-wise normalization, $\phi_* \in \mathcal{F}_{1, p+q+1, \infty, \text{diam}(\mathcal{S}_N)}$.

Now we employ Theorem Yarotsky (2.34) to assert the existence of a neural network architecture Φ_n weights $\theta_{\text{DF}} \in \Theta(\Phi_n)$, such that its realization $\phi_n := R_\rho(\Phi_n(\theta_{\text{DF}})) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^n$ attains

$$\|\phi_* - \phi_n\|_{W^{0, \infty}(\Theta \times \Theta' \times [0, T])} < \frac{m}{6C_3} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon. \quad (4.2.12)$$

This architecture has at most $L_{\Phi_n} = c \log(\varepsilon^{-1})$ layers and $\omega_{\Phi_n} = cn\varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights, again with the argument of neural network parallelization.

Starting from the last inequality in (4.2.4) we can perform the split in a clean fashion using the triangle inequality and reminding ourselves of the Assumption (4), which guarantees $q = \psi_*(\psi'_*)$, and thus gives with both neural network estimates in Equations (4.2.10) and (4.2.12)

$$\begin{aligned} &\sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} \|q_P(\mu, \nu, t) - \hat{q}_P(\mu, \nu, t)\| \\ &\leq \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} (\|\psi_*(\psi'_*(\mu, \nu, t)) - \psi(\phi_*(\mu, \nu, t))\| + \|\psi(\phi_*(\mu, \nu, t)) - \psi(\phi(\mu, \nu, t))\|) \\ &\leq \sup_{v \in \mathcal{V}_N} \|\psi_*(v) - \psi(v)\| + C_3 \sup_{(\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]} (\|\psi(\phi_*(\mu, \nu, t)) - \psi(\phi(\mu, \nu, t))\|) \\ &< \frac{m}{6} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon + C_3 \frac{m}{6C_3} |\Theta \times \Theta' \times [0, T]|^{-1/2} \varepsilon = \frac{\varepsilon}{3} m |\Theta \times \Theta' \times [0, T]|^{-1/2}, \end{aligned}$$

which cancels with $m^{-1} |\Theta \times \Theta' \times [0, T]|^{1/2}$ to get

$$\mathcal{E}_{\text{NN}} < \frac{\varepsilon}{3}. \quad (4.2.13)$$

Now putting together all three estimates (4.2.2), (4.2.3) and (4.2.13) with the reasoning of Theorem (4.3) finally gives

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{\text{POD}} + \mathcal{E}_{\text{NN}} < \frac{\varepsilon}{3} + \frac{\varepsilon}{3} + \frac{\varepsilon}{3} = \varepsilon,$$

with higher probability than $1 - \delta$. $\square$

Building upon the discussion in the beginning of this Section (4.2), we may explore the implication of a more regular optimal DOD for $\mathcal{X} = \mathcal{P}_2$ from Definition (4.6). Indeed, if such an assumption is given, then Yarotsky Theorem can be invoked for the implied construction of the inner DOD module in Remark (4.9).

Assumption 6. Let the objective DOD functional $J : \Theta \times [0, T] \times [-1, 1]^{N_h \times N'}$ be given for $\mathcal{X} = \mathcal{P}_2$ as in Definition (4.6), and the corresponding optimal DOD

$$s : \Theta \times [0, T] \mapsto [0, 1]^{N_h \times N'}; \quad (\mu, t) \mapsto \underset{\tilde{\mathbf{V}} \in [-1, 1]^{N_A \times N'}}{\text{argmin}} \quad J(\mu, t, \mathbb{A} \tilde{\mathbf{V}})$$

be $\hat{L}$ -Lipschitz.

This assumption will now provide us with the ability to use Yarotsky Theorem retroactively in Proposition (4.8) to achieve bounds on the complexity for the inner DOD module. However, the relative error bound, we discussed in Theorem (4.10) needs an assertion for N_A to be close to N_h to achieve the actual benefit of a (μ, t) -dependent projector.

As a matter of fact, in the following we set $N_A = N_h$ to avoid cumbersome notation and make use of the Assumption (1) later on. This is within the framework of Assumption (5), but note, that this can be relaxed similarly to the POD error $\mathcal{E}_{\text{POD}}$, as has been done in Theorem (4.12). In fact, this constitutes that N_A must be chosen in the same order as N has been for the POD-DL-ROM, since the ambient error $c_{\mathbb{A}}$ from Proposition (2.12) has the same inhibition of scaling with the total POD eigenvalues.

Theorem 4.13. *Let the preliminaries of the DOD-DL-ROM be given as in the introduction of this Section (4.2), $N_A = N_h$ explicitly and the Assumption (6) be true. Then there is $N_{\text{data}} = N_{\text{data}}(\delta, \varepsilon)$, a constant $c = c(\Theta, \Theta', T, L, \hat{L}, C_1, C_2, p, q, n, s, s')$, an architecture and a choice of neural network weights for that architecture, such that the DOD-DL-ROM approximate parameter-to solution map $(\mu, \nu, t) \mapsto \mathbb{V}_{\mu, t} \cdot \hat{q}_D := \mathbb{V}_{\mu, t} \cdot (\psi_{N'} \circ \phi_n) : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N_h}$ attains*

$$\mathbb{P} \left[\mathcal{E}_R^{\text{DOD-DL-ROM}} < \varepsilon \right] > 1 - \delta.$$

Further, the complexity of the underlying architecture is at most

- $L_{\Phi_{\tilde{\mathbb{V}}}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Phi_{\tilde{\mathbb{V}}}} = c N_A N' \varepsilon^{-(p+1)} \log(\varepsilon^{-1})$ active weights,

for the inner DOD module,

- $L_{\Psi_{N'}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Psi_{N'}} = c N' \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1})$ active weights,

for the Decoder, and

- $L_{\Phi_n} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Phi_n} = c n \varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights,

for the Deep Feedforward Neural Network.

Proof. Similar as to the proof of Theorem (4.12), we will try to bound each quantity of the statement in Theorem (4.10), but with the catch, that $\mathcal{E}_{\text{DOD}}$ is stochastic in nature, and thus we will employ Remark (4.9) to find an auxiliary term, which is independent of the data, to which the approximation of the neural net of the DOD is sufficiently "close". Here, we instantaneously remark, that taking $N_A = N_h$ implies $\mathcal{E}_{\mathbb{A}} = 0$, and we will not mention this contribution again.

The sampling error can be bounded in the same way as before, i.e. we can directly bound $\mathcal{E}_S = \mathcal{E}_S(N_{s_2})$ independently of N' , under the Assumption (2) by the Weak Law of Large Numbers, i.e. for all $1 > \delta > 0$ and $\varepsilon > 0$ there is N_{s_2} , such that

$$\mathbb{P}[\mathcal{E}_S < \varepsilon/4] > 1 - \delta. \quad (4.2.14)$$

Now recall the definition of $\mathcal{E}_{\text{DOD}}$ in the proof of Theorem (4.10) and further setting the $\epsilon > 0$ from the upstream Remark (4.9), realizing an "optimal" discrete approximation for a fixed data size N_{s_2} ,

as

$$\epsilon = \frac{\epsilon m}{4|\Theta \times [0, T]|^{1/2}}.$$

Hence, concluding, under the additional use of definition of N' in Equation (4.2.1), in the following approximation for the DOD error:

$$\begin{aligned} \mathcal{E}_{\text{DOD}} &= \frac{|\Theta \times [0, T]|^{1/2}}{m} \left(\epsilon + \sup_{(\mu, t) \in \Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)_k^2} \right) \\ &< \frac{\epsilon}{4} + \left(\frac{|\Theta \times [0, T]|^{1/2}}{m} \sup_{(\mu, t) \in \Theta \times [0, T]} \sqrt{\sum_{k > N'} \sigma(\mu, t)_k^2} \right) \\ &\leq \frac{\epsilon}{4} + \left(\frac{|\Theta \times [0, T]|^{1/2}}{m} \cdot \frac{\epsilon m}{4|\Theta \times [0, T]|^{1/2}} \right) = \frac{\epsilon}{2} \end{aligned} \quad (4.2.15)$$

Under the Assumption (6), a neural network architecture $[\tilde{\mathbb{V}}_0]_{ij}$ with corresponding weights θ_{ij}^* can indeed be found, such that

$$\|[s]_{ij} - R_\rho([\tilde{\mathbb{V}}_0]_{ij}(\theta_{ij}^*))\|_{L^\infty(\Theta \times [0, T]; \mathbb{R})} \leq \epsilon,$$

for each $(i, j) \in \{1, \dots, N_h\} \times \{1, \dots, N'\}$. This architecture has at most

- $L_{[\tilde{\mathbb{V}}_0]_{ij}} = c'' \log(\epsilon^{-1})$ layers and
- $\omega_{[\tilde{\mathbb{V}}_0]_{ij}} = c'' \epsilon^{-(p+1)} \log(\epsilon^{-1})$ active weights,

for some $c'' = c''(\Theta, T, p, \hat{L}) > 0$ by Yarotsky Theorem (2.34). The use of this theorem is due to the $\hat{L}$ -Lipschitz property of the optimal DOD, justifying $W^{1, \infty}$ -regularity with an $\hat{L}$ bound by Theorem (2.11).

From this point, using the steps in the proof of Proposition (4.8) under the specifications of Remark (4.9), we can find a neural network architecture $\Phi_{\tilde{\mathbb{V}}}$ for the inner DOD module and corresponding optimal weight $\theta_{\text{DOD}}^* \in \Theta(\Phi_{\tilde{\mathbb{V}}})$, such that

$$\|s - R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}^*))\|_{L^\infty(\Theta \times [0, T]; \mathbb{R}^{N_h \times N'})} \leq \epsilon,$$

with at most

- $L_{\Phi_{\tilde{\mathbb{V}}}} = c' \log(\epsilon^{-1})$ layers and
- $\omega_{\Phi_{\tilde{\mathbb{V}}}} = c' N_A N' \epsilon^{-(p+1)} \log(\epsilon^{-1})$ active weights,

where $c' = 3 + c''$.

Moreover, we can use the exact same arguments as in the bound for $\mathcal{E}_{\text{NN}}$ for the POD-DL-ROM as in the proof of Theorem (4.12), which yields

$$\mathcal{E}_{\text{NN}} < \frac{\epsilon}{4}. \quad (4.2.16)$$

For sake of uniformity, we then take $c > 0$ to be the maximal of each individual constant of all three separate neural networks. Note, that here we imply, that the reduction is achieved with the replacement of N with N' in each of the arguments (and also the use of potentially different weights

and architectures), hence also getting slightly different values for the number of layers and weights. Finally, putting each Equation, i.e. (4.2.14), (4.2.15) and (4.2.16), together, we get

$$\mathcal{E}_R \leq \mathcal{E}_S + \mathcal{E}_{\text{DOD}} + \mathcal{E}_{\text{NN}} < \frac{\varepsilon}{4} + \frac{\varepsilon}{2} + \frac{\varepsilon}{4} = \varepsilon,$$

with probability greater than $1 - \delta$ concluding our proof. $\square$

In the next step, we can follow up this complexity PAC bound with the same statement for the slightly simpler architecture of the DOD+DFNN. It is clear, that the omission of the reduction by an autoencoder simplifies the reduction procedure by one step. This does not change basically anything regarding the proof, as the choice of N' ensures a bound here too.

Theorem 4.14. *Let the preliminaries of the DOD+DFNN be given as in the introduction of this Section (4.2), $N_A = N_h$ explicitly and the Assumption (6) be true. Then there is $N_{\text{data}} = N_{\text{data}}(\delta, \varepsilon)$, a constant $c = c(\Theta, \Theta', T, L, \hat{L}, p, q, N')$, an architecture and a choice of neural network weights for that architecture, such that the DOD+DFNN approximate parameter-to solution map $(\mu, \nu, t) \mapsto \mathbb{V}_{\mu, t} \cdot \hat{q}_D := \mathbb{V}_{\mu, t} \cdot \phi_{N'} : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N_h}$ attains*

$$\mathbb{P} \left[\mathcal{E}_R^{\text{DOD+DFNN}} < \varepsilon \right] > 1 - \delta.$$

Further, the complexity of the underlying architecture is at most

- $L_{\Phi_{\tilde{\mathbb{V}}}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Phi_{\tilde{\mathbb{V}}}} = c N_A N' \varepsilon^{-(p+1)} \log(\varepsilon^{-1})$ active weights,

for the inner DOD module, and

- $L_{\Phi_{N'}} = c \log(\varepsilon^{-1})$ layers,
- $\omega_{\Phi_{N'}} = c N' \varepsilon^{-(p+q+1)} \log(\varepsilon^{-1})$ active weights,

for the Deep Feed-Forward Neural Network.

Proof. This proof is in its essence the same as the one of Theorem (4.13); it leverages Theorem (4.10) and the preliminaries to attain the bounds

$$\mathcal{E}_{\text{DOD}} < \frac{\varepsilon}{2} \quad \text{and} \quad \mathcal{E}_S < \frac{\varepsilon}{4}$$

in the exact same way, and then uses the Lipschitz property of $\mathcal{G}$ under Assumption (3). Indeed, let $\Phi_* : \mathbb{R}^{p+q+1} \rightarrow \mathbb{R}^{N'}$ define the optimal coefficient map, that satisfies

$$\Phi_*(\mu, \nu, t) := \mathbb{V}_{\mu, t}^T \mathbb{G} u_h^{\mu, \nu, t}, \quad \text{for all } (\mu, \nu, t) \in \Theta \times \Theta' \times [0, T].$$

This map then inherits the Lipschitz continuity, i.e. for any pair $(\mu, \nu, t), (\mu', \nu', t') \in \Theta \times \Theta' \times [0, T]$, we have

$$\begin{aligned} |\Phi_*(\mu, \nu, t) - \Phi_*(\mu', \nu', t')| &= |\mathbb{V}_{\mu, t}^T \mathbb{G} (u_h^{\mu, \nu, t} - u_h^{\mu', \nu', t'})| \\ &\leq |u_h^{\mu, \nu, t} - u_h^{\mu', \nu', t'}| \\ &\leq L \cdot (|\mu - \mu'| + |\nu - \nu'| + |t - t'|). \end{aligned}$$

Here, we have exploited Lemma (2.6) because $\mathbb{V}_{\mu,t}$ is $\mathbb{G}$ -orthonormal for any parameter choice. Following that, we use Theorem (2.11), to achieve the placement of this optimal function in the space $\mathcal{F}_{1,p+q+1,\infty,L \cdot \text{diam}(\Theta \times \Theta' \times [0,T])}$. With that argument, we can use Yarotsky Theorem (2.34) to get the complexities entry-wise, as seen before, with error margin

$$\mathcal{E}_{\text{NN}} < \frac{\varepsilon}{4}.$$

Combining all these, eventually gets us the wanted bounds. $\square$

4.3 Comparison between existing Techniques

In this section, we will compare the presented data-driven ROM techniques with each other in the context of the Theorems (4.12), (4.13) and (4.14), or in other words; we will examine the differences of the upper complexity bounds for the POD-based neural network approach and both DOD-based neural network approaches. As another step, these complexities will be coarsely compared to other techniques, not yet presented in the paper, which have been proposed in the context of data-driven Reduced Order Modelling. For this, we will recite some results from relevant literature and argue the advantages and disadvantages of the DOD-based ROMs in comparison.

Within this framework, we will play around with potentially sharper assumptions, especially hypothetically improving regularity of the parameter-to-solution map $\mathcal{G}$ in Assumption (3) and the regularity of the optimal DOD s in Assumption (6). These two regularities namely determine the effectivity of the neural net under Yartosky (and/or Gühring) Theorem (2.34). To have a substantial difference in the use of the DOD, as opposed to the POD, we want to focus the reader's attention on Assumption (1), which ensures

1. For $\{\sigma_{k,\infty}^2\}_k$ the eigenvalues of the POD matrix given under the continuous correlation matrix of the entire solution manifold $\mathcal{S} = \{u_h^{\mu,\nu,t} \mid (\mu, \nu, t) \in \Theta \times \Theta' \times [0, T]\}$ a slow decay, i.e.

$$\sqrt{\sum_{k>n} \sigma_{k,\infty}^2} \leq C n^{-\alpha},$$

for $C > 0$ a constant and $0 < \alpha < 1$, and

2. For $\{\sigma(\mu, t)_{k,\infty}^2\}_k$ the eigenvalues of the (μ, t) -fixed POD matrix given under the continuous non-reduced correlation matrix of the (μ, t) -dependent solution manifold $\mathcal{S}_{\mu,t} = \{u_h^{\mu,\nu,t} \mid \nu \in \Theta'\}$ a quick decay, i.e.

$$\sup_{(\mu,t) \in \Theta \times [0,T]} \sqrt{\sum_{k>n} \sigma(\mu, t)_{k,\infty}^2} \leq C' n^{-\beta},$$

for $C' > 0$ a constant and $\beta \geq 1$.

This fact derives as a direct consequence of Proposition (2.16) and the definitions of the continuous correlation matrices in Definition (2.13). With these two bounds, we can presumably take for very large data $N_s, N_t \rightarrow \infty$ and $N_A = N_h$

$$N = \mathcal{O}\left(\varepsilon^{-\frac{1}{\alpha}}\right), \quad N' = \mathcal{O}\left(\varepsilon^{-\frac{1}{\beta}}\right), \quad (4.3.1)$$

for the guaranteed PAC bounds of the POD-DL-ROM, the DOD+DFNN and the DOD-DL-ROM of fixed precision $\varepsilon > 0$ as is dictated in the beginning of Section (4.2). In fact, we know that by

Proposition (2.16) $\sum_{k>n} \sigma_k^2 \rightarrow \sum_{k>n} \sigma_{k,\infty}^2$ for $N_s, N_t \rightarrow \infty$, and therefore

$$N = \arg \min_{n \in \mathbb{N}} \left\{ \sum_{k>n} \sigma_k^2 \geq \frac{m^2}{9} \varepsilon^2 \right\} \xrightarrow{N_s, N_t \rightarrow \infty} \arg \min_{n \in \mathbb{N}} \left\{ C n^{-2\alpha} \geq \frac{m^2}{9} \varepsilon^2 \right\} = \mathcal{O}(\varepsilon^{-\frac{1}{\alpha}}),$$

and a similar argument for N' . These rough estimates for the order of N and N' then let us get an idea for the upcoming comparisons, which mainly will involve the weighing of the higher complexity cost of the DOD-based ROMs against the lighter, but potentially less reductive, POD-based methods. Note, that the assumption of $N_A = N_h$ could be relaxed, if the pre-reduced (μ, t) -dependent solution manifold also has fast KnW decay.

4.3.1 Comparison between POD-based and DOD-based ROMs

Within the subsection, our purpose will be to list the differences between the POD-based techniques for model order reduction and the DOD-based ones depending on the regularity imposed on the optimal DOD s as given by Definition (4.6), for $N_A = N_h$, and the regularity imposed on the parameter-to-solution map $\mathcal{G}$. As such, assume additionally to the framework of the Section (4.2),

1. $\mathcal{G} \in W^{r,\infty}(\Theta \times \Theta' \times [0, T]; \mathbb{R}^{N_h})$ for some $r \geq 1$,
2. $s \in W^{m,\infty}(\Theta \times [0, T]; \mathbb{R}^{N_h \times N'})$ for some $m \geq 1$.

Let also $0 < \varepsilon < 1$ in general for the sake of monotonicity later on. To cluster the total complexity of a ROM, we can just add the respective layers and weights of each neural network contained in each method. Such a procedure leaves us with

$$\begin{aligned} - \omega_{\text{POD-DL-ROM}} &= \mathcal{O} \left(N \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1}) + n \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right), \\ - \omega_{\text{DOD-DL-ROM}} &= \mathcal{O} \left(N_h N' \varepsilon^{-(p+1)/m} \log(\varepsilon^{-1}) + N' \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1}) + n \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right), \\ - \omega_{\text{DOD+DFNN}} &= \mathcal{O} \left(N_h N' \varepsilon^{-(p+1)/m} \log(\varepsilon^{-1}) + N' \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right), \end{aligned}$$

and the same layer bound for all of them by insertion of the regularity in the respective proofs of the upper complexity bounds, where Yarosky Theorem (2.34) has been used. We remark, that in Assumption (4), we set $n \leq 2(p+q) + 3$ in general, and $s \gg s' \geq 2$ in the preliminaries of Section (4.2). This quantifies all relevant complexity numbers. Namely, we want to analyze three distinct cases, starting with a general agnostic one, where r, m are close to 1, in a sense that $n/(s-1) \leq (p+1)/m$ and $n/(s-1) \leq (p+q+1)/r$. This directly results in

$$\begin{aligned} \omega_{\text{POD-DL-ROM}} &= \mathcal{O} \left(N \varepsilon^{-n/(s-1)} \log(\varepsilon^{-1}) + n \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right) \\ &\leq \mathcal{O} \left(N \varepsilon^{-(p+1)/m} \log(\varepsilon^{-1}) + n \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right) \\ &\lesssim \mathcal{O} \left(N_h N' \varepsilon^{-(p+1)/m} \log(\varepsilon^{-1}) + N' \varepsilon^{-(p+q+1)/r} \log(\varepsilon^{-1}) \right) \\ &= \omega_{\text{DOD+DFNN}}, \end{aligned}$$

where we used, that $N < N_h$ and the fact that $n < N'(\beta, \varepsilon), N(\alpha, \varepsilon)$ by Assumption (5). The same heuristic can be done for DOD-DL-ROM active weights bound, since essentially the number of weights are dominated by the DOD neural network contribution anyway. The situation changes, however, if we can assert a high regularity for the optimal DOD s , i.e. if $m \rightarrow \infty$ such that

$(p+1)/m \approx 0$, and N_h "small enough" in relation to the other summands. Then the contribution of the DOD to the complexity is neglectable, and we get

$$\begin{aligned}\omega_{\text{DOD-DL-ROM}} &\approx \mathcal{O}\left(N'\varepsilon^{-n/(s-1)}\log(\varepsilon^{-1}) + n\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\lesssim \mathcal{O}\left(N\varepsilon^{-n/(s-1)}\log(\varepsilon^{-1}) + n\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\approx \omega_{\text{POD-DL-ROM}}.\end{aligned}$$

This can be attributed to the heuristic, that $N' + n < N$, because of Equation (4.3.1). Here, we have also still prescribed the dominance of the decoder complexity over the forward pass, therefore awarding the DOD-DL-ROM a bonus due to the inherently better approximation under existence of a smooth optimal decoder map $\zeta_* \in C^s(\mathbb{R}^n; \mathbb{R}^{N'})$. In fact,

$$\begin{aligned}\omega_{\text{DOD-DL-ROM}} &\approx \mathcal{O}\left(N'\varepsilon^{-n/(s-1)}\log(\varepsilon^{-1}) + n\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\lesssim \mathcal{O}\left(N'\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1}) + n\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\approx \omega_{\text{DOD+DFNN}}.\end{aligned}$$

In the third scenario, where we set both r, m to be very large, in a sense that $(p+1)/m \approx 0$ and $n/(s-1) > (p+q+1)/r$, and N_h still small in a relative proportion to the other summands, we get a better complexity for the more direct approach of the DOD+DFNN as opposed to the DOD-DL-ROM, i.e.

$$\begin{aligned}\omega_{\text{DOD+DFNN}} &\approx \mathcal{O}\left(N'\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\lesssim \mathcal{O}\left(N'\varepsilon^{-n/(s-1)}\log(\varepsilon^{-1}) + n\varepsilon^{-(p+q+1)/r}\log(\varepsilon^{-1})\right) \\ &\approx \omega_{\text{DOD-DL-ROM}}.\end{aligned}$$

We can still show, that even in this case, both DOD-based approaches work better, than the POD-DL-ROM in general, since the argument of the lower chosen N' can be applied. As a less relevant note, we can see in (Brivio et al., 2023, p.32-33) that indeed in the case of low parameter-to-solution map regularity, we would actually get a similar comparison between a naive POD+DFNN and the POD-DL-ROM. This is not surprising, because it follows the same line of thought as our argument does. To briefly sum up, we want to pose the following result:

If the optimal DOD map s , mapping

$$(\mu, t) \mapsto \arg \min_{\mathbb{V} \in [-1,1]^{N_h \times N'}} \frac{|\Theta'|}{N_{s_2}} \sum_{\nu \in \mathcal{P}_2} \|u_h^{\mu, \nu, t} - \mathbb{V}\mathbb{V}^T \mathbb{G}u_h^{\mu, \nu, t}\|^2,$$

has high regularity in a Sobolev sense, i.e. $m \gg 1$, then it can generally be of advantage to use a DOD-based ROM. In addition, if we know the parameter-to-solution map $\mathcal{G}$, mapping

$$(\mu, \nu, t) \mapsto u_h^{\mu, \nu, t},$$

to be of high regularity in a Sobolev sense, i.e. $r \gg 1$, then it is advised to use the "cruder" architecture of the DOD+DFNN above the DOD-DL-ROM. If this is not the case, then the employment of an autoencoder can exploit the presumably superior nonlinear KnW with the control inherent to autoencoder training, and hence attain a similar result with less cost.

Finally, one can attest to the rather non-rigorous treatment of the second and third cases, where we denoted $(p+1)/m \approx 0$ and N_h "small enough", which is not a very precise formulation. To remedy this, one can reformulate it without changing the discussion in the following manner:

Proposition 4.15. Let a precision $0 < \varepsilon < 1$ be given, which bounds the respective relative errors of the three proposed ROMs with probability $0 < 1 - \delta < 1$ as is given by Theorems (4.12), (4.13) and (4.14) under the Assumptions (2, 3, 4, 5), i.e.

$$\mathcal{E}_R^{\text{POD-DL-ROM}} < \varepsilon, \quad \mathcal{E}_R^{\text{DOD-DL-ROM}} < \varepsilon, \quad \mathcal{E}_R^{\text{DOD+DFNN}} < \varepsilon,$$

and if the constants $N_h, n, s \in \mathbb{N}$ attain

$$\frac{n}{s-1} > \frac{\log(N_h)}{\log(\varepsilon^{-1})},$$

and the optimal DOD map $s \in W^{m,\infty}(\Theta \times [0, T])$ has $m \in \mathbb{N}$, such that

$$m \gtrsim \frac{p+1}{\frac{n}{s-1} - \log_\varepsilon \left(\frac{N-N'}{N_h N'} \right)}$$

is true, then it is guaranteed that either

$$\omega_{\text{DOD+DFNN}} \lesssim \omega_{\text{DOD-DL-ROM}} \lesssim \omega_{\text{POD-DL-ROM}}, \quad \text{for } r \geq \frac{1}{n}(s-1)(p+q+1),$$

or only

$$\omega_{\text{DOD-DL-ROM}} \lesssim \omega_{\text{POD-DL-ROM}}, \quad \text{for } r \leq \frac{1}{n}(s-1)(p+q+1),$$

depending on $r \in \mathbb{N}$ setting the regularity of the parameter-to-solution map $\mathcal{G} \in W^{r,\infty}(\Theta \times \Theta' \times [0, T])$. If we additionally set up Assumption (1) with the constants $0 < \alpha < 1$ and $\beta \geq 1$, and let $N_s, N_t \rightarrow \infty$, we may infer a sharper minimal order of m , with

$$m \gtrsim \frac{p+1}{\frac{n}{s-1} - \frac{\log(N_h)}{\log(\varepsilon^{-1})} + \frac{\beta-\alpha}{\beta\alpha}}.$$

Proof. For the following, we want to start with the wanted statement and simply rearrange the targeted inequality until it poses a lower bound for m . Note beforehand, that in the case $r \geq (s-1)(p+q+1)/n$, we know the number of maximal weights for the DOD-DL-ROM to be larger than for the DOD+DFNN, excusing the distinction. We can now set up

$$\omega_{\text{DOD-DL-ROM}} \lesssim \omega_{\text{POD-DL-ROM}}$$

as a starting point. This itself implies, since we can subtract the latent dynamics contributions without changing the order, that

$$N_h N' \varepsilon^{-(p+1)/m} + N' \varepsilon^{-n/(s-1)} \lesssim N \varepsilon^{-n/(s-1)}.$$

From here, we subtract the decoder contribution from the left to fit it to the right, then divide by the positive term $\varepsilon^{-n/(s-1)}$ and set the constants on one side, to attain

$$\varepsilon^{-\left(\frac{p+1}{m} - \frac{n}{s-1}\right)} \lesssim \frac{N - N'}{N_h N'},$$

where, by Assumption (5), we have that $0 < (N - N')/(N_h N') \leq (N + N')/(N_h N') < 1$. Then taking the logarithm with regard to $\varepsilon < 1$ as base, hence being a monotonely decreasing function, therefore flipping the inequality, and subsequently rearranging and multiplying by negative one, leaves us with

$$\frac{p+1}{m} \lesssim \frac{n}{s-1} - \log_\varepsilon \left(\frac{N - N'}{N_h N'} \right).$$

With the note, that $\log_\varepsilon(\cdot) = -\log(\cdot)/\log(\varepsilon^{-1})$ we can conclude, through the assumption on N_h, n and s from the statement of the proposition that both sides are strictly positive, hence resulting in the wanted bound after inverting and multiplying by $(p+1)$.

Let us now assume $N_s, N_t \rightarrow \infty$ and the Assumption (1) in order to prove the extension of the proposition. Immediately, one can recognize from the beginning of the section, that $N = \mathcal{O}(\varepsilon^{-1/\alpha})$ and $N' = \mathcal{O}(\varepsilon^{-1/\beta})$. We now deduce, that

$$\frac{N - N'}{N_h N'} = \frac{1}{N_h} \left(\frac{N}{N'} - 1 \right) = \mathcal{O} \left(\frac{1}{N_h} \left(\varepsilon^{-\frac{1}{\alpha} + \frac{1}{\beta}} - 1 \right) \right) \lesssim \frac{1}{N_h} \varepsilon^{\frac{\alpha - \beta}{\alpha \beta}},$$

thus implying by monotonicity of the logarithm

$$\log_\varepsilon \left(\frac{N - N'}{N_h N'} \right) = -\frac{\log \left(\frac{N - N'}{N_h N'} \right)}{\log(\varepsilon^{-1})} \gtrsim -\frac{\log(1/N_h)}{\log(\varepsilon^{-1})} + \frac{\alpha - \beta}{\alpha \beta} = \frac{\log(N_h)}{\log(\varepsilon^{-1})} - \frac{\beta - \alpha}{\beta \alpha},$$

which concludes our proof. Note, that we rewrite the logarithms in such a way, that the signs make sense in the context of the constants. $\square$

Remark 4.16 (Discussion). One can quickly deduce some implication of Proposition (4.15) for large scale applicational purposes, namely, we expect the number of degrees of freedom N_h of the basis to be $\approx 10^6$ – 10^{18} , and the precision level ε to be very small maybe $\approx 10^{-6}$ – 10^{-3} .

From the heuristic for $s \gg s' \geq 2$, and usually $p, q \approx 10^0$ – 10^1 suggesting $n/(s-1) \leq 2(p+q+1)/(s-1)+3 \approx 10^0$ – 10^1 , we might infer, that the contribution of $n/(s-1) - \log(N_h)/\log(\varepsilon^{-1})$ could be of order $\approx 10^0$ – 10^1 , thus offering m to be of order $\approx 10^0$ – 10^1 , depending on the difference $\beta - \alpha$. We may specifically point out, that a higher difference constitutes a smaller regularity imposition on the optimal DOD map. This estimate is nothing but very crude, but it does point to the fact, that the levels of regularity, which need to be assumed, are not completely unrealistic, and might be met by some problems.

As a final difference between the POD-based ROMs and the DOD-based ROMs, one can focus the lower bound Theorems (4.4) for the POD-based and (4.11) for the DOD-based methods. These are of course highly connected to the proposition above, as we have just assumed, that we might want to take N, N' arbitrarily such that the subsequent loss under the projection attains the needed fraction of the fixed $\varepsilon > 0$ precision level, but in a case, where the complexity cost poses no issue, and we want to simply have the most accurate possible method, the DOD-based ROMs take the win. Generally, such a situation is mostly fictitious because we only coarsely know, that there is an advantage, but the cost is never completely ignored, especially since these theorems do not deny the possibility, that the cost blows up before any significant benefit can be gained; it only describes the character of a hypothetical limit.

4.3.2 Literature Comparison to other data-driven ROMs

As a final step in the theoretical setting, we would like to introduce a few important other data-driven ROMs and compare them to the proposed DOD-based architectures, we have analyzed. These are meant to be only crude introductions, as the main focus lies on the comparability of the POD versus the DOD and most of the other architectures simply adhere to a different set of problems and assumptions.

Linear Residual Nets (lin+Res) Residual network architectures, originally popularized in the context of computer vision, have also been adapted as linear regression surrogates for PDE models. In our setting, they may be understood as learning an approximation of the parameter-to-solution map $\mathcal{G}$ by means of a sequence of affine layers with skip connections. Such skip connections are simply defined as

$$x \mapsto R_\rho(\Phi)(x) + x,$$

where Φ is some neural network. In order to overcome the limitation of necessarily retaining the same dimension as the input, they simply append their architecture with the definition of projection connections, which simply replace x with (non-)trainable projectors. The skip structure allows the network to model perturbations of a baseline linear evolution in a stable fashion. This method was first proposed in (He et al., 2016), and in our main source (Brivio et al., 2023) we can find the technical complexity bound of

$$L_{\text{lin+ResNet}} = \mathcal{O}(\varepsilon^{-1}), \quad \omega_{\text{lin+ResNet}} = \mathcal{O}(Nk\varepsilon^{-1}),$$

where $k \in \mathbb{N}$ represents the latent dimension to which the residual network is reduced, and $N \ll N_h$ is the linear projection potentially achieved by a non-trainable POD matrix. Hence, we can only assert that it should be outperformed by both DOD- and POD-based ROMs in terms of layer numbers and, by the POD-based ROMs in terms of the non-zero weights

Deep Operator Networks (DeepONets) DeepONets, introduced in (Lu et al., 2021), are designed to learn nonlinear operators in a discretization-independent manner. This completely separates them from all the methods proposed up to this point, since their general framing allows the approximation of the overarching true PPDE problem, without a technical step down to a FOM. As such, they pose a very broad class of approximation problems; for any continuous function u of an underlying compact subset of a Banach space, the DeepONet tries to find the mapping $u \mapsto \mathcal{G}_\infty(u)$, where $\mathcal{G}_\infty$ is some operator on the space of these continuous functions by using so-called sensor points, that try assuming the structure of u before minimizing. This is done using a *branch net* b , which consists of an encoder $\mathcal{E}$ employing the aforementioned sensors in order to send the continuous function u to an empirical set of evaluation points, and an approximation network $\mathcal{A}$ attaining the wanted dimension and a *trunk net* τ (or their stacked linear combinations) lifting the reduced dimension up to the continuous function output space of $\mathcal{G}_\infty$, i.e.

$$\mathcal{G}_\infty(u)(y) \approx b(u)\tau(y) = (\mathcal{E} \circ \mathcal{A})(u) \cdot \tau(y).$$

This is practical, when considering the parameters as, for example, some known boundary functions, but in our case, this approach can be specified to the *POD-DeepONet* introduced in (Lu et al., 2022), where the trunk net is specifically given by an a priori POD on the snapshots and the given function u is meant as the inferred parameter realization function (the identity). Therefore, we realistically aim at an approximation of the parameter-to-solution map $\mathcal{G}$ using

$$\mathcal{G}_{\text{POD-DeepONet}}(\mu, \nu, t) = \mathbb{A} \cdot R_\rho(\mathcal{A})(\mu, \nu, t),$$

for the POD matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N}$. It is to be remarked, that as discussed in (Brivio et al., 2023) it is completely within the framework to introduce the solution part of the parameter-to-solution map, as an actual continuous function with regard to the time dimension, which would then significantly vary this approach from the discussed POD-based ROM approach. How it stands, the POD-DeepONet can be interpreted as a more general version of the POD+DNN (either POD-DL-ROM or POD+DFNN), but in case the domain introduces μ and t as a part of the output domain, i.e. considering it being a function only mapping ν to its solution across all possible (μ, t) , we could interpret the DOD-based architectures as a special case of the DeepONets.

Convolutional Neural Network (CNN) CNNs, firstly used during the ImageNet competition and afterwards introduced in the paper (Krizhevsky et al., 2012), are widely used as surrogates for PDE solutions defined on regular grids. In our notation, the CNN directly approximates the parameter-to-solution map $\mathcal{G}$ by applying successive local convolutions and pooling operations over the grid. The main difference between these and our inherent use of autoencoders in the midst of DOD-DL-ROM and POD-DL-ROM is, that the CNNs actually leverage on the spatial structure of the FOM and require as such an equidistant grid. As a result, the CNNs are more closely related to Fourier Neural Networks (Kovachki et al., 2021), since they try approximating the underlying "image-type" nature of the model. Since the analysis does also rely on such Fourier arguments, it is necessary to assert additionally, that the map resulting from the FOM is continuous, i.e. $u_h(\mu, \nu, t) \in C^\gamma(\Omega)$ for $\gamma \geq 1$. Then a bound for the non-zero weights is

$$\omega_{\text{CNN}} = \mathcal{O} \left(\varepsilon^{-\frac{2}{2\gamma-1}} \left(\varepsilon^{-\frac{p+q+1}{r}} (\log(\varepsilon^{-1}) + \log(N_h)) \right) \right),$$

from the discussion in (Brivio et al., 2023) and the fact $h^{-1} = \mathcal{O}(N_h)$ by the argument of equidistant spacing. For the reasons of the specifications of the grid, both DOD-DL-ROM and POD-DL-ROM can be seen as more flexible alternatives and if we know the decoder map to be sufficiently regular, in a sense that $n/(s-1) > (p+q+1)/r$ similar to the discussions in Proposition (4.15), then the POD-DL-ROM outperforms the CNN architecture in terms of complexity efficiency (see (Brivio et al., 2023) for detailed discussion). In the same line of thought, we know that if then additionally the regularity of the optimal DOD suffices and the bottleneck property of the Proposition (4.15) is met for N_h and ε , we can infer a potential superiority of the DOD-DL-ROM above both.

Chapter 5

Numerical Experiments

In this chapter, we will present some numerical experiments and subsequent comparative analysis of DOD-based and POD-based ROMs, as well as the stationary informed CoLoRA architecture for which we will only produce some qualitative results in a later subsection. The structure of this chapter will be the following:

First, we will provide the reader with some specifications of the gathering of data snapshots and the general training workflow employed for the statistical learning of the neural network weights for the corresponding models. This is uniform in context of the different ROMs to retain comparability. As a next step, we want to introduce two examples, a one dimensional heat advection problem with geometric initial condition and an advection-diffusion problem with constant Neumann condition. In both of these, we will graph the KnW as given by Definition (2.17), or at least a numerical approximation of it, then show one qualitative realization of a neural network architecture with trained weights for the POD-DL-ROM, the DOD+DFNN and the DOD-DL-ROM plotted against a FOM solution, and finally set up a comparison in relative error in relation to the active weights to compare the theoretical result to numerical ones.

For all models, we uniformly use the LeakyReLU instead of the ReLU to form the neural networks contained in the models. This activation function is mathematically equivalent to the ReLU as justified by the paper (Geist et al., 2020). Hence, we overwrite the definition of the ReLU with

$$\rho(x) := \begin{cases} x, & \text{if } x \geq 0 \\ ax, & \text{if } x < 0. \end{cases}$$

We will put $a = 0.1$ as the constant in every instance of the activation unit.

In order to connect to the brief discussion of Statistical Learning in Subsection (2.3.5), we will start with the specific definitions of the training procedure, after attaining a set of data in the form of

$$\left\{ (\mu_i, \nu_j, k\Delta t), u_h^{\mu_i, \nu_j, k\Delta t} \right\}_{(i,j,k)=(1,1,1)}^{(N_{s1}, N_{s2}, N_t)} \in (\mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T}) \times \mathcal{G}((\mathcal{P}_1 \times \mathcal{P}_2 \times \mathcal{T})) \subset (\Theta \times \Theta' \times [0, T]) \times \mathbb{R}^{N_h}.$$

Here, we generate the solutions using the `pymor` module (Milk et al., 2016), and its builtin discretizers relying on `gmsh` meshing for the second, more complex, example. In general, we use a CG discretizer technique and its native solver, but this can of course be modified freely. Crucially, we perform the POD and each of the error-estimates employing the gram matrix generated by the $W_0^{1,2}$ -norm on the deaffinized solution space. For this, we subtract a Dirichlet shift before training and re-add it afterwards, so that any model is being trained on the correct space. Hence, any difference between solutions in $\|\cdot\|$ should be seen as close in a $W^{1,2}$ -sense, since the matrix gram matrix $\mathbb{G}$ is taken from that inherent inner product.

The data can be understood in terms of solution snapshot matrices $S \in \mathbb{R}^{N_h \times N_{s_1} N_{s_2} N_t}$ and their respective parameter stacks $M \in \mathbb{R}^{p \times N_{s_1}}, N \in \mathbb{R}^{q \times N_{s_2}}$:

$$S = \left[u_h^{\mu_1, \nu_1, \Delta t} \mid \dots \mid u_h^{\mu_{N_{s_1}}, \nu_1, \Delta t} \mid u_h^{\mu_2, \nu_1, \Delta t} \mid \dots \mid \dots \mid u_h^{\mu_{N_{s_1}}, \nu_{N_{s_2}}, N_t \Delta t} \right]$$

After this, we use a common validation strategy, where we split each batch into a training data and a validation data, such that $M = [M^{\text{train}}, M^{\text{val}}], N = [N^{\text{train}}, N^{\text{val}}]$ and $S = [S^{\text{train}}, S^{\text{val}}]$ and a splitting factor of $\alpha = 0.8$.

For the normalization, we use the aforementioned min-max normalization in a generalized sense; for each parameter set we define

$$M_{\max}^i = \max_{j=1, \dots, N_{s_1}} M_{ij}^{\text{train}}, \quad M_{\min}^i = \min_{j=1, \dots, N_{s_1}} M_{ij}^{\text{train}}$$

and its subsequent min-max normalization via

$$M_{ij}^{\text{train}} \mapsto \frac{M_{ij}^{\text{train}} - M_{\max}^i}{M_{\max}^i - M_{\min}^i}, \quad \text{for } i = 1, \dots, p, j = 1, \dots, N_{s_1}. \quad (5.0.1)$$

For N^{train} the analogous approach can be used. The solution batch is normalized in the same way, after defining the minimum and maximum as

$$S_{\max} = \max_{i=1, \dots, N_h} \max_{j=1, \dots, N_s N_t} S_{ij}^{\text{train}}, \quad S_{\min} = \min_{i=1, \dots, N_h} \min_{j=1, \dots, N_s N_t} S_{ij}^{\text{train}}.$$

All offline training phases will be performed with the AdamW optimizer and the CosineAnnealingWarmRestarts scheduler to achieve adaptive learning rates. Moreover, an early stopping criterion will be imposed upon the validation loss. The relevant hyperparameters will be discussed for each example separately, as we choose them with regard to the specifics of the problem at hand. However, in the following Algorithm (5) we describe the exact offline procedure for the DOD-DL-ROM training collectively. The non-DOD part of the DOD+DFNN is a less complex adaptation of a straight-forward concatenation of some linear layers, and can hence be inferred from the DOD-DL-ROM learning procedure and its definition in Subsection (3.2.2), since the a priori treatment of the data is the same. The POD-DL-ROM offline and online procedure can be found in (Fresca & Manzoni, 2022).

For evaluation, we rely on the following discretized version of the relative error $\mathcal{E}_R \in [0, \infty)$ over a test sample $\{(\mu_i, \nu_i), u_h^{\mu_i, \nu_i}\}_{i=1}^{N_{\text{test}}}$ with time-trajectories in $\mathcal{T}$ defined as

$$\mathcal{E}_R = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \left(\frac{\sqrt{\frac{1}{N_t} \sum_{k=1}^{N_t} \|u_h^{\mu_i, \nu_i, k \Delta t} - \hat{u}_h^{\mu_i, \nu_i, k \Delta t}\|^2}}{\sqrt{\frac{1}{N_t} \sum_{k=1}^{N_t} \|u_h^{\mu_i, \nu_i, k \Delta t}\|^2}} \right)$$

as well as the pointwise time-trajectory error $\epsilon_{\text{rel}} \in [0, \infty)^{N_h \times N_t}$ for visualization, given by

$$\epsilon_{\text{rel}} = \frac{|u_h^{\mu_i, \nu_i, k \Delta t} - \hat{u}_h^{\mu_i, \nu_i, k \Delta t}|_{\text{pw}}}{\sqrt{\frac{1}{N_t} \sum_{k=1}^{N_t} \|u_h^{\mu_i, \nu_i, k \Delta t}\|^2}}$$

where we set $|\cdot|_{\text{pw}}$ as the pointwise absolute difference of the N_h -dimensional vector, and $\hat{u}_h^{\mu, \nu, t} \in \mathbb{R}^{N_h}$ denoting some solution prediction.

Model training is performed using the Python library PyTorch (Paszke et al., 2019) on an Nvidia Quadro RTX 6000. All source code required to reproduce the presented results, accompanied by a concise README.md, is openly available at <https://github.com/dawid-kotowski/doddlrom>.

Algorithm 4 DOD training algorithm (offline)

Require: Parameter matrices $M \in \mathbb{R}^{p \times N_{s1}}$, snapshot matrix $S \in \mathbb{R}^{N_h \times N_t N_s}$, training-validation splitting fraction α , starting learning rate η^0 , batch size N_b , batch number N_{nb} , maximum number of epochs N_{epochs} , dimensions n, N', N_A

Ensure: Optimal model parameters θ_{DOD}^*

- 1: Compute pre-reduction POD basis matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ via Definition (2.13)
 - 2: Randomly shuffle M and S
 - 3: Split data in $M = [M^{\text{train}}, M^{\text{val}}]$ and $S = [S^{\text{train}}, S^{\text{val}}]$ according to α
 - 4: $S_{N_A}^{\text{train}} \leftarrow \mathbb{A}^T \mathbb{G} S^{\text{train}}$ and $S_{N_A}^{\text{val}} \leftarrow \mathbb{A}^T \mathbb{G} S^{\text{val}}$
 - 5: Normalize data in M and $S_{N_A} = [S_{N_A}^{\text{train}}, S_{N_A}^{\text{val}}]$
 - 6: Initialize inner DOD module $\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}^0) \leftarrow \text{RandomInit}(\theta_{\text{DOD}}^0)$
 - 7: $n_e = 0$
 - 8: **while** ($\neg$ early-stopping and $n_e \leq N_{\text{epochs}}$) **do**
 - 9: **for** $k = 1 : N_{nb}$ **do**
 - 10: Sample a batch $(M^{\text{batch}}, N^{\text{batch}}, S_{N_A}^{\text{batch}}) \subset (M^{\text{train}}, N^{\text{train}}, S_{N_A}^{\text{train}})$
 - 11: **for** $t = 1 : N_t$ **do**
 - 12: $\tilde{\mathbb{V}}_{M^{\text{batch}}, t} \leftarrow R_\rho(\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}^{n_e N_b + k}))(M^{\text{batch}}, t) \in \mathbb{R}^{N_b \times N_h \times N'}$
 - 13: **end for**
 - 14: Accumulate loss $\mathcal{L}_{\text{DOD}}$ of Definition (3.5) on $(M^{\text{batch}}, S_{N_A}^{\text{batch}})$ and compute $\nabla_{\theta} \mathcal{L}$
 - 15: $\theta_{\text{DOD}}^{n_e N_b + k + 1} \leftarrow \text{AdamW}(\eta^{n_e}, \nabla_{\theta} \mathcal{L}, \theta_{\text{DOD}}^{n_e N_b + k})$
 - 16: $\eta^{n_e + 1} \leftarrow \text{CAWR}(\mathcal{L}_{\text{DOD}})$
 - 17: **end for**
 - 18: Repeat instructions 9–17 on $(M^{\text{val}}, S_{N_A}^{\text{val}})$ with the updated weights $\theta_{\text{DOD}}^{n_e N_b + k + 1}$
 - 19: Accumulate loss $\mathcal{L}_{\text{DOD}}$ of Definition (3.5) on $(M^{\text{val}}, S_{N_A}^{\text{val}})$ to evaluate early-stopping criterion
 - 20: $n_e = n_e + 1$
 - 21: **end while**
-

Algorithm 5 DOD-DL-ROM training algorithm (offline)

Require: Parameter matrices $M \in \mathbb{R}^{p \times N_{s1}}$, $N \in \mathbb{R}^{q \times N_{s2}}$, snapshot matrix $S \in \mathbb{R}^{N_h \times N_t N_s}$, training-validation splitting fraction α , starting learning rate η^0 , batch size N_b , batch number N_{nb} , maximum number of epochs N_{epochs} , dimensions n, N', N_A

Ensure: Optimal model parameters $\theta_{\text{DOD-DL-ROM}}^* = (\theta_{\text{DOD}}^*, \theta_{\text{D}}^*, \theta_{\text{DF}}^*)$

- 1: Compute pre-reduction POD basis matrix $\mathbb{A} \in \mathbb{R}^{N_h \times N_A}$ via Definition (2.13)
- 2: Randomly shuffle M and S
- 3: Split data in $M = [M^{\text{train}}, M^{\text{val}}]$, $N = [N^{\text{train}}, N^{\text{val}}]$ and $S = [S^{\text{train}}, S^{\text{val}}]$ according to α
- 4: $S_{N_A}^{\text{train}} \leftarrow \mathbb{A}^T \mathbb{G} S^{\text{train}}$ and $S_{N_A}^{\text{val}} \leftarrow \mathbb{A}^T \mathbb{G} S^{\text{val}}$
- 5: Normalize data in M, N and $S_{N_A} = [S_{N_A}^{\text{train}}, S_{N_A}^{\text{val}}]$
- 6: Train inner DOD module $\Phi_{\tilde{\mathbb{V}}}(\theta_{\text{DOD}}^*)$ with Algorithm (4)
- 7: Initialize DL-ROM weights $(\theta_{\text{D}}^0, \theta_{\text{E}}^0, \theta_{\text{DF}}^0) \leftarrow \text{RandomInit}(\theta_{\text{D}}^0, \theta_{\text{E}}^0, \theta_{\text{DF}}^0)$
- 8: $n_e = 0$
- 9: **while** ($\neg$ early-stopping and $n_e \leq N_{\text{epochs}}$) **do**
- 10: **for** $k = 1 : N_{nb}$ **do**
- 11: Sample a batch $(M^{\text{batch}}, N^{\text{batch}}, S_{N_A}^{\text{batch}}) \subset (M^{\text{train}}, N^{\text{train}}, S_{N_A}^{\text{train}})$
- 12: **for** $t = 1 : N_t$ **do**
- 13: $S_{N'}^{\text{batch}, t} \leftarrow \tilde{\mathbb{V}}_{M^{\text{batch}, t}}^T \cdot S_{N_A}^{\text{batch}, t} \in \mathbb{R}^{N_b \times N'}$
- 14: $S_{N'}^{\text{batch}, t} \leftarrow \text{reshape}(S_{N'}^{\text{batch}, t}) \in \mathbb{R}^{N_b \times \sqrt{N'} \times \sqrt{N'}}$
- 15: $z^{(n_e N_{nb} + k), t} \leftarrow R_{\rho}(\Psi'_n(\theta_{\text{E}}^{n_e N_{nb} + k}))(S_{N'}^{\text{batch}, t})$
- 16: $\hat{S}_{N'}^{\text{batch}, t} \leftarrow R_{\rho}(\Psi_{N'}(\theta_{\text{D}}^{n_e N_{nb} + k}) \odot \Phi_n(\theta_{\text{DF}}^{n_e N_{nb} + k}))(M^{\text{batch}}, N^{\text{batch}}, t)$
- 17: $\hat{S}_{N'}^{\text{batch}, t} \leftarrow \text{reshape}(\hat{S}_{N'}^{\text{batch}, t}) \in \mathbb{R}^{N_b \times N'}$
- 18: **end for**
- 19: Accumulate loss $\mathcal{L}_{\text{DOD-DL-ROM}}$ from Subsection (3.2.2) on $(M^{\text{batch}}, N^{\text{batch}}, S_{N_A}^{\text{batch}})$
- 20: $\theta^{n_e N_{nb} + k + 1} = \text{AdamW}(\eta^0, \nabla_{\theta} \mathcal{J}, \theta^{n_e N_{nb} + k})$
- 21: $\eta^{n_e + 1} \leftarrow \text{CAWR}(\mathcal{L}_{\text{DOD-DL-ROM}})$
- 22: **end for**
- 23: Repeat instructions 9–24 on $(M^{\text{val}}, N^{\text{val}}, S_{N_A}^{\text{val}})$ with the updated weights $\theta^{n_e N_{nb} + k + 1}$
- 24: Accumulate loss $\mathcal{L}_{\text{DOD-DL-ROM}}$ from Subsection (3.2.2) on $(M^{\text{val}}, N^{\text{val}}, S_{N_A}^{\text{val}})$
- 25: $n_e = n_e + 1$
- 26: **end while**

5.1 Two Visual Presentations

In the following, we want to show the reader two advection-reaction-diffusion equations, which meet the criterion of Assumption (1) in a sense; both have significantly faster KnW decay for fixed parameter $(\mu, t) \in \Theta \times [0, T]$.

5.1.1 Example 1: Wind from a Hole in a complex domain

We want to start with a simple problem on a relatively complex domain; a rotating wind from a hole. To circumvent the CFL-barrier, we impose the time domain to $T = 0.009$. Consider a

polygonal domain $\Omega \subset (0, 1)^2$ with two circular holes (see Figure 5.1). Let $u^{\mu, \nu}$ be called a strong solution to the problem, if for each $\mu \in (\pi + 0.5, 2\pi)$ and $\nu \in (0.01, 0.3)$ we have

$$\begin{cases} \partial_t u^{\mu, \nu}(x, t) + b(\mu) \cdot \nabla u^{\mu, \nu}(x, t) - \nabla \cdot (\kappa(x; \nu) \nabla u^{\mu, \nu}(x, t)) + \alpha u^{\mu, \nu}(x, t) = 0, & (x, t) \in \Omega \times (0, T], \\ u^{\mu, \nu}(x, t) = 3, & (x, t) \in \Gamma_D \times (0, T], \\ (\kappa(x; \nu) \nabla u^{\mu, \nu})(x, t) \cdot n(x) = 0, & (x, t) \in \Gamma_N \times (0, T], \\ u^{\mu, \nu}(x, 0) = 0, & x \in \Omega, \end{cases}$$

where the advection, diffusion and reaction terms are given by

$$b(\mu) = \begin{pmatrix} 40 \cos(\mu) \\ 40 \sin(\mu) \end{pmatrix}, \quad \kappa(x; \nu) = \nu(1 - x_1) + x_1, \quad \alpha = 10^{-1}.$$

Here, Γ_D denotes the inner boundary of the small circular hole centered at $(0.65, 0.35)$ with radius 0.05, on which Dirichlet boundary conditions are imposed, while Γ_N denotes the outer boundary and the larger hole centered at $(0.35, 0.65)$ with radius 0.10, on which homogeneous Neumann boundary conditions are imposed.

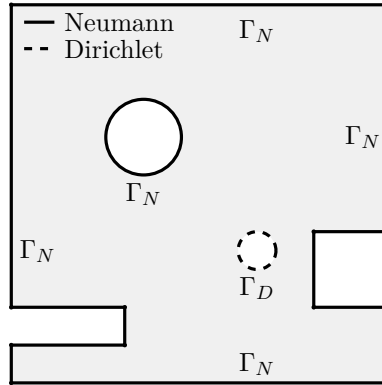
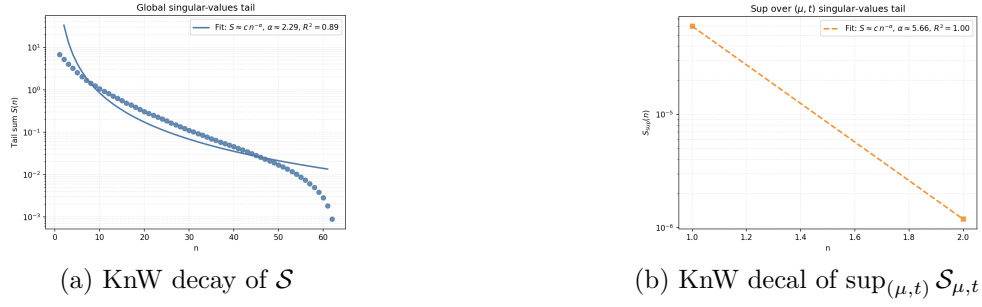


Figure 5.1: Polygonal subdomain of $(0, 1)^2$ with Dirichlet and Neumann boundaries.

The training is achieved with $N_{\text{epochs}} = 300$ and 8 restarts per model, meaning for each instance of a separate trainer, thus including the DOD (that uses 1000 epochs instead) by default as well, and patience of 40. Furthermore, we attain $N_{s_1}, N_{s_2} = 30$ and $N_t = 30$ for the sample sizes. The discretizer here is the **gmsh**-discretizer collecting a size of $N_h = 6376$ for the FOM results.

As for the reduction, we set up $n = 2 < 2(p + q + 1) + 3 = 9$, to reasonably satisfy Assumption (4), and $N_A = 64$ as the pre-reduction via POD to make the DOD easier to train. This achieves a retaining of about $\approx 98\%$ of the energy for the problem. Furthermore, we use $N = 32$ and $N' = 8$ to achieve a noticeable difference in reduction, and present the adaptive method as one, which may substitute or even overcome the POD. Assumption (1) is only used for the conditional result of Proposition (4.15), whose first part is true already if Assumption (5) is met, specifically if $N' < N$. To infer this, we can simply look at the KnW decays of both manifolds specified there. We can see in Figure (5.3), that indeed the solution manifold exhibits significantly slower decay than the submanifold for fixed (μ, t) . Note, that in order to guarantee comparability, we employ a separate way of gathering samples from Θ' to get a similar amount for the POD reduction, as the POD had in regards to the global solution manifold.

Figure 5.2: Linear Kolmogorov n -width decay of Example 1

Finally, this leads us to the discussion regarding the hyperparameters. The averaging parameter for the POD-DL-ROM and the DOD-DL-ROM is chosen in unison as $\omega = 0.6$, since this appears to provide the most reasonable results. Instead of providing reasonable data to show this behavior, we refer to (Fresca et al., 2020, p. 21–22), whose authors have already shown, that equidistant error fitting proves the most rentable. The architecture of models for the following visual presentation given in Figure (5.3) and (5.4) can be found in the Table (5.1) below. Here, we define for the DOD the layer composition of one of N' root modules as given in Definition (3.5), and the Conv2d-networks can be understood as concatenations of two-dimensional convolutional operators with hidden channel switch, symmetric kernel size "k", stride "s" and padding "p". The DFNN is only a straight forward implementation of the respective layer size list connecting in a dense manner.

Model	DOD	Encoder	Decoder	DFNN
POD-DL-ROM	-	Conv2d(1 $\rightarrow$ 8, k3, s1, p1) Conv2d(8 $\rightarrow$ 16, k3, s2, p1)	Conv2dT(16 $\rightarrow$ 8, k3, s2, p1) Conv2d(8 $\rightarrow$ 1, k3, s1, p1)	[16, 8, 4]
DOD-DL-ROM	[128, 64]	Conv2d(1 $\rightarrow$ 8, k3, s2, p1)	Conv2dT(8 $\rightarrow$ 1, k3, s2, p1)	[16, 8, 4]
DOD+DFNN	[128, 64]	-	-	[16, 8, 4]

Table 5.1: Model architectures in Example 1.

As discussed in the introduction, Reduced Order Models are mainly of interest, when they can significantly cut down execution time with an acceptable error rate. For the purpose of the analysis of such speed up, we want to compare the execution time of these architectures in regards to their number of active weights, and the natural computation time of a Full Order Model.

Model	#parameters	training time	forward time [ms]	speed-up
Full Order Model(CPU)	-	-	220.99	1
POD-DL-ROM(GPU)	91678	10 m	87.46	0.39
DOD-DL-ROM(GPU)	107847	15 + 8 m	108.35	0.49
DOD+DFNN(GPU)	104226	15 + 7 m	100.17	0.45

Table 5.2: Runtime comparison.

Qualitatively, one can observe, that all three algorithms seem to appreciate the physics of the system to a noticeable degree, but also, that the newly proposed methods provide worse results for this example in general. Even though the reduced basis provided by the DOD should in theory

show more adaptive behavior, this is not what we see in the figures so far. To be fair, the dimension N used for the POD reduced basis is high enough in this problem, that most of the energy can be preserved without the need for an adaptive basis. This quality can be seen nicely in Figure (5.2), where we can observe, that already around $N \approx 30$ should suffice to describe this example. Additionally to that, we can see in Table (5.2), that indeed both DOD-based alternatives provide a significantly worse speed up and also cost more time for training.

Hence, the first example can be seen as a pathological one, in a case, where the KnW decay *is* slower for the whole manifold, but not *slow enough*.

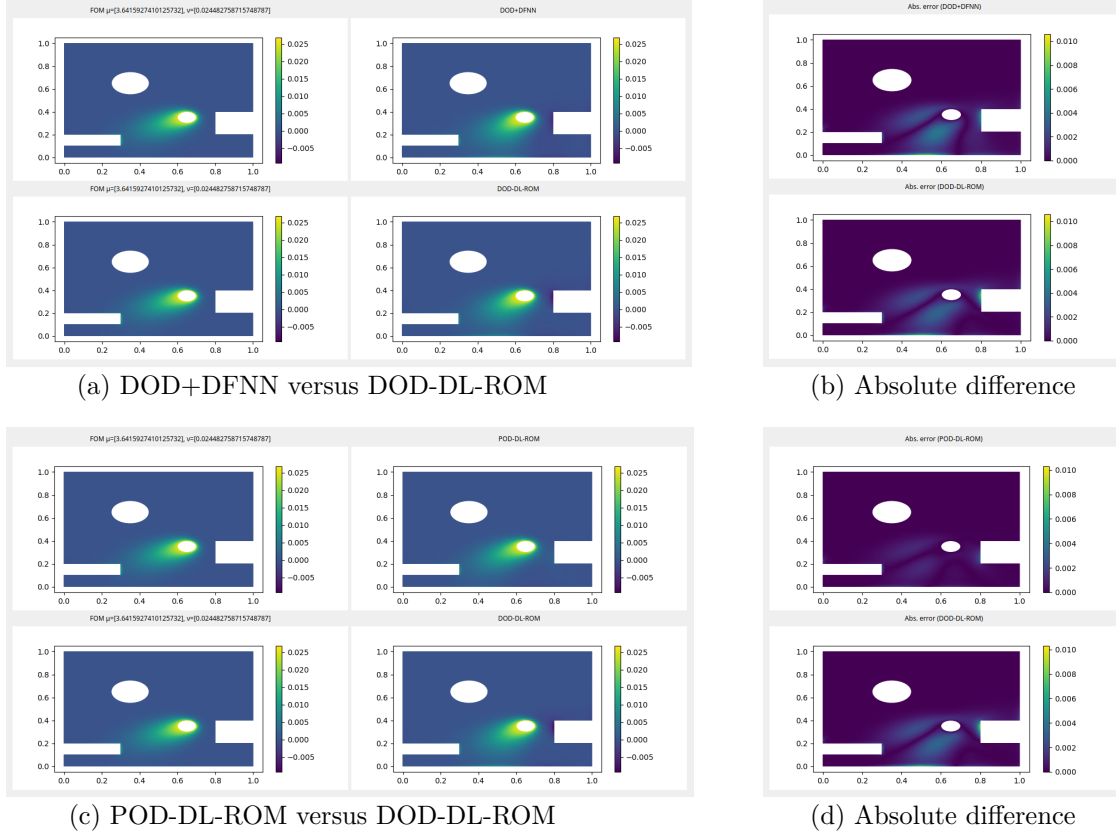


Figure 5.3: Comparison of ROMs for $\mu \approx 3.64$, $\nu \approx 2.4 \cdot 10^{-2}$ and $t = 8.4 \cdot 10^{-3}$.

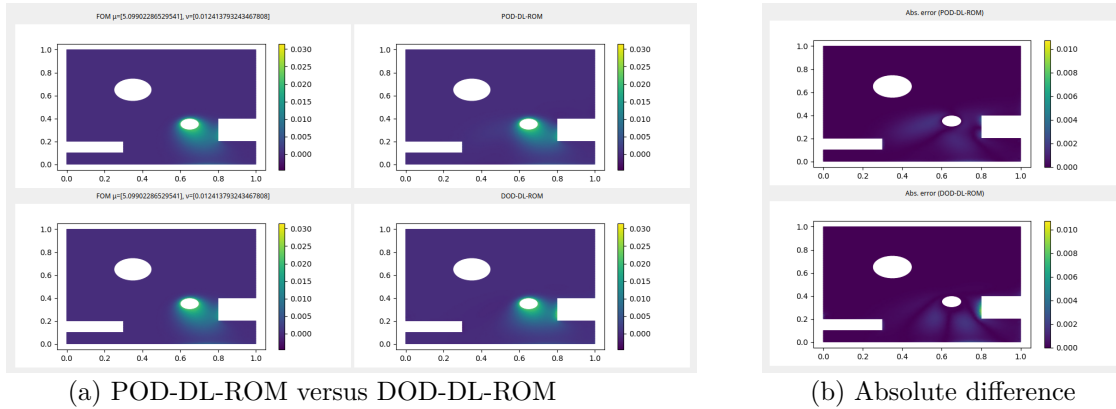


Figure 5.4: Comparison of DL-ROMs for $\mu \approx 5.09$, $\nu \approx 1.2 \cdot 10^{-2}$ and $t = 7.2 \cdot 10^{-3}$.

The main result of this thesis however, is the comparative analysis of both architectures with regard to the number of weights for a fixed relative error $\mathcal{E}_R$. For this, one can simply plot the growing complexity (in exponential terms) with regard to the development of the relative error for each model. As already discussed, we notice the general inferiority of both DOD-based approaches, but on a more hopeful note, can also infer the potentially better development of the DOD-DL-ROM for very low relative errors and high complexity as seen in the slopes of Figure (5.5). The existence of such a crosspoint outside the figures bound intuitively makes sense, as the bottleneck for the feasibility of the DOD-DL-ROM is mostly given by

$$\frac{n}{s-1} > \frac{\log(N_h)}{\log(\varepsilon^{-1})},$$

from Proposition (4.15) for any fixed relative error tolerance $\varepsilon > 0$. Obviously, this is satisfied most likely for very small errors and containable N_h , but since s is not known, one can not be sure how small the right-hand side needs to be. Note, that the proposition uses $N_h = N_A$ and thus the actual bottleneck might be a bit less tight, because the complexity bound is only imposed upon the neural network parts of the model. If the earlier results of Theorem (4.13) would be expanded to situations of $N_A < N_h$, one might potentially achieve that.

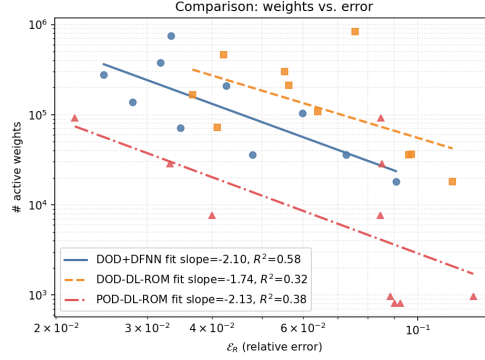


Figure 5.5: Comparison of active weights between ROMs

5.1.2 Example 2: Wind Generation in a plain domain

For the second example, we will look at a much simpler square domain $\Omega = (0,1)^2$ with last time point $T = 1$. Let $u^{\mu,\nu}$ be called a strong solution to the problem, if for some choice of $\mu = (\mu_1, \mu_2, \mu_3) \in (0.3, 0.7)^3$ and $\nu \in (0.002, 0.005)$ we have

$$\begin{cases} \partial_t u^{\mu,\nu}(x, t) + b(\mu_3) \cdot \nabla u^{\mu,\nu}(x, t) - \nu \Delta u^{\mu,\nu}(x, t) = f(x; \mu), & (x, t) \in \Omega \times (0, T], \\ (\nu \nabla u^{\mu,\nu})(x, t) \cdot n(x) = 0, & (x, t) \in \partial\Omega \times (0, T], \\ u^{\mu,\nu}(x, 0) = 0, & x \in \Omega, \end{cases}$$

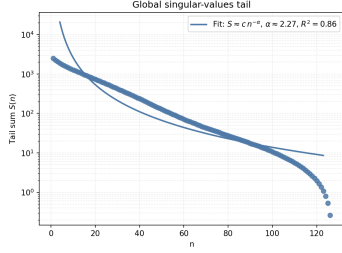
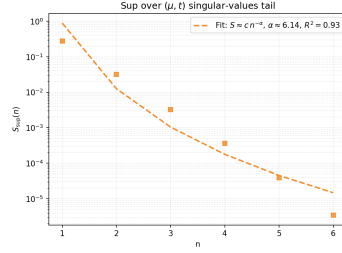
where the advection and right-hand side terms are given by

$$b(\mu_3) = \begin{pmatrix} \cos\left(\frac{\pi}{100\mu_3}\right) \\ \sin\left(\frac{\pi}{100\mu_3}\right) \end{pmatrix}, \quad f(x; \mu) = 10 \exp\left(-\frac{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}{0.07^2}\right).$$

We use the same training parameters, as above. However, the discretizer for this example is the discretizer inherent to the unit square domain, producing $N_h = 2381$ degrees of freedom.

Furthermore, we employ most of the same training hyperparameters, such as N_{epochs} and N_t and N_{s_1}, N_{s_2} exactly as in the previous example. Significant changes are only the reduction dimension, as the problem here has much more obvious dependency on μ , and hence slower KnW decay as can be seen in Figure (5.6).

Therefore, for the reduction we choose $n = 2 < 2(p + q + 1) + 3 = 13$, $N' = 8$, $N = 64$ and $N_A = 128$.

(a) KnW decay of $\mathcal{S}$ (b) KnW decal of $\sup_{(\mu,t)} \mathcal{S}_{\mu,t}$ Figure 5.6: Linear Kolmogorov n -width decay of Example 2

Since we can clearly see, that we already had non-significant speed-up in the first example and the FOM there had more dofs to solve than here, we will skip the table for it, and only focus on architecture as well as the complexity comparison. Note, that the `test.py` in the repository produces timings for any possible choice of an architecture.

Regarding the specific build of the models, the reader can look up Table (5.3), which is written in the same convention as before.

Model	DOD	Encoder	Decoder	DFNN
POD-DL-ROM	-	Conv2d(1 $\rightarrow$ 8, $k3$, $s1$, $p1$) Conv2d(8 $\rightarrow$ 16, $k3$, $s2$, $p1$)	Conv2dT(16 $\rightarrow$ 8, $k3$, $s2$, $p1$) Conv2dT(8 $\rightarrow$ 1, $k3$, $s1$, $p1$)	[16, 8, 4]
DOD-DL-ROM	[256, 128]	Conv2d(1 $\rightarrow$ 8, $k3$, $s2$, $p1$)	Conv2dT(8 $\rightarrow$ 1, $k3$, $s2$, $p1$)	[16, 8, 4]
DOD+DFNN	[254, 128]	-	-	[16, 8]

Table 5.3: Model architectures in Example 2.

These models then produce the following comparative results as displayed in Figures (5.7) and (5.8). In this case, we get suggestive results for the advantage of the DOD-DL-ROM above the POD-DL-ROM. The significant difference in the coefficient dimensions $N' = 8 < 64 = N$ sets up an optimistic interpretation, that even with such a relatively low dimension, the DOD has an edge over the POD, as it can capture the underlying solution manifold more adaptively. Additionally, the Autoencoder architecture might be favorable for this specific translation invariant problem, since the simple dense layer based neural network in the DOD+DFNN need to work harder to attain the details of the problem for each parameter choice independently and the convolutional kernel can be applied to all of them better. Notice though, that here the convolutional Autoencoder does not work directly on the "picture" but instead on the POD modes, what makes the explanation more of an optimistic guess.

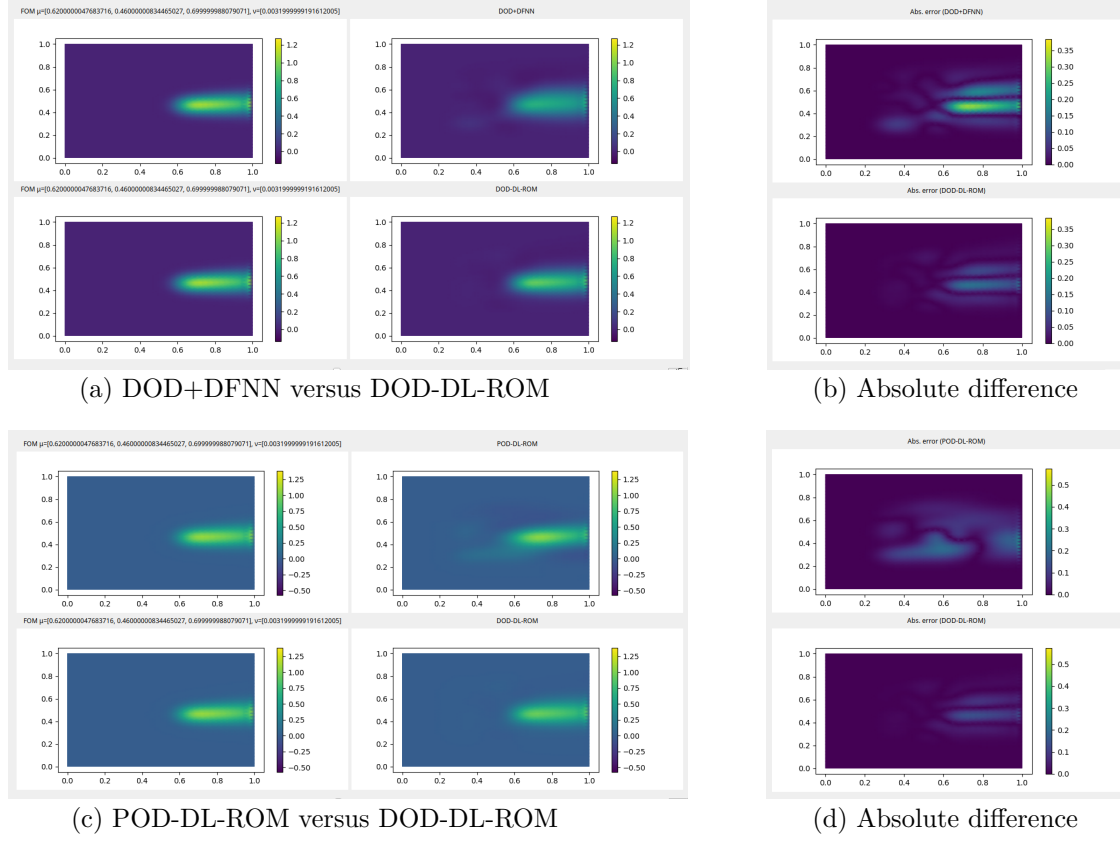


Figure 5.7: Comparison of ROMs for $\mu \approx (0.62, 0.46, 0.7)$, $\nu \approx 3.1 \cdot 10^{-3}$ and $t = 0.5$.

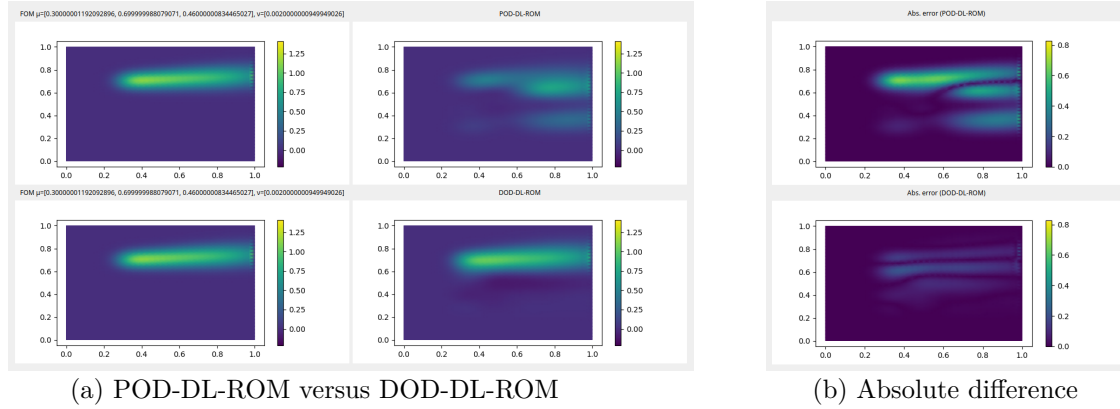


Figure 5.8: Comparison of DL-ROMs for $\mu \approx (0.3, 0.7, 0.46)$, $\nu \approx 2 \cdot 10^{-3}$ and $t = 0.77$.

An interesting detail in the absolute differences can be observed in these figures: The DOD-based models provide errors, which propagate in parallel to the wind direction, hinting on the intrinsic potential of the DOD to appreciate the physical nature of the problem within its modes and only making mistakes, in a shifted regard. Finally, in a concluding manner, the Figure (5.9) shows the relation between relative errors $\mathcal{E}_R$ and the complexities of their respective models as has already been done in the first example.

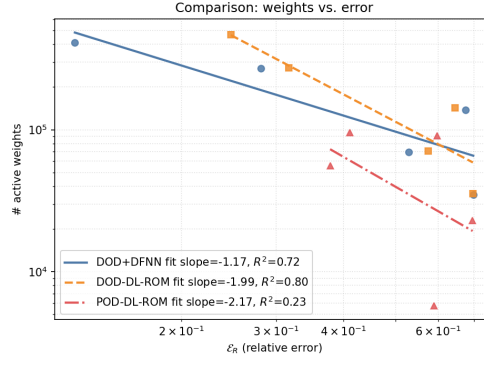


Figure 5.9: Comparison of active weights between ROMs

5.2 An Error Decomposition Example

The earlier examples both were obstructed by the fact, that N_h is large and thus the direct exposition of the experimental results can not be nicely compared to those of the theory provided in Chapter (4). As a case study, an example introducing $N_h = N_A$ might be of interest for the error decomposition formulas, which state, that both errors can be unilaterally separated in three forms. Notice, that this is not true if $N_h \neq N_A$, since then the DOD performs an additional error with regard to the pre-reduction. However, such a result is more didactic in nature, since the speed up generated by the use of such a reduced order model would be virtually non-existent.

For the following, we remind the reader of Theorems (4.3) and (4.10), that provide us with upper bounds for the relative error for DOD-based and POD-based ROMs, and Theorems (4.4) and (4.11), which do the same for lower bounds. As such, in the case of using a POD-DL-ROM and a DOD-DL-ROM, we can provide a relatively transferable framework of comparison. Reduction to the coefficient dimensions N and N' can then be nicely examined in a context of minimal and maximal error. The reason to use both DL-ROMs is to have the neural network error $\mathcal{E}_{NN}$ be comparable. The example, we will be providing for this is the upcoming heat equation with initial condition.

Let $u^{\mu,\nu}$ be called a strong solution to the problem, if for some choice of $\mu \in (0.2, 0.8)$ and $\nu \in (10^{-4}, 10^{-2})$ we have

$$\begin{cases} \partial_t u^{\mu,\nu}(x, t) - \partial_x(\nu \partial_x u^{\mu,\nu}(x, t)) = 0, & (x, t) \in (0, 1) \times (0, T], \\ u^{\mu,\nu}(0, t) = 0, \quad u^{\mu,\nu}(1, t) = 0, & t \in (0, T], \\ u^{\mu,\nu}(x, 0) = u_0^\mu(x), & x \in (0, 1), \end{cases}$$

where the initial condition is given by

$$u_0^\mu(x) = \frac{1}{2} \left(1 + \tanh\left(\frac{x-\mu}{\lambda}\right) \right), \quad \lambda = 10^{-2}.$$

In order to keep this section short and concise, we provide a short run-down of the relevant reduction values and two instances of the comparison between both DL-ROMs; the FOM size is $N_h = N_A = 101$ to keep training as light as possible, and we have set $n = 4$, as the parameter spaces both have only one dimension, and it is reasonable to assume that the latent dynamics are of low complexity here.

To stay true to our main framework of our problems, we quickly show the KnW decay in Figure (5.10).

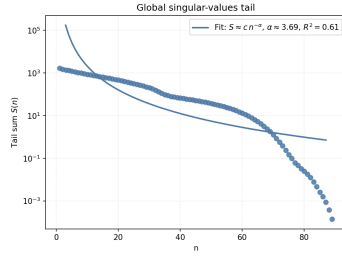
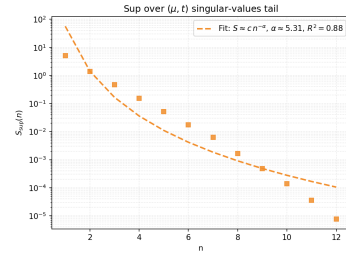

 (a) KnW decay of $\mathcal{S}$

 (b) KnW decal of $\sup_{(\mu, t)} \mathcal{S}_{\mu, t}$

 Figure 5.10: Linear Kolmogorov n -width decay of Example 3

Moreover, Figure (5.11) shows two results of the DOD-DL-ROM in an overlay with the actual FOM solution and its point-wise difference.

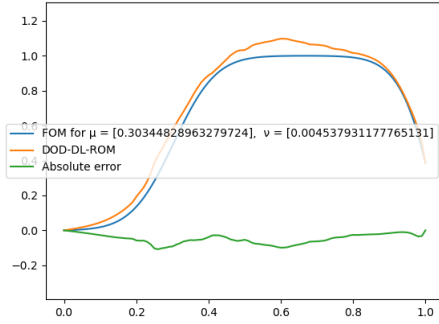
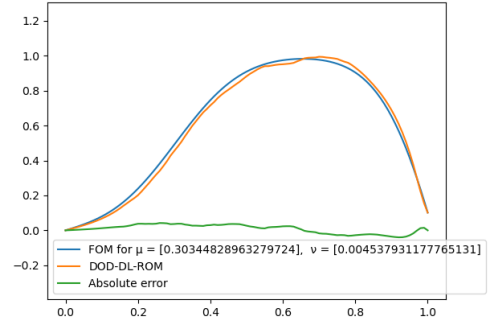
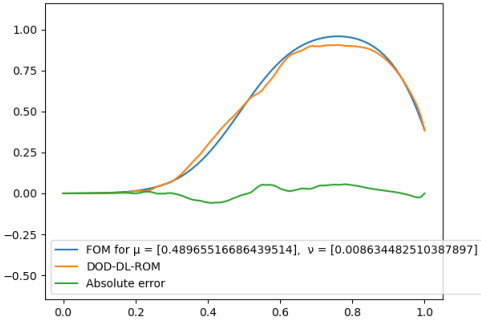
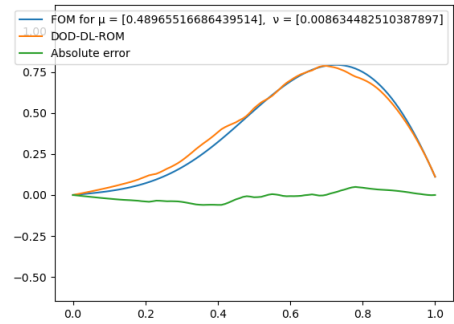

 (a) $t = 1$

 (b) $t = 2.4$

 (c) $t = 1$

 (d) $t = 2.4$

Figure 5.11: Two solutions of the DOD-DL-ROM

It is directly apparent, that the decay here again does not really satisfy Assumption (1), but at least shows a clear superiority in case the geometric and time parameters are kept in place.

Connecting to the discussion in the head of this section, we notice in Figure (5.12), that the decay of both sample errors introduced in Theorems (4.3) and (4.10) is of different behavior, since the DOD's precision for each $(\mu, t) \in \mathcal{P}_1 \times \mathcal{T}$ only depends on the size of the training data for this choice $\mathcal{P}_2$. This makes the error fall onto the line of $N_{s_2}^{-1/4}$ instead, hence greatly inducing the need for more sampling, since in general $N_{s_1} > 1$. Note, that numerically speaking, we cannot truly predict the actual errors as an integral, and we thus simply cheat by using the quadrature regarding

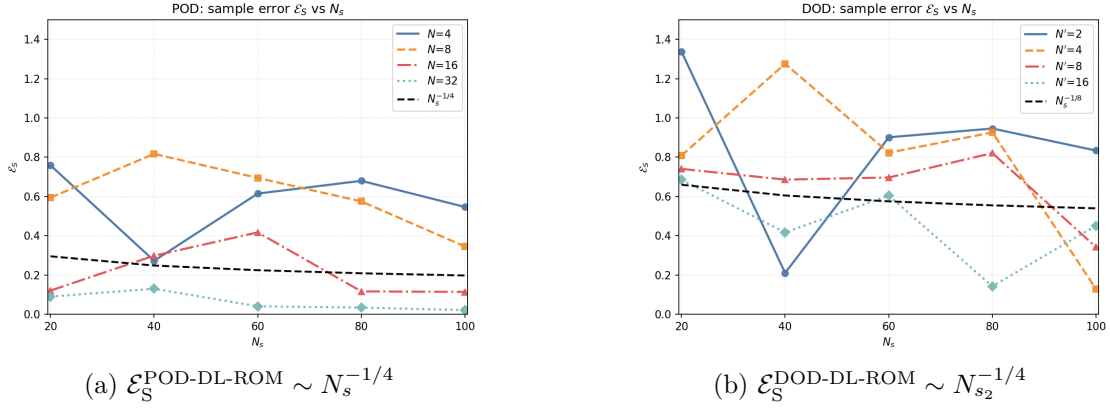


Figure 5.12: Dependency on sample size for DOD-DL-ROM and POD-DL-ROM

the full size of the data we collected a priori. This produces a sample with $N_s = 900$ and $N_t = 30$ to predict the integrals, which should suffice for a general qualitative description.

Attending to this reasoning, we might conclude, that therefore the size of N_{s1} and N_t does not matter, but what we conveniently left out is that the error E_S is an average over the whole space $\Theta \times [0, T]$, and thus a good sampling here is necessary as well. The exact nature of this dependency is not the main focus, since the average could be estimated in a similar sense, but the order of convergence is determined by the size of the second parameter if seen as one over the worst mistake.

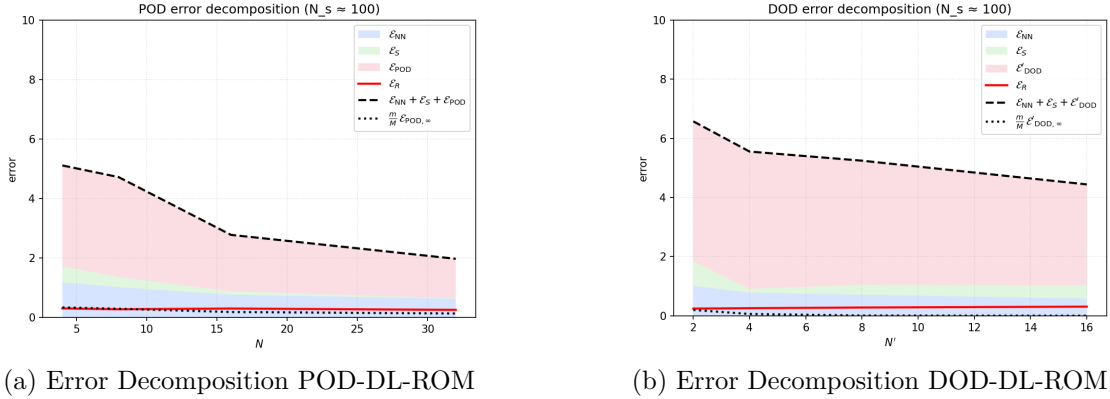


Figure 5.13: Comparison of Lower and Upper Error Bounds

Lastly, we point the reader to Figure (5.13), which is the main result of this thesis, as it shows the specific decomposition of the DOD-based time-dependent ROMs in a side by side to the one of the POD, which is inspired by (Brivio et al., 2023), where the authors have shown the same decomposition. For the discussion, we can easily remark, that the error decomposition of both methods is highly dominated by the error contributions of $\mathcal{E}_{DOD}$ and $\mathcal{E}_{POD}$ respectively. Furthermore, the DOD-based method produces similar relative errors compared to the POD-based one, even though the maximal error is clearly larger, since the DOD-type one includes the mistake made by the network and the POD is already naturally the best fit. However, the interesting and hopeful interpretation of these graphs can also be, that the lower bound is significantly smaller than the one of the POD, thanks to the KnW decays.

This, as shown by the theory, can only mean something in the case that the approximation capabilities of the choice of architecture we deliberately made here, is enough to fit the inner DOD

module to the optimal DOD function, that might not even have enough smoothness to be accurately approximated by a network of containable size.

Chapter 6

Conclusion

In this final chapter, the focus will be to briefly review the results, and present potential further aspect of scientific research in that area. For this, we want to lastly separated the portion into two sections, answering both concluding questions; what was achieved in this thesis and what might future developments be.

6.1 Review

This thesis introduced in Chapter (3) the newly proposed adaptive Deep Orthogonal Decomposition based Deep Learning Reduced Order Model (DOD-DL-ROM) based on the founding idea of (Franco et al., 2024) and a crossing with the POD-DL-ROM given by (Fresca & Manzoni, 2022), and concluded in its theoretical framework in Chapter (4) of approximation theoretic statements, that the Deep Orthogonal Decomposition based Reduced Order Model might pose a viable alternative to the Proper Orthogonal Decomposition based one, as long as the aimed upon error tolerance ε is significantly low, and enough regularity onto the optimal Deep Orthogonal Decomposition can be implied. The bottleneck criterion for this is most significantly given by

$$\frac{n}{s-1} > \frac{\log(N_h)}{\log(\varepsilon^{-1})},$$

where s denotes the implied regularity of the optimal non-linear embedding function between the coefficient space and the latent dynamics.

We reached this conclusion through a rigorous application of results about arbitrary proximity to the optimal projection derived from the paper (Franco et al., 2024) in a context of approximation theory lifted, in notation and content, from (Gühring et al., 2019). Furthermore, we could provide a distinct error decomposition upper and lower bound for any such Deep Orthogonal Decomposition based model, which we went on to visualize in the last of three examples, where the other two served the purpose of showing one instance of potential superiority of the method over the legacy Proper Orthogonal data-driven model, and another of its limitations.

6.2 Research

In the following, a few different hot spots of potential sources of interest will be highlighted, even though the exact nature of their respective potential has not been investigated. For this, we will list a few talking points regarding the proposed DOD-DL-ROM and any other adaptations of the newly time-dependent DOD Definition (3.5).

- In Assumption (6), another imposition on the optimal DOD map has been made, and further even $W^{m,\infty}$ -smoothness has been conditioned. There has been made no effort to show or discuss neither an example nor a criterion to guarantee such regularity yet.
- For the Upper Complexity Bounds, we have assumed, that $N_h = N_A$ to make the discussion regarding pre-reduction on the solution manifold less cumbersome, but this could be systematically reworked in a broader class of problems, if the KnW behavior can be inherited through such a linear pre-reduction.
- A simple comparison to a more direct approach like the older DL-ROM proposed by (Fresca et al., 2020) in terms of complexity.
- In the same sense as the POD-DeepONet could be derived from the statement of the POD, there might be a possibility to undertake a generalization of the DOD projection to DeepONets in order to consider features slowing the KnW in a less strictly "parametric" way, leading to an adaptation of function space influence. In such a way, the approach could be a candidate to overcome the *curse of dimensionality*, if the features could be sampled in a dimension-independent way.
- As it stands, the DOD uses $t \in [0, T]$ as a simple parameter and thus introduces time by injection as a factor of slow KnW decay. This might be not necessary in some cases. Hence, an alternative formulation might be of interest.

Acknowledgement

We hereby acknowledge and thank Björn Balkow and Felix Tümpner for review and help with notation.

Bibliography

- Alt, H. W. (2016, July). *Linear functional analysis*. Springer London. <https://doi.org/10.1007/978-1-4471-7280-2>
- Barraut, M., Maday, Y., Nguyen, N. C., & Patera, A. T. (2004). An 'empirical interpolation' method: Application to efficient reduced-basis discretization of partial differential equations. *Comptes Rendus Mathématique*, 339(9), 667–672. <https://doi.org/10.1016/j.crma.2004.08.006>
- Bauer, F. L., & Fike, C. T. (1960). Norms and exclusion theorems. *Numerische Mathematik*, 2, 137–141. <https://api.semanticscholar.org/CorpusID:121278235>
- Berman, J., & Peherstorfer, B. (2024). Colora: Continuous low-rank adaptation for reduced implicit neural modeling of parameterized partial differential equations. <https://arxiv.org/abs/2402.14646>
- Brezis, H. (2011). *Functional analysis, sobolev spaces and partial differential equations* (1st ed.). Springer New York. <https://doi.org/10.1007/978-0-387-70914-7>
- Brezis, H., & Browder, F. (1998). Partial differential equations in the 20th century. *Advances in Mathematics*, 135(1), 76–144. <https://doi.org/https://doi.org/10.1006/aima.1997.1713>
- Brivio, S., Fresca, S., Franco, N. R., & Manzoni, A. (2023). Error estimates for pod-dl-roms: A deep learning framework for reduced order modeling of nonlinear parametrized pdes enhanced by proper orthogonal decomposition. <https://arxiv.org/abs/2305.04680>
- Buchfink, P., Glas, S., & Haasdonk, B. (2023). Approximation bounds for model reduction on polynomially mapped manifolds. *arXiv preprint arXiv:2312.00724*.
- Carlberg, K., Bou-Mosleh, C., & Farhat, C. (2011). Efficient non-linear model reduction via a least-squares petrov-galerkin projection and compressive tensor approximations. *International Journal for Numerical Methods in Engineering*, 86, 155–181. <https://doi.org/10.1002/nme.3050>
- Carlberg, K., Farhat, C., Cortial, J., & Amsallem, D. (2013). The gnat method for nonlinear model reduction: Effective implementation and application to computational fluid dynamics and turbulent flows. *Journal of Computational Physics*, 242, 623–647. <https://doi.org/10.1016/j.jcp.2013.02.028>
- Chaturantabut, S., & Sorensen, D. C. (2010). Nonlinear model reduction via discrete empirical interpolation. *SIAM Journal on Scientific Computing*, 32(5), 2737–2764. <https://doi.org/10.1137/090766498>
- Ciampiconi, L., Elwood, A., Leonardi, M., Mohamed, A., & Rozza, A. (2024). A survey and taxonomy of loss functions in machine learning. <https://arxiv.org/abs/2301.05579>
- Courant, R., & Hilbert, D. (1953). *Methods of mathematical physics, vol. i*. Interscience Publishers.
- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314. <https://doi.org/10.1007/BF02551274>
- Dacorogna, B. (2004, November). *Introduction to the calculus of variations*. <https://doi.org/10.1142/p361>

- Duchi, J., Hazan, E., & Singer, Y. Adaptive subgradient methods for online learning and stochastic optimization. In: *Proceedings of the 24th international conference on neural information processing systems. 1*. 2011, 257–264.
- Franco, N. R., Manzoni, A., & Zunino, P. (2023). A deep learning approach to reduced order modelling of parameter dependent partial differential equations [Published electronically November 16, 2022]. *Mathematics of Computation*, 92, 483–524. <https://doi.org/10.1090/mcom/3781>
- Franco, N. R., Manzoni, A., Zunino, P., & Hesthaven, J. S. (2024). Deep orthogonal decomposition: A continuously adaptive data-driven approach to model order reduction [arXiv:2404.18841]. <https://arxiv.org/abs/2404.18841>
- Fresca, S., Dede, L., & Manzoni, A. (2020). A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes. <https://arxiv.org/abs/2001.04001>
- Fresca, S., & Manzoni, A. (2022). Pod-dl-rom: Enhancing deep learning-based reduced order models for nonlinear parametrized pdes by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388, 114181. <https://doi.org/10.1016/j.cma.2021.114181>
- Geist, M., Petersen, P., Raslan, M., Schneider, R., & Kutyniok, G. (2020). Numerical solution of the parametric diffusion equation by deep neural networks. <https://arxiv.org/abs/2004.12131>
- Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations*. Johns Hopkins University Press. <https://doi.org/10.56021/9781421407944>
- Grepl, M. A., & Patera, A. T. (2005). A posteriori error bounds for reduced-basis approximations of parametrized parabolic partial differential equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 39, 157–181. <https://doi.org/10.1051/m2an:2005006>
- Gühring, I., Kutyniok, G., & Petersen, P. (2019). Error bounds for approximations with deep relu neural networks in $W^{s,p}$ norms. <https://arxiv.org/abs/1902.07896>
- Gühring, I., Raslan, M., & Kutyniok, G. (2020). Expressivity of deep neural networks. <https://arxiv.org/abs/2007.04759>
- Haasdonk, B., & Ohlberger, M. (2008). Reduced basis method for finite volume approximations of parametrized linear evolution equations. *ESAIM: Mathematical Modelling and Numerical Analysis*, 42, 277–302. <https://doi.org/10.1051/m2an:2008001>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- Heinonen, J. (2005). Lectures on lipschitz analysis [[Online; accessed 20-September-2018]].
- Hesthaven, J., Pagliantini, C., & Rozza, G. (2022). Reduced basis methods for time-dependent problems. *Acta Numerica*, 31, 265–345. <https://doi.org/10.1017/S0962492922000058>
- Holmes, P., Lumley, J. L., & Berkooz, G. (1996). *Turbulence, coherent structures, dynamical systems and symmetry*. Cambridge University Press.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257. [https://doi.org/https://doi.org/10.1016/0893-6080\(91\)90009-T](https://doi.org/https://doi.org/10.1016/0893-6080(91)90009-T)
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
- Kovachki, N., Lanthaler, S., & Mishra, S. (2021). On universal approximation and error bounds for fourier neural operators. <https://arxiv.org/abs/2107.07562>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. In F. Pereira, C. Burges, L. Bottou, & K. Weinberger (Eds.), *Advances in neural information processing systems* (Vol. 25). Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2012/file/c399862d3b9d6b76c8436e924a68c45b-Paper.pdf

- Kunisch, K., & Volkwein, S. (2001). Galerkin proper orthogonal decomposition methods for a general equation in fluid dynamics. *SIAM Journal on Numerical Analysis*, 40(2), 492–515. <https://doi.org/10.1137/S0036142900382612>
- Kuratowski, K., & Ryll-Nardzewski, C. (1965). A general theorem on selectors. *Bull. Acad. Polon. Sci. Sér. Sci. Math. Astronom. Phys.*, 13, 397–403.
- LeCun, Y., Boser, B., Denker, J. S., Henderson, D., Howard, R. E., Hubbard, W., & Jackel, L. D. (1989). Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4), 541–551. <https://doi.org/10.1162/neco.1989.1.4.541>
- Loshchilov, I., & Hutter, F. (2017). Sgdr: Stochastic gradient descent with warm restarts. *International Conference on Learning Representations (ICLR)*.
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*.
- Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via deepnet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3), 218–229. <https://doi.org/10.1038/s42256-021-00302-5>
- Lu, L., Meng, X., Cai, S., Mao, Z., Goswami, S., Zhang, Z., & Karniadakis, G. E. (2022). A comprehensive and fair comparison of two neural operators (with practical extensions) based on fair data. *Computer Methods in Applied Mechanics and Engineering*, 393, 114778. <https://doi.org/https://doi.org/10.1016/j.cma.2022.114778>
- Lumley, J. L. (1967). The structure of inhomogeneous turbulent flows. In A. M. Yaglom & V. I. Tatarsky (Eds.), *Atmospheric turbulence and radio wave propagation* (pp. 166–178). Nauka.
- Maday, Y., Patera, A. T., & Rovas, D. V. (2002). A blackbox reduced-basis output bound method for noncoercive linear problems. In *Proceedings of the 12th international conference on domain decomposition methods*. Domain Decomposition Press.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115–133. <https://doi.org/10.1007/BF02478259>
- Milk, R., Rave, S., & Saak, J. (2016). Pymor – generic algorithms and interfaces for model order reduction. *SIAM Journal on Scientific Computing*, 38(5), S194–S216. <https://doi.org/10.1137/15M1026614>
- Newey, W. K., & McFadden, D. (1994). Chapter 36 large sample estimation and hypothesis testing. Elsevier. [https://doi.org/https://doi.org/10.1016/S1573-4412\(05\)80005-4](https://doi.org/https://doi.org/10.1016/S1573-4412(05)80005-4)
- Noor, A. K. (1980). Recent advances in reduction methods for nonlinear problems. *Computers & Structures*, 12(1), 31–44. [https://doi.org/10.1016/0045-7949\(80\)90101-6](https://doi.org/10.1016/0045-7949(80)90101-6)
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. <https://arxiv.org/abs/1912.01703>
- Patera, A. T. (1984). A spectral element method for fluid dynamics: Laminar flow in a channel expansion. *Journal of Computational Physics*, 54(3), 468–488. [https://doi.org/10.1016/0021-9991\(84\)90128-1](https://doi.org/10.1016/0021-9991(84)90128-1)
- Peherstorfer, B., Willcox, K., & Gunzburger, M. (2016). Optimal model management for multifidelity monte carlo estimation. *SIAM Journal on Scientific Computing*, 38(5), A3163–A3194. <https://doi.org/10.1137/15M1046472>
- Prud’homme, C., Rovas, D. V., Veroy, K., Machiels, L., Maday, Y., Patera, A. T., & Turinici, G. (2002). Reliable real-time solution of parametrized partial differential equations: Reduced-basis output bound methods. *Journal of Fluids Engineering*, 124(1), 70–80. <https://doi.org/10.1115/1.1448332>

- Qian, N. On the momentum term in gradient descent learning algorithms. In: In *Neural networks: Tricks of the trade*. Springer. 1999, 195–200.
- Quarteroni, A., Manzoni, A., & Negri, F. (2015). *Reduced basis methods for partial differential equations*. Springer. <https://doi.org/10.1007/978-3-319-15431-2>
- Robbins, H., & Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. <https://doi.org/10.1037/h0042519>
- Rowley, C. W., Mezić, I., Bagheri, S., Schlatter, P., & Henningson, D. S. (2009). Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641, 115–127. <https://doi.org/10.1017/S0022112009992059>
- Rudin, W. (1976). *Principles of mathematical analysis* (3rd ed.). McGraw-Hill.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536.
- Schmid, P. J. (2010). Dynamic mode decomposition of numerical and experimental data. *Proceedings of the 61st Annual Meeting of the APS Division of Fluid Dynamics*.
- Siena, P., Girfoglio, M., & Rozza, G. (2022). An introduction to pod-greedy-galerkin reduced basis method. <https://arxiv.org/abs/2203.08532>
- Sirovich, L. (1987). Turbulence and the dynamics of coherent structures. part i: Coherent structures. *Journal of Fluid Mechanics*, 175, 419–427. <https://doi.org/10.1017/S0022112087000199>
- Smith, L. N. (2017). Cyclical learning rates for training neural networks. *2017 IEEE Winter Conference on Applications of Computer Vision (WACV)*, 464–472.
- Telgarsky, M. (2015). Representation benefits of deep feedforward networks. <https://arxiv.org/abs/1509.08101>
- Tieleman, T., & Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*.
- Yarotsky, D. (2017). Error bounds for approximations with deep relu networks. *Neural Networks*, 94, 103–114. <https://doi.org/https://doi.org/10.1016/j.neunet.2017.07.002>

Declaration of Academic Integrity

I hereby confirm that this thesis, entitled "*Deep Orthogonal Decomposition for surrogate modelling of time dependent PDEs*" is solely my own work and that I have used no sources or aids other than the ones stated. All passages in my thesis for which other sources, including electronic media, have been used, be it direct quotes or content references, have been acknowledged as such and the sources cited. I am aware that plagiarism is considered an act of deception which can result in sanction in accordance with the examination regulations.

(date, signature of student)

I consent to having my thesis cross-checked with other texts to identify possible similarities and to having it stored in a database for this purpose.

I confirm that I have not submitted the following thesis in part or whole as an examination paper before.

(date, signature of student)